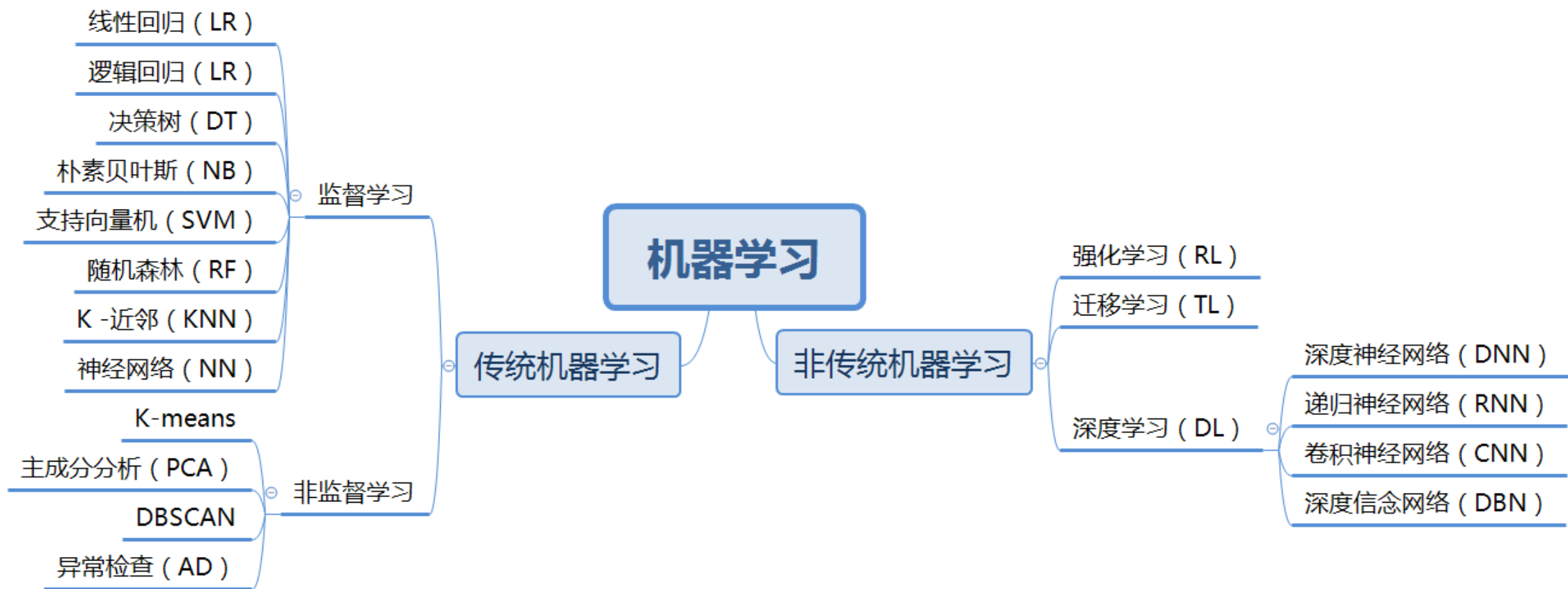


# 监 督 学 习

2025年2月

**TRANSWARP**  
星 环 科 技





# 目录 >

## CONTENTS

- 1 线性回归
- 2 逻辑回归
- 3 K 近邻
- 4 支持向量机
- 5 决策树
- 6 朴素贝叶斯



1  
chapter

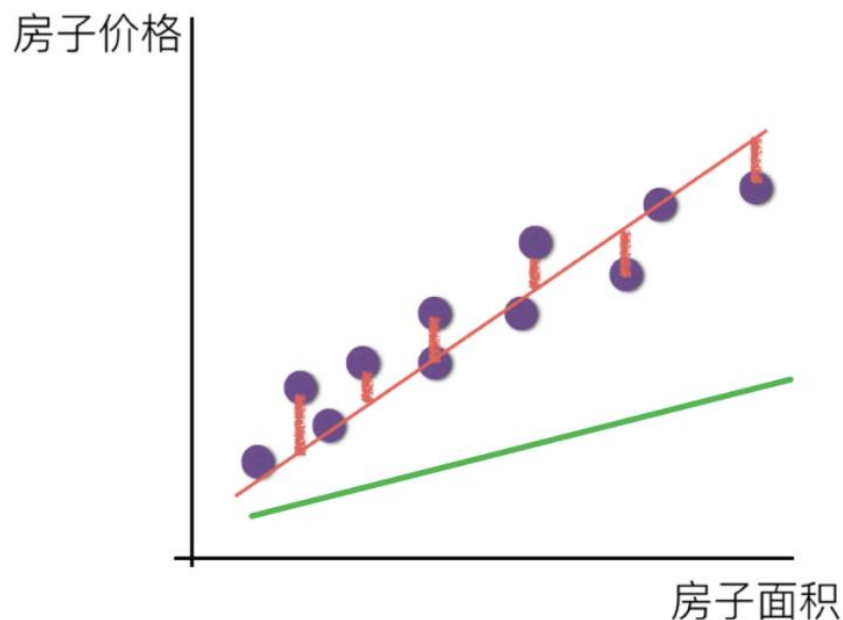
# 线性回归



**回归问题** 就是拟合 **输入变量 $X$** 与 **数值型**的 **目标变量 $y$** 之间的 **关系**，而线性回归就是假定了 $X$ 与 $Y$ 之间是线性关系，数学表示如下：

$$y = f(x_1, x_2, x_3, \dots) = w_1x_1 + w_2x_2 + w_3x_3 + \dots + b$$

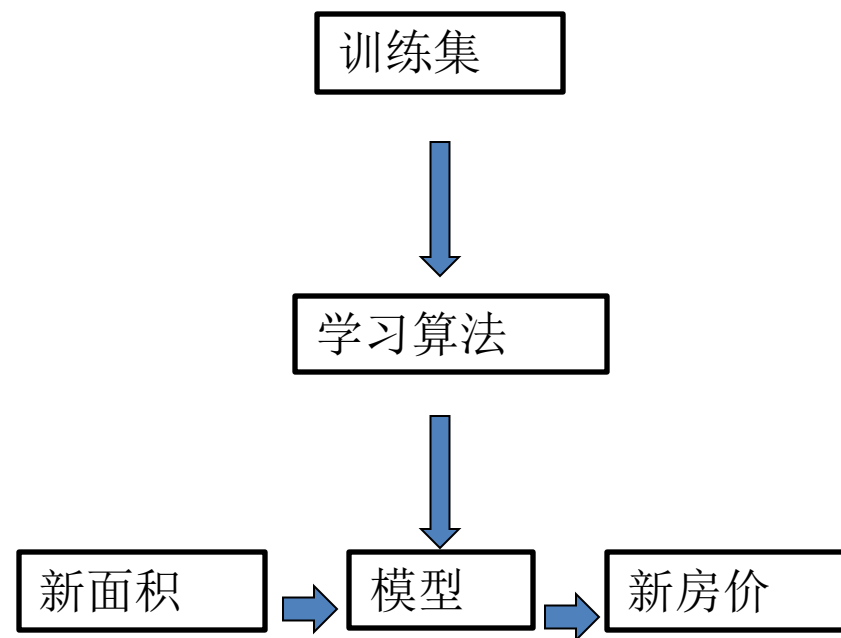
示例：如下图所示，我们可以通过绘制 $(x, y)$ 散点图的方式来查看 $x$ 和 $y$ 之间是否有线性关系，**回归模型的目标是寻找一条穿过这些散点的直线，让所有的点离这条直线的距离最近**。这条完美直线对应的参数就是我们要寻找的**回归模型参数**： $w_1, w_2, w_3, \dots, b$



一维输入特征的线性回归模型

请问，如果现在有一个房屋面积为55平，请问最终的租赁价格是多少比较合适？

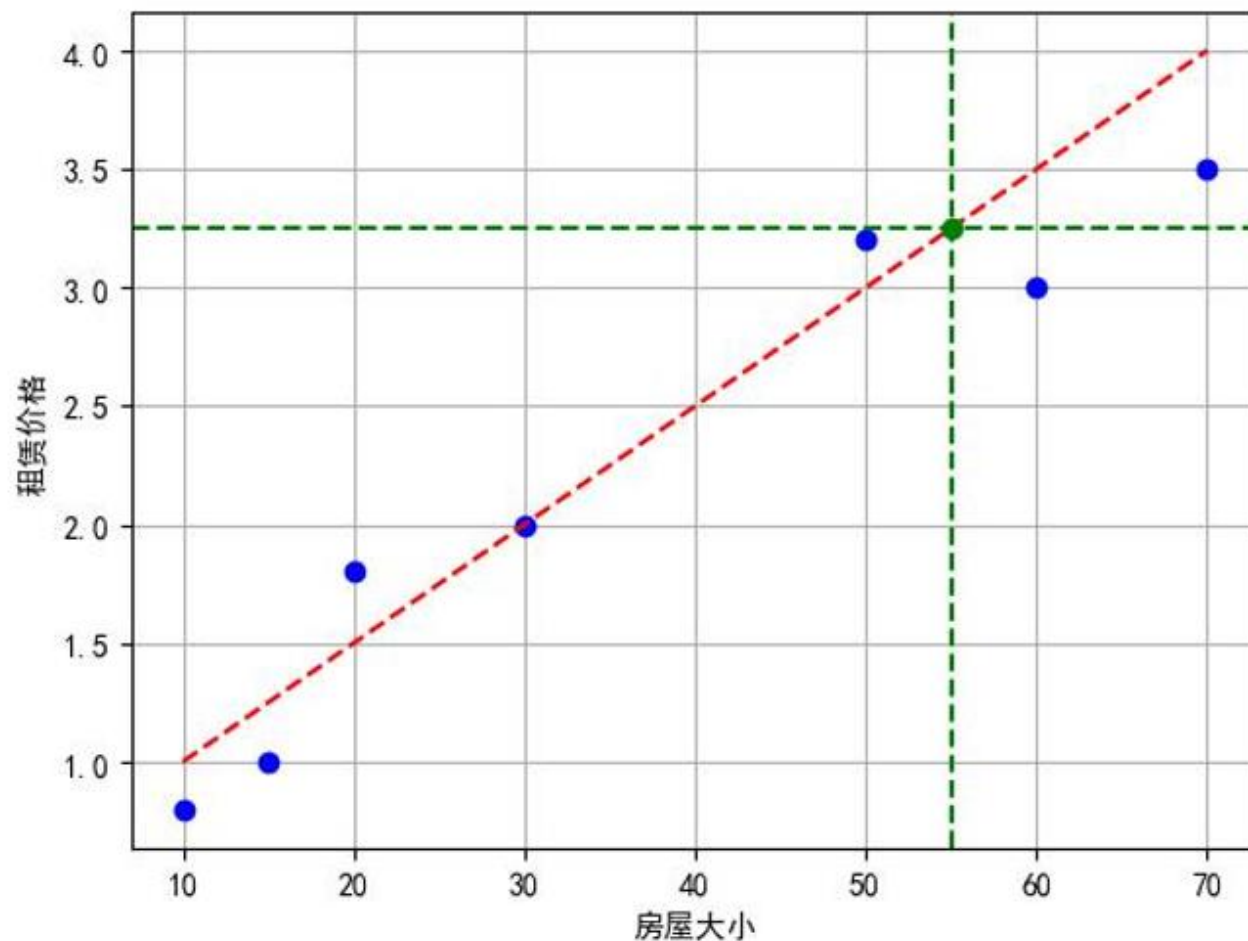
| 房屋面积(m <sup>2</sup> ) | 租赁价格(1000¥) |
|-----------------------|-------------|
| 10                    | 0.8         |
| 15                    | 1           |
| 20                    | 1.8         |
| 30                    | 2           |
| 50                    | 3.2         |
| 60                    | 3           |
| 60                    | 3.1         |
| 70                    | 3.5         |



$$H(x) = \theta_0 + \theta_1 x_1$$

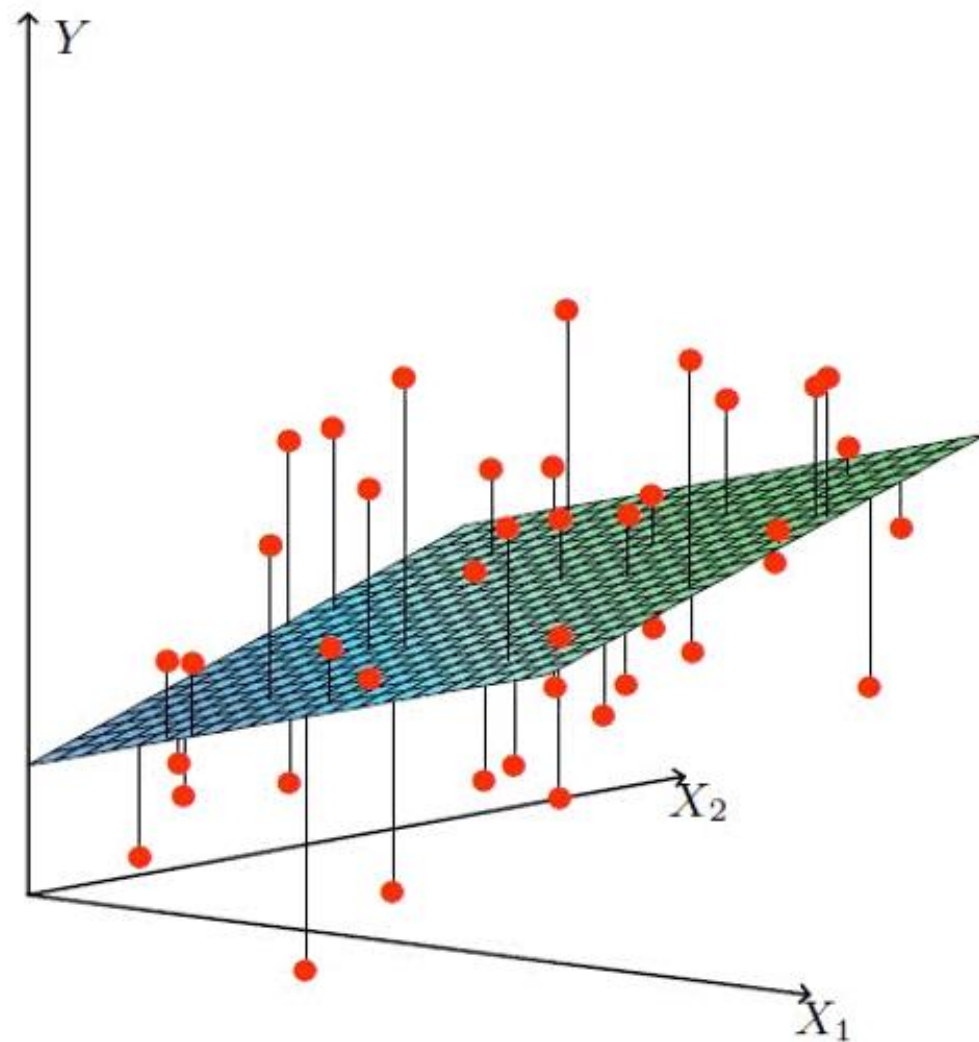
假设: hypothesis

| 房屋面积(m <sup>2</sup> ) | 租赁价格(1000¥) |
|-----------------------|-------------|
| 10                    | 0.8         |
| 15                    | 1           |
| 20                    | 1.8         |
| 30                    | 2           |
| 50                    | 3.2         |
| 60                    | 3           |
| 60                    | 3.1         |
| 70                    | 3.5         |



$$H(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

| 房屋面积  | 房间数量  | 租赁价格  |
|-------|-------|-------|
| 10    | 1     | 0.8   |
| 20    | 1     | 1.8   |
| 30    | 1     | 2.2   |
| 30    | 2     | 2.5   |
| 70    | 3     | 5.5   |
| 70    | 2     | 5.2   |
| ..... | ..... | ..... |





## 算法三要素

- **模型  $H(x)$** : 限定了假设的形式。
- **损失函数/代价函数/目标函数  $J(w)$** : 对假设做一个连续的度量,值越小,假设越好,将模型求解问题转换成数学优化问题.
- **优化求解算法**: 求解能够让损失函数取得最小值的参数  $w$ 。

## 线性回归模型

$$\begin{aligned}h_{\theta}(x) &= \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n \\&= \theta_0 1 + \theta_1 x_1 + \cdots + \theta_n x_n \\&= \theta_0 x_0 + \theta_1 x_1 + \cdots + \theta_n x_n \\&= \sum_{i=0}^n \theta_i x_i = \theta^T x\end{aligned}$$

- 最终要求是计算出 $\theta$ 的值，并选择最优的 $\theta$ 得到最终的函数

真实值 = 模型预测值(可解释部分) + 残差(不可解释部分)

The diagram illustrates the linear regression equation  $y^{(i)} = \theta^T x^{(i)} + \epsilon^{(i)}$  with handwritten annotations in Chinese:

- $y^{(i)}$  is circled in red, with a red arrow pointing to the text "训练样本的真实值" (True value of training sample) below it.
- $\theta^T x^{(i)}$  is circled in blue, with a blue arrow pointing to the text "训练模型预测值" (Training model prediction value) below it.
- $\epsilon^{(i)}$  is circled in purple, with a purple arrow pointing to the text "真实值和预测值差" (Difference between true value and prediction value) below it.
- A black arrow points from the text "样本次数" (Sample count) to the superscript  $(i)$  in  $y^{(i)}$ .

假设：残差服从均值为 0 的正态分布

$$y^{(i)} = \theta^T x^{(i)} + \epsilon^{(i)} \quad \text{公式1-1}$$

如前所述， $\epsilon$  满足高斯分布，其概率分布如右

$$p(\epsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(\epsilon^{(i)})^2}{2\sigma^2}\right)$$

又从公式1-1可得： $\epsilon = y - \theta x$ ，代入公式1-1，得下满公式

$$p(y^{(i)} | x^{(i)}; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

## 极大似然估计：联合概率最大化

求最大似然估计  $\Rightarrow L(\theta) = \prod_{i=1}^m p(y^{(i)} | x^{(i)}; \theta)$

$$= \prod_{i=1}^m \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$



## 极大似然估计：联合概率最大化

假如有一个罐子，里面有黑白两种颜色的球，数目多少不知，两种颜色的比例也不知。我们想知道罐中白球和黑球的比例，但我们不能把罐中的球全部拿出来数。现在我们可以每次任意从已经摇匀的罐中拿一个球出来，记录球的颜色，然后把拿出来的球再放回罐中。这个过程可以重复，我们可以用记录的球的颜色来估计罐中黑白球的比例。假如在前面的一百次重复记录中，有七十次是白球，请问罐中白球所占的比例最有可能是多少？



设白球概率为  $p$ ，黑球  $1-p$

$$L(p) = \underbrace{p \cdot p \cdot p \cdots p}_{70} \cdot \underbrace{(1-p) \cdot (1-p) \cdots (1-p)}_{30}$$
$$= p^{70} \cdot (1-p)^{30}$$

$$l(p) = \ln L(p) = \ln(p^{70} \cdot (1-p)^{30})$$
$$= 70 \ln p + 30 \ln(1-p)$$

$$l'(p) = \frac{70}{p} + \frac{-30}{1-p} = 0 \Rightarrow p = \frac{7}{10}$$

## 对数似然函数：连乘变连加

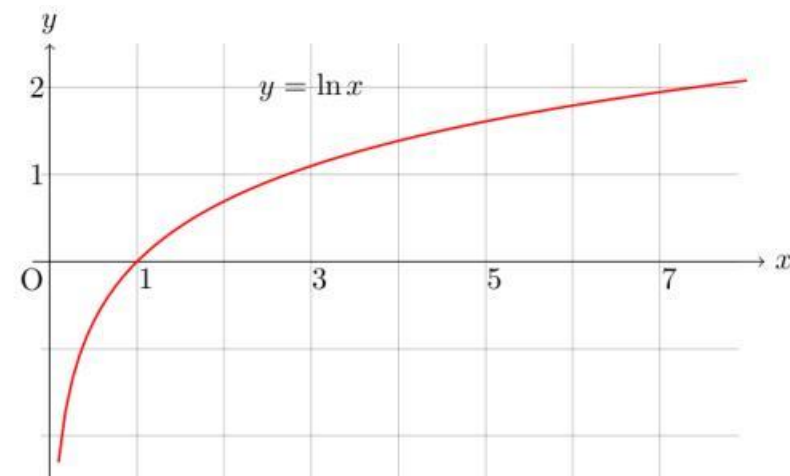
$$\begin{aligned}
 \ell(\theta) &= \log L(\theta) \\
 &= \log \prod_{i=1}^m \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \\
 &= \sum_{i=1}^m \log \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right) \\
 &= \overset{\text{定值优化不考虑}}{m \log \frac{1}{\sqrt{2\pi}\sigma}} - \frac{1}{\sigma^2} \cdot \frac{1}{2} \sum_{i=1}^m (y^{(i)} - \theta^T x^{(i)})^2 \\
 &\overset{\text{求最小值}}{J(\theta)} = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \quad \text{二乘法}
 \end{aligned}$$

$$\textcircled{1} \log_a^{(M \cdot N)} = \log_a^M + \log_a^N$$

$$\textcircled{2} \log_a^{M/N} = \log_a^M - \log_a^N$$

$$\textcircled{3} \log_a^{M^n} = n \log_a^M \quad (n \in R)$$

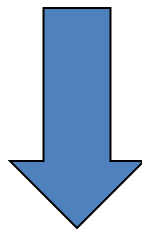
$$a^{\log_a N} = N$$



除以样本个数：消除样本量的影响

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \quad \text{二乘法}$$

求最小值



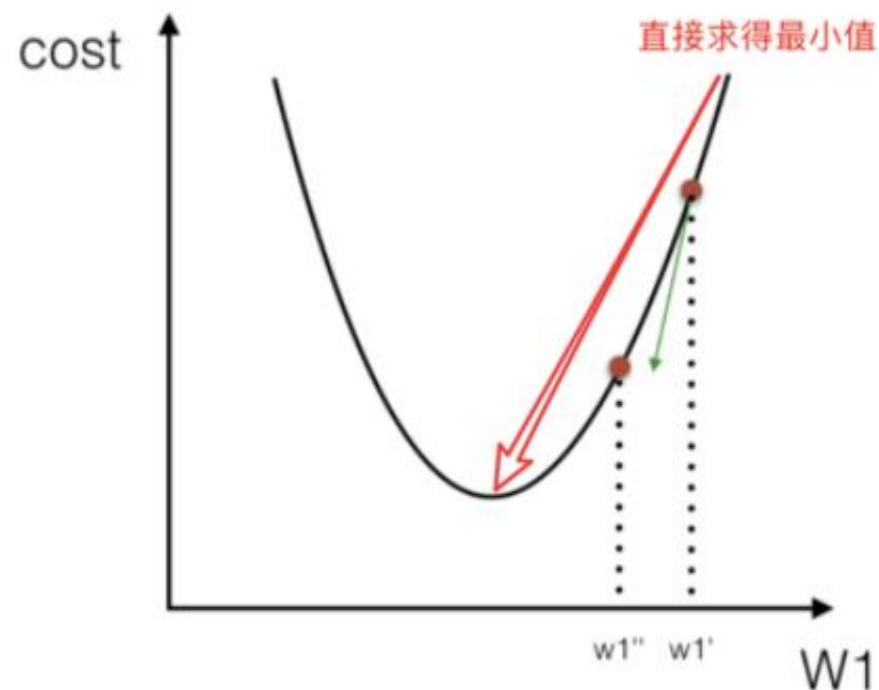
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

MSE 均方误差

## 最小二乘法

$$w = (X^T X)^{-1} X^T y$$

- **思想**：求导=0。
- **理解**：X为特征值矩阵，y为目标值矩阵。直接求到最好的结果
- **缺点**：当特征过多时，求解速度太慢(求逆计算量大)并且可能得不到结果(不满秩)。



## 最小二乘法

$$w = (X^T X)^{-1} X^T y$$

|       | Size (feet <sup>2</sup> ) | Number of bedrooms | Number of floors | Age of home (years) | Price (\$1000) |
|-------|---------------------------|--------------------|------------------|---------------------|----------------|
| $x_0$ | $x_1$                     | $x_2$              | $x_3$            | $x_4$               | $y$            |
| 1     | 2104                      | 5                  | 1                | 45                  | 460            |
| 1     | 1416                      | 3                  | 2                | 40                  | 232            |
| 1     | 1534                      | 3                  | 2                | 30                  | 315            |
| 1     | 852                       | 2                  | 1                | 36                  | 178            |

$$X = \begin{bmatrix} 1 & 2104 & 5 & 1 & 45 \\ 1 & 1416 & 3 & 2 & 40 \\ 1 & 1534 & 3 & 2 & 30 \\ 1 & 852 & 2 & 1 & 36 \end{bmatrix}$$

$$\left\{ \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2104 & 1416 & 1534 & 852 \\ 5 & 3 & 3 & 2 \\ 1 & 2 & 2 & 1 \\ 45 & 40 & 30 & 36 \end{bmatrix} \times \begin{bmatrix} 1 & 2104 & 5 & 1 & 45 \\ 1 & 1416 & 3 & 2 & 40 \\ 1 & 1534 & 3 & 2 & 30 \\ 1 & 852 & 2 & 1 & 36 \end{bmatrix} \right\}^{-1} \times \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2104 & 1416 & 1534 & 852 \\ 5 & 3 & 3 & 2 \\ 1 & 2 & 2 & 1 \\ 45 & 40 & 30 & 36 \end{bmatrix} \times \begin{bmatrix} 460 \\ 232 \\ 315 \\ 178 \end{bmatrix}$$

## 最小二乘法推导

$$J(\theta) = \frac{1}{2} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 = \frac{1}{2} (X\theta - Y)^T (X\theta - Y) \rightarrow \min_{\theta} J(\theta)$$

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \left( \frac{1}{2} (X\theta - Y)^T (X\theta - Y) \right) = \nabla_{\theta} \left( \frac{1}{2} (\theta^T X^T - Y^T) (X\theta - Y) \right)$$

$$= \nabla_{\theta} \left( \frac{1}{2} (\theta^T X^T X\theta - \theta^T X^T Y - Y^T X\theta + Y^T Y) \right)$$

$$= \frac{1}{2} (2X^T X\theta - X^T Y - (Y^T X)^T)$$

$$= X^T X\theta - X^T Y$$

$$\theta = (X^T X)^{-1} X^T Y$$

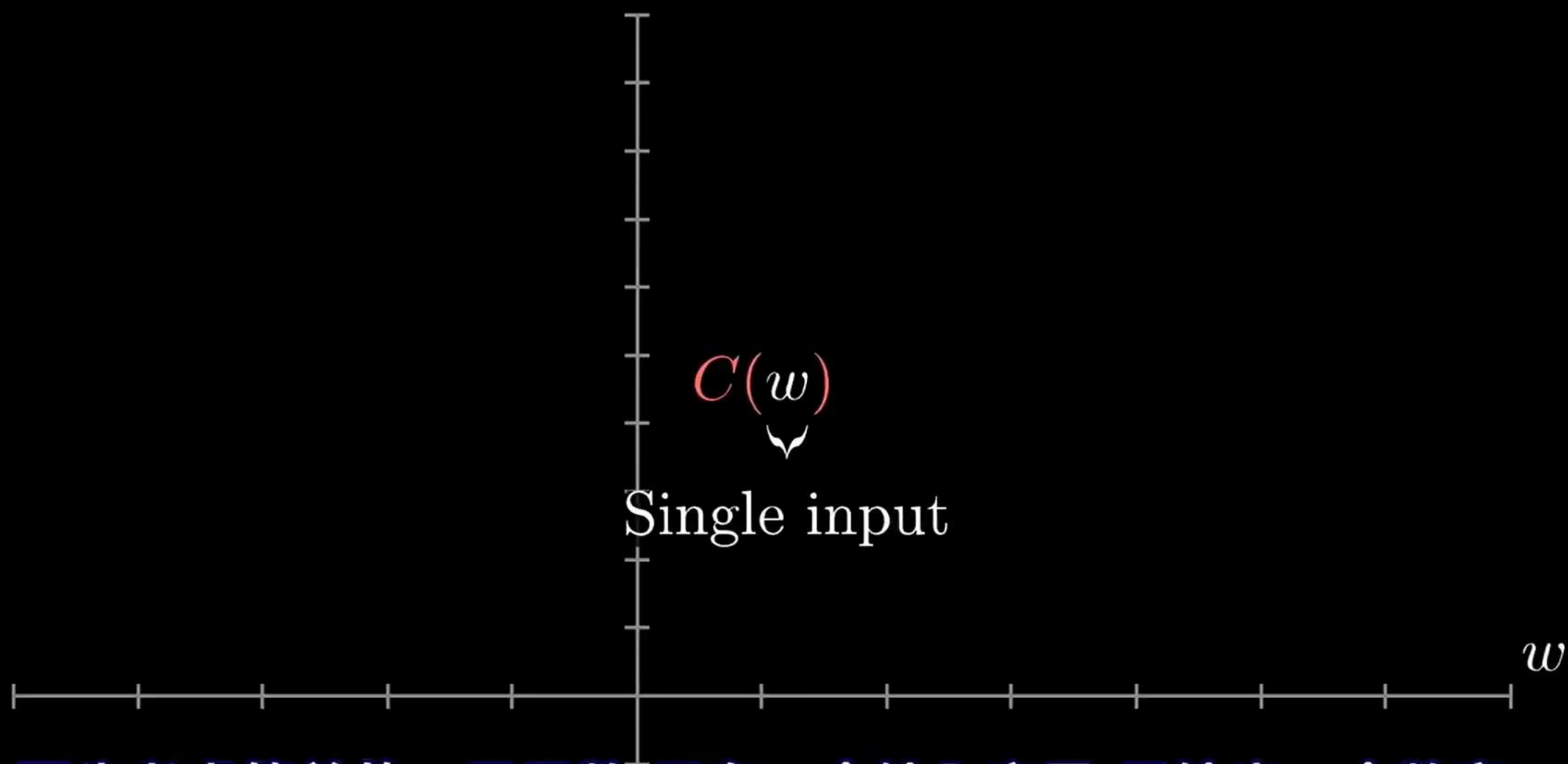


## 梯度下降法

$$\theta_{i+1} = \theta_i - \alpha \frac{\partial}{\partial \theta_i} J(\theta)$$

**梯度**是微积分中一个很重要的概念

- 在**单变量**的函数中，梯度其实就是函数的微分，是一个**标量**，只有**正负号**之分；
- 在**多变量**函数中，梯度是一个**向量**，向量有方向，梯度的方向就指出了函数在给定点的**上升最快**的方向；
- **学习率 $\alpha$** ：学习率或者步长，意味着我们可以通过 $\alpha$ 来控制每一步走的距离，以保证不要步子跨的太大扯着蛋
- **负梯度**：梯度前加一个负号，就意味着朝着梯度相反的方向前进！



**而先考虑简单的一元函数 只有一个输入变量 只输出一个数字**

just imagine a simple function that has one number as an input and one number as an output.

## 梯度下降法：计算案例-单变量

我们假设有一个单变量的函数： $J(\theta) = \theta^2$

函数的微分： $J'(\theta) = 2\theta$

初始化，起点为： $\theta_0 = 1$

学习率： $\alpha = 0.4$

$$\theta_{i+1} = \theta_i - \alpha \frac{\partial}{\partial \theta_i} J(\theta)$$

$$\theta^0 = 1$$

$$\begin{aligned}\theta^1 &= \theta^0 - \alpha * J'(\theta^0) \\ &= 1 - 0.4 * 2 \\ &= 0.2\end{aligned}$$

$$\begin{aligned}\theta^2 &= \theta^1 - \alpha * J'(\theta^1) \\ &= 0.04\end{aligned}$$

$$\theta^3 = 0.008$$

$$\theta^4 = 0.0016$$

## 梯度下降法：计算案例-多变量

我们假设有一个多变量的函数： $J(\theta) = \theta_1^2 + \theta_2^2$

函数的微分： $\nabla J(\theta) = \langle 2\theta_1, 2\theta_2 \rangle$

初始化，起点为： $\theta_0 = \langle 1, 3 \rangle$

学习率： $\alpha = 0.1$

$$\theta_{i+1} = \theta_i - \alpha \frac{\partial}{\partial \theta_i} J(\theta)$$

$$\Theta^0 = (1, 3)$$

$$\Theta^1 = \Theta^0 - \alpha \nabla J(\Theta)$$

$$= (1, 3) - 0.1(2, 6)$$

$$= (0.8, 2.4)$$

$$\Theta^2 = (0.8, 2.4) - 0.1(1.6, 4.8)$$

$$= (0.64, 1.92)$$

$$\Theta^3 = (0.512, 1.536)$$

$$\Theta^4 = (0.4096, 1.2288000000000001)$$

$\vdots$

$$\Theta^{10} = (0.10737418240000003, 0.32212254720000005)$$

$\vdots$

$$\Theta^{50} = (1.1417981541647683e^{-05}, 3.425394462494306e^{-05})$$

$\vdots$

$$\Theta^{100} = (1.6296287810675902e^{-10}, 4.888886343202771e^{-10})$$

线性回归梯度下降法推导：计算导数

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

$$\begin{aligned} \frac{\partial}{\partial \theta_j} J(\theta) &= \frac{\partial}{\partial \theta_j} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \\ &= 2 \cdot \frac{1}{2m} \cdot \sum_{i=1}^m \left[ (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot \frac{\partial}{\partial \theta_j} (h_{\theta}(x^{(i)}) - y^{(i)}) \right] \\ &= \frac{1}{m} \cdot \sum_{i=1}^m \left[ (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot \frac{\partial}{\partial \theta_j} (\theta_0 x_0^i + \theta_1 x_1^i + \theta_2 x_2^i + \cdots + \theta_n x_n^i - y^{(i)}) \right] \\ &= \frac{1}{m} \cdot \sum_{i=1}^m [(h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^i] \end{aligned}$$

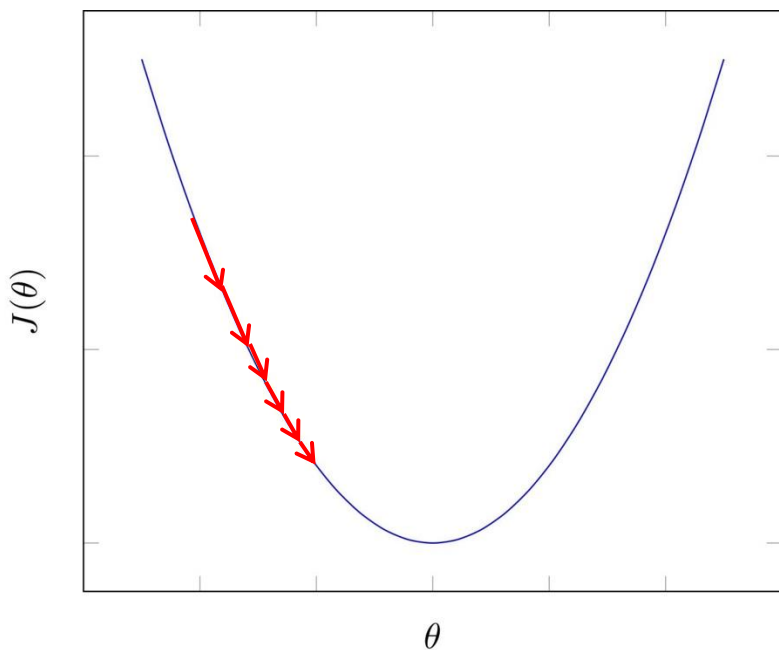
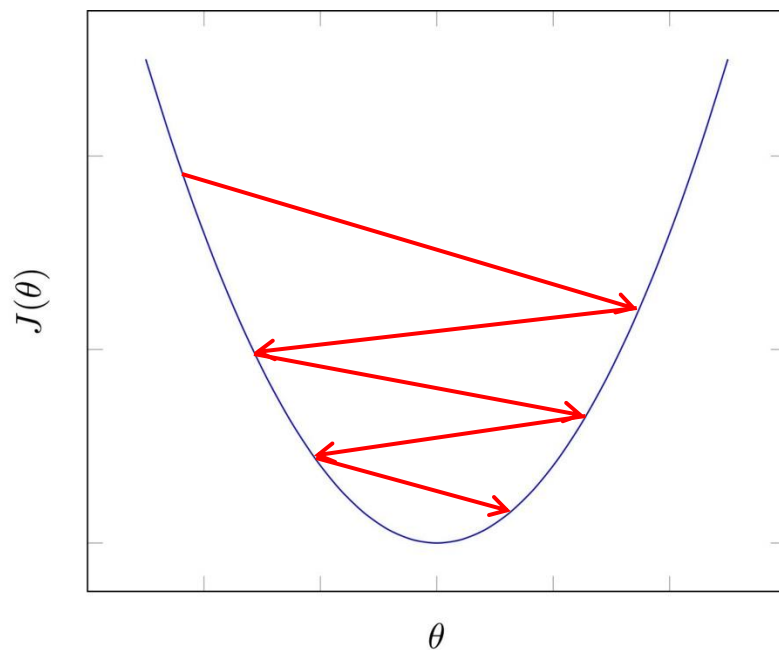
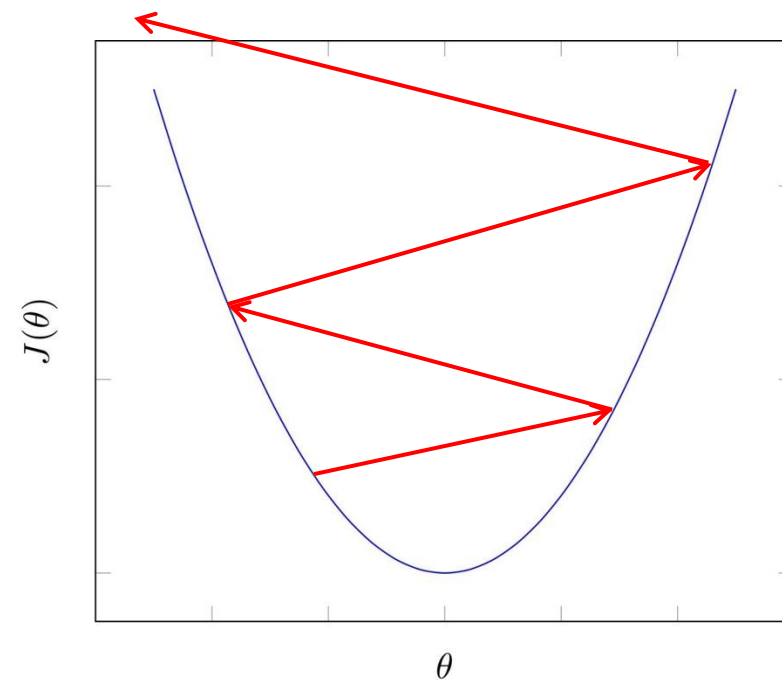
## 线性回归梯度下降实现

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \cdot \sum_{i=1}^m [(h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^i]$$
$$\theta_{j+1} = \theta_j - \alpha \cdot \frac{1}{m} \cdot \sum_{i=1}^m [(h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^i]$$
$$\theta_{i+1} = \theta_i - \alpha \frac{\partial}{\partial \theta_i} J(\theta)$$



## 梯度爆炸和之字形下降

$$\theta_{j+1} = \theta_j - \alpha \cdot \frac{1}{m} \cdot \sum_{i=1}^m [(h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^i]$$

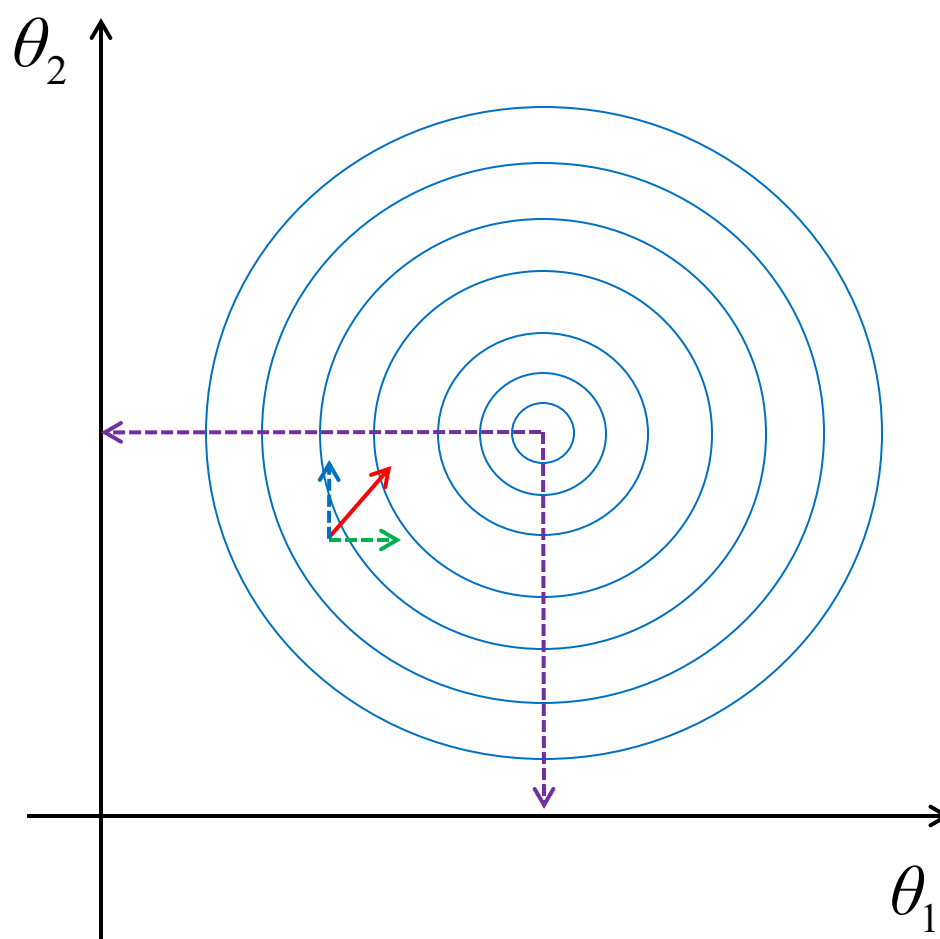
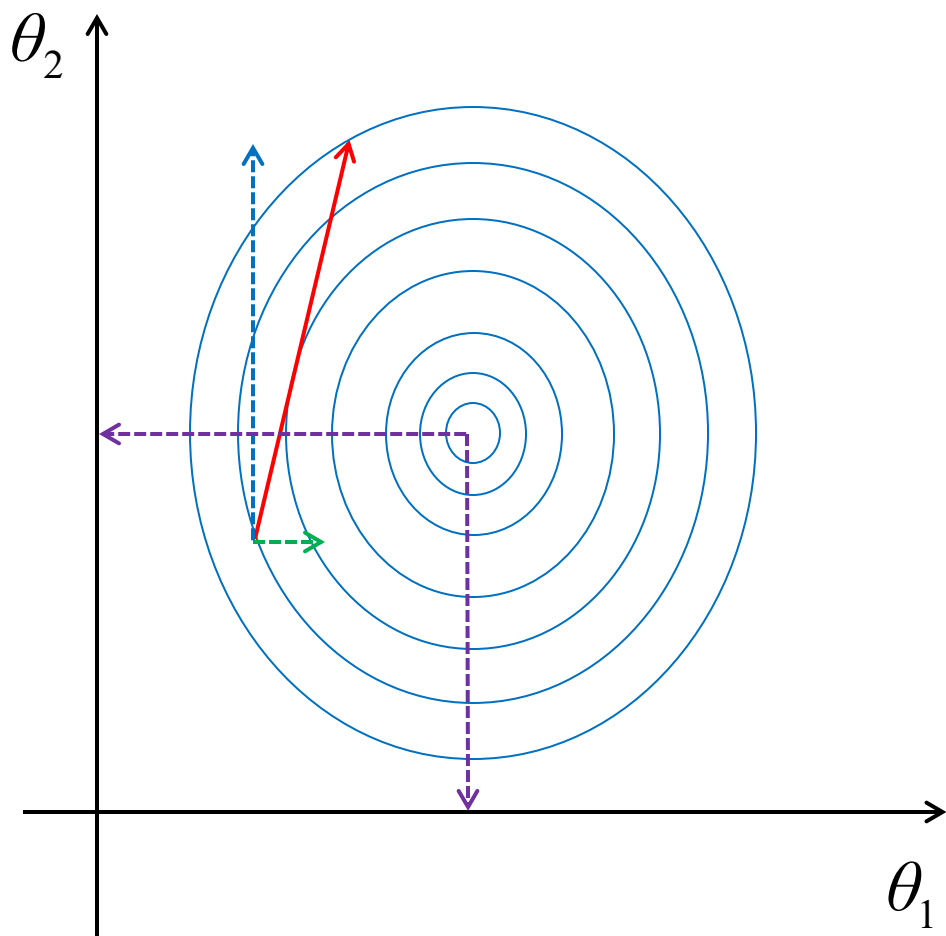
 $\alpha$ 较小 $\alpha$ 较大 $\alpha$ 太大导致梯度爆炸

## 线性回归使用梯度下降法需要不需要对数据做标准化?

$$\theta_{j+1} = \theta_j - \alpha \cdot \frac{1}{m} \cdot \sum_{i=1}^m [(h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x_j^i]$$

- 偏导数跟权重对应的特征大小相关
- 学习率是所有权重**共用**的
- 如果某个**特征太大**,将导致该特征对应的权重出现**梯度爆炸**而无法优化

## 线性回归使用梯度下降法需不需要对数据做标准化?



## 梯度下降法本质：一阶泰勒展开

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(x_0)}{n!} (x - x_0)^n$$

■ 定义：泰勒公式是一个用函数在某点的信息描述其附近取值的公式。局部有效性

■ 一阶泰勒展开：  $f(x) \approx f(x_0) + f'(x_0)(x - x_0)$

- 迭代公式：  $\theta^t = \theta^{t-1} + \Delta\theta$
- 将  $L(\theta^t)$  在  $\theta^{t-1}$  处进行一阶泰勒展开：

$$\begin{aligned} L(\theta^t) &= L(\theta^{t-1} + \Delta\theta) \\ &\approx L(\theta^{t-1}) + L'(\theta^{t-1})\Delta\theta \end{aligned}$$

- 要使得  $L(\theta^t) < L(\theta^{t-1})$ ，可取：  $\Delta\theta = -\alpha L'(\theta^{t-1})$ ，则：  $\theta^t = \theta^{t-1} - \alpha L'(\theta^{t-1})$

## 导入类和函数

```
#线性回归器  
from sklearn.linear_model import LinearRegression  
#波士顿房价数据  
from sklearn.datasets import load_boston  
#用于划分测试集和训练集两部分  
from sklearn.model_selection import train_test_split  
#用于计算均方误差回归损失  
from sklearn.metrics import mean_squared_error  
#用于数据标准化  
from sklearn.preprocessing import StandardScaler
```

## 回归模型评价指标

$$MSE = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE} = \sqrt{\frac{1}{m} \sum_{i=1}^m (y_i - \hat{y}_i)^2}$$

越小越好，跟y的方差有关

$$y = \theta^T x + \varepsilon$$

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^m (y_i - \hat{y}_i)^2}{\sum_{i=1}^m (y_i - \bar{y})^2}$$

$$\bar{y} = \frac{1}{m} \sum_{i=1}^m y_i$$

真实值

预测值

误差

总方差

可解释方差

不可解释方差

可解释方差比例，0-1之间，越大越好

不可解释方差比例

## 小结

本节我们主要学习了线性回归的基本原理：

- 算法三要素：模型/损失函数/优化求解方法
- 模型：
$$h_{\theta}(x) = \sum_{i=0}^n \theta_i x_i = \theta^T x$$
- 损失函数：
$$J(\theta) = \frac{1}{2m} \sum_{i=1}^m \left( h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$
- 优化求解：最小二乘法、梯度下降法
- 评估指标：MSE、RMSE、 $R^2$



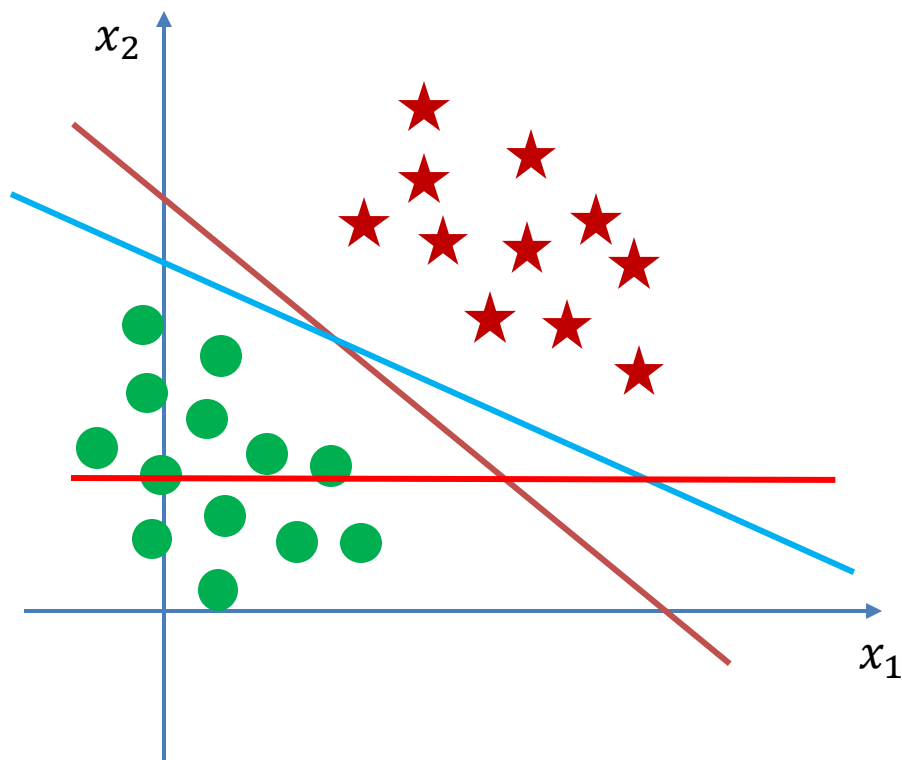




# 1 2 chapter

## 逻辑回归

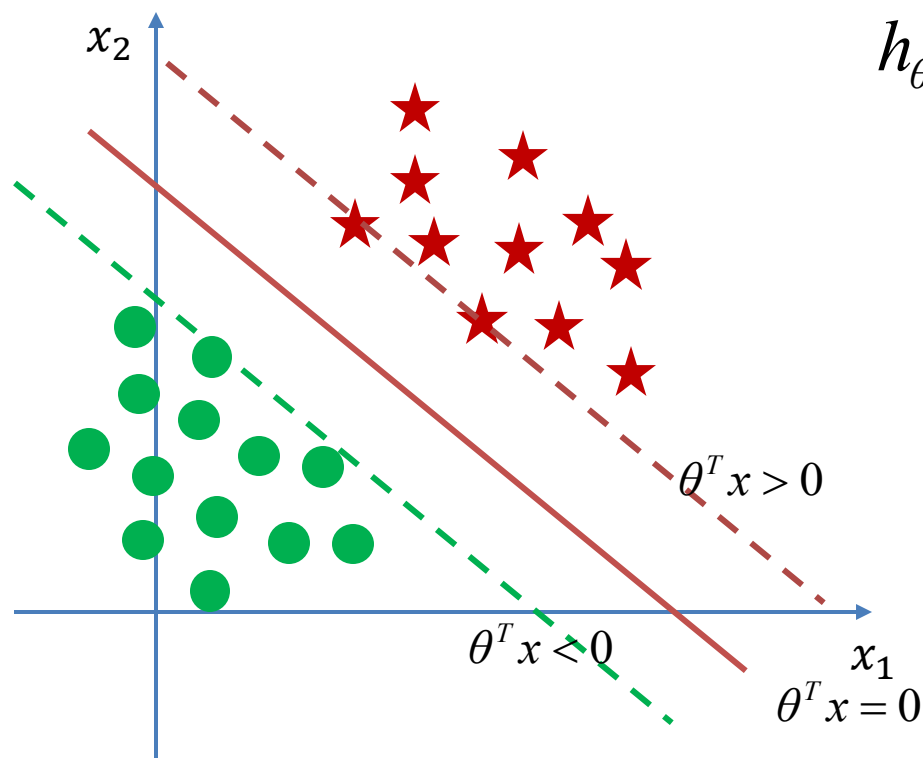
## 逻辑回归直观理解：一条线进行分隔



$$h_{\theta}(x) = \theta_1 x_1 + \theta_2 x_2 + \theta_0$$

## 逻辑回归的模型：使用回归做分类

- 输出:连续值  $(-\infty, +\infty)$
- 目标 $y$ : $\{0,1\}$



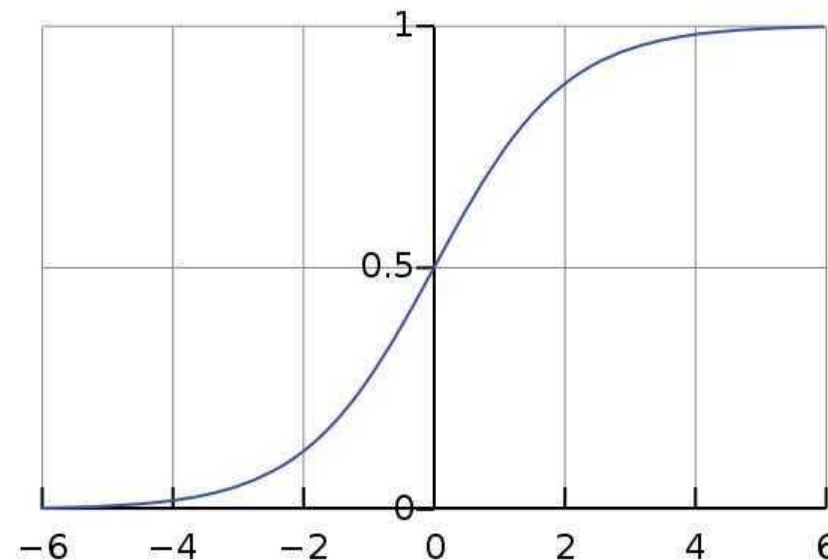
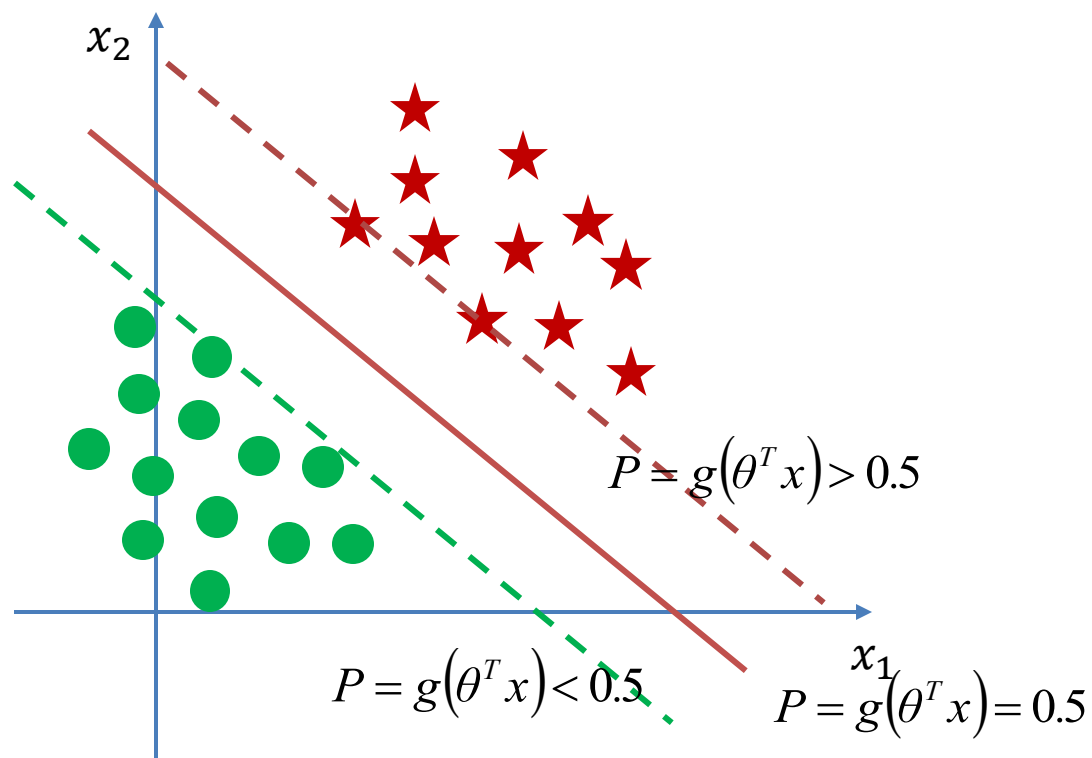
$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \theta^T x$$

$$h_{\theta}(x) = g(\theta^T x) = \begin{cases} 1 & \theta^T x > 0 \\ 0 & \theta^T x \leq 0 \end{cases}$$

## 逻辑回归的模型：如何解决不连续的问题？--sigmoid函数

■ 作用:将线性回归的输出  $(-\infty, +\infty)$  映射到区间  $(0, 1)$

$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n = \theta^T x$$

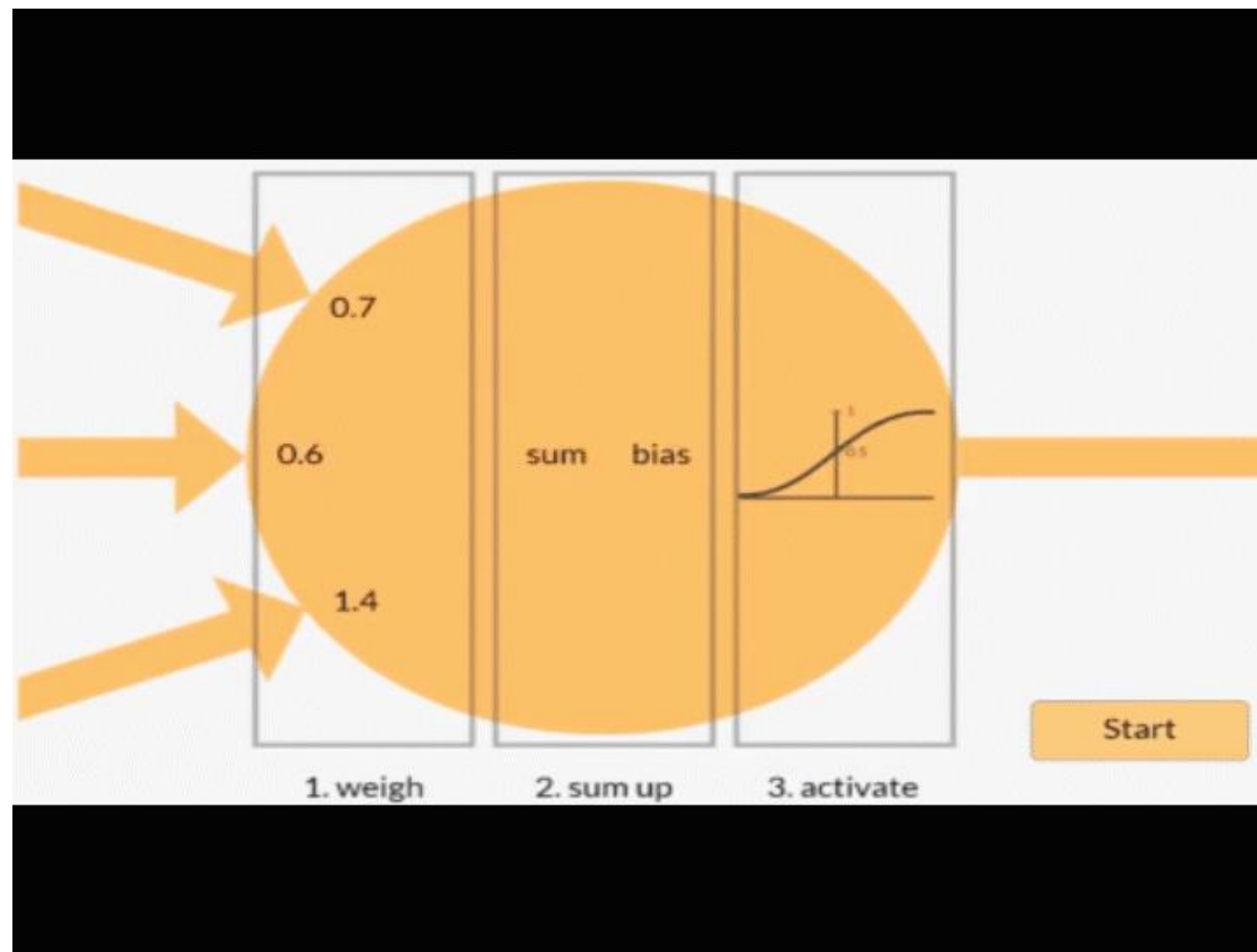
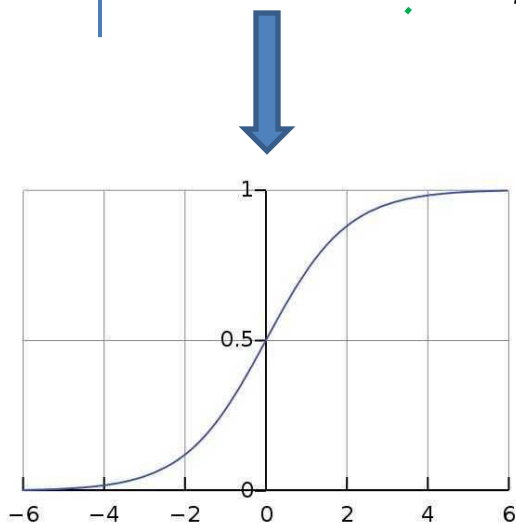
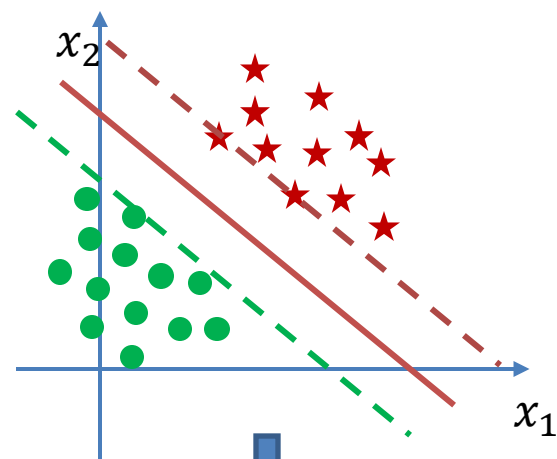


$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

$$P(y = 1|x; \theta) = \frac{1}{1 + e^{-\theta^T x}} = h_{\theta}(x)$$

$$P(y = 0|x; \theta) = 1 - \frac{1}{1 + e^{-\theta^T x}} = 1 - h_{\theta}(x)$$

## 逻辑回归的模型：如何解决不连续的问题?--sigmoid函数



## 逻辑回归的模型：对数几率回归

$$p = \frac{1}{1 + e^{-(wx+b)}} \quad \text{等价于} \quad \ln\left(\frac{p}{1-p}\right) = wx + b$$

- 如果我们把  $\ln(p/1-p)$  当做  $y$  看待，那么  $y=wx+b$  就是线性回归了。
- 我们习惯把  $p/1-p$  称为 **odds**, 中文可以叫 **几率**，就是代表类别发生和不发生的概率比值
- 这样逻辑回归又可以表达为：  **$\ln(odds)=wx+b$**
- 所以：逻辑回归算法属于**分类**算法而**不是回归**算法，但是它的算法形式与线性回归很**相似**。

## 逻辑回归的损失函数推导：极大似然估计

$$X \rightarrow Y \quad X: \{x^{(1)}, x^{(2)}, x^{(3)}, \dots, x^{(m)}\} \quad Y: \{y^{(1)}, y^{(2)}, y^{(3)}, \dots, y^{(m)}\}$$

$$P(Y|X; \theta) = P(y^{(1)}|x^{(1)}; \theta) * P(y^{(2)}|x^{(2)}; \theta) * \dots * P(y^{(m)}|x^{(m)}; \theta) = \prod_{i=1}^m P(y^{(i)}|x^{(i)}; \theta)$$

$$P(y = 1|x; \theta) = \frac{1}{1 + e^{-\theta^T x}} = h_{\theta}(x)$$

$$\longrightarrow P(y|x; \theta) = (h_{\theta}(x))^y (1 - h_{\theta}(x))^{1-y}$$

$$P(y = 0|x; \theta) = 1 - \frac{1}{1 + e^{-\theta^T x}} = 1 - h_{\theta}(x)$$

■ 目标y:{0,1}

■ 伯努利分布



## 逻辑回归的损失函数推导：对数似然

$$P(Y|X; \theta) = \prod_{i=1}^m P(y^{(i)} | x^{(i)}; \theta)$$

$$P(y|x; \theta) = (h_{\theta}(x))^y (1 - h_{\theta}(x))^{1-y}$$

$$L(\theta) = \prod_{i=1}^m P(y^{(i)} | x^{(i)}; \theta) = \prod_{i=1}^m (h_{\theta}(x^{(i)}))^{y^{(i)}} (1 - h_{\theta}(x^{(i)}))^{1-y^{(i)}}$$

取对数

$$l(\theta) = \log L(\theta) = \sum_{i=1}^m \left( y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right)$$

## 逻辑回归的损失函数推导：对数似然损失/交叉熵损失

$$l(\theta) = \log L(\theta) = \sum_{i=1}^m \left( y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right) \quad J(\theta) = -\frac{1}{m} l(\theta)$$


$$Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

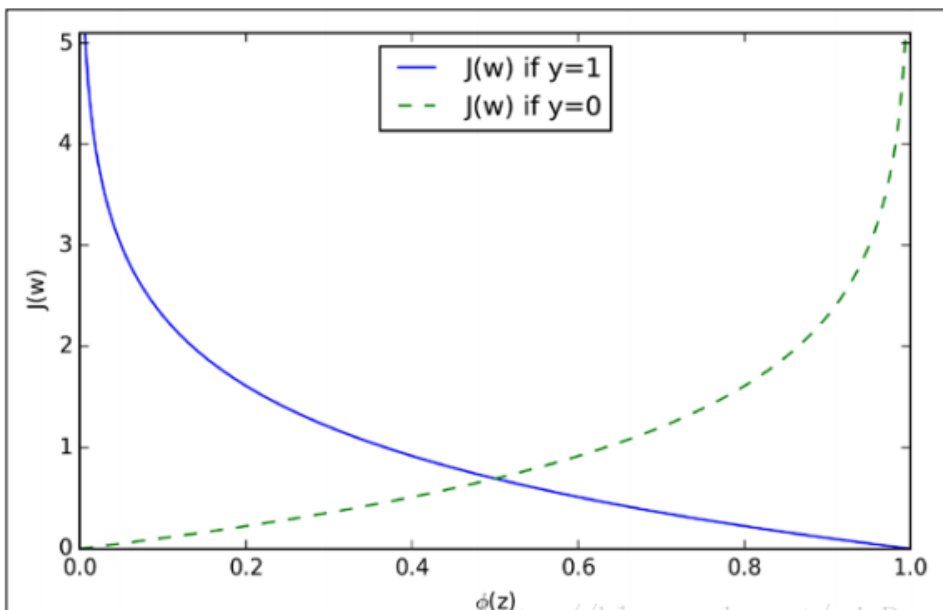
$$J(\theta) = \frac{1}{m} \sum_{i=1}^m Cost(h_{\theta}(x^{(i)}), y^{(i)}) = -\frac{1}{m} \sum_{i=1}^m \left( y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right)$$

## 逻辑回归的损失函数计算案例

| obs | x     | y | p(y=1) | p(y=0) | p(y) | -ln(p)  |
|-----|-------|---|--------|--------|------|---------|
| 1   | 72.74 | 1 | 0.95   | 0.05   | 0.95 | 0.05129 |
| 2   | 38.51 | 0 | 0.23   | 0.77   | 0.77 | 0.26136 |
| 3   | 9.00  | 1 | 0.67   | 0.33   | 0.67 | 0.40048 |
| 4   | 36.00 | 1 | 0.85   | 0.15   | 0.85 | 0.16252 |
| 5   | 33.38 | 0 | 0.23   | 0.77   | 0.77 | 0.26136 |
| 6   | 9.31  | 0 | 0.34   | 0.66   | 0.66 | 0.41552 |
| 7   | 2.80  | 0 | 0.13   | 0.87   | 0.87 | 0.13926 |
| 8   | 50.09 | 1 | 0.87   | 0.13   | 0.87 | 0.13926 |
| 9   | 34.18 | 0 | 0.45   | 0.55   | 0.55 | 0.59784 |
| 10  | 53.85 | 0 | 0.32   | 0.68   | 0.68 | 0.38566 |
| 11  | 65.06 | 1 | 0.69   | 0.31   | 0.69 | 0.37106 |
| 12  | 75.32 | 1 | 0.78   | 0.22   | 0.78 | 0.24846 |
| 13  | 10.73 | 0 | 0.32   | 0.68   | 0.68 | 0.38566 |
| 14  | 53.45 | 0 | 0.26   | 0.74   | 0.74 | 0.30111 |
|     |       |   |        |        | avg  | 0.29435 |

## 逻辑回归优化思想

- $y=1$ , 增大概率
- $y=0$ , 减小概率



$$Cost(h_{\theta}(x), y) = \begin{cases} -\log(h_{\theta}(x)) & \text{if } y = 1 \\ -\log(1 - h_{\theta}(x)) & \text{if } y = 0 \end{cases}$$

$$J(\theta) = \frac{1}{m} \sum_{i=1}^n Cost(h_{\theta}(x^{(i)}), y^{(i)}) = -\frac{1}{m} \sum_{i=1}^n (y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})))$$

| obs | x     | y | p(y=1) | p(y=0) | p(y) | <b>-ln(p)</b> |
|-----|-------|---|--------|--------|------|---------------|
| 1   | 72.74 | 1 | 0.95   | 0.05   | 0.95 | 0.05129       |
| 2   | 38.51 | 0 | 0.23   | 0.77   | 0.77 | 0.26136       |
| 3   | 9.00  | 1 | 0.67   | 0.33   | 0.67 | 0.40048       |
| 4   | 36.00 | 1 | 0.85   | 0.15   | 0.85 | 0.16252       |
| 5   | 33.38 | 0 | 0.23   | 0.77   | 0.77 | 0.26136       |
| 6   | 9.31  | 0 | 0.34   | 0.66   | 0.66 | 0.41552       |
| 7   | 2.80  | 0 | 0.13   | 0.87   | 0.87 | 0.13926       |
| 8   | 50.09 | 1 | 0.87   | 0.13   | 0.87 | 0.13926       |
| 9   | 34.18 | 0 | 0.45   | 0.55   | 0.55 | 0.59784       |
| 10  | 53.85 | 0 | 0.32   | 0.68   | 0.68 | 0.38566       |
| 11  | 65.06 | 1 | 0.69   | 0.31   | 0.69 | 0.37106       |
| 12  | 75.32 | 1 | 0.78   | 0.22   | 0.78 | 0.24846       |
| 13  | 10.73 | 0 | 0.32   | 0.68   | 0.68 | 0.38566       |
| 14  | 53.45 | 0 | 0.26   | 0.74   | 0.74 | 0.30111       |
| avg |       |   |        |        |      | 0.29435       |

## 逻辑回归梯度下降：导数推导

$$\begin{aligned}
\frac{\delta}{\delta \theta_j} J(\theta) &= -\frac{1}{m} \sum_{i=1}^m \left( y^{(i)} \frac{1}{h_{\theta}(x^{(i)})} \frac{\delta}{\delta \theta_j} h_{\theta}(x^{(i)}) - (1-y^{(i)}) \frac{1}{1-h_{\theta}(x^{(i)})} \frac{\delta}{\delta \theta_j} h_{\theta}(x^{(i)}) \right) \\
&= -\frac{1}{m} \sum_{i=1}^m \left( y^{(i)} \frac{1}{g(\theta^T x^{(i)})} - (1-y^{(i)}) \frac{1}{1-g(\theta^T x^{(i)})} \right) \frac{\delta}{\delta \theta_j} g(\theta^T x^{(i)}) \\
&= -\frac{1}{m} \sum_{i=1}^m \left( y^{(i)} \frac{1}{g(\theta^T x^{(i)})} - (1-y^{(i)}) \frac{1}{1-g(\theta^T x^{(i)})} \right) g(\theta^T x^{(i)}) (1-g(\theta^T x^{(i)})) \frac{\delta}{\delta \theta_j} \theta^T x^{(i)} \\
&= -\frac{1}{m} \sum_{i=1}^m \left( y^{(i)} (1-g(\theta^T x^{(i)})) - (1-y^{(i)}) g(\theta^T x^{(i)}) \right) x_j^{(i)} \\
&= -\frac{1}{m} \sum_{i=1}^m \left( y^{(i)} - g(\theta^T x^{(i)}) \right) x_j^{(i)} \\
&= \frac{1}{m} \sum_{i=1}^m \left( h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}
\end{aligned}$$

- sigmoid函数能映射到(0,1)
- sigmoid函数求导方便

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = \left( \frac{1}{1 + e^{-z}} \right)' = \frac{e^{-z}}{(1 + e^{-z})^2}$$

$$\begin{aligned}
&= \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \frac{1}{1 + e^{-z}} \cdot \left( 1 - \frac{1}{1 + e^{-z}} \right) \\
&= g(z) \cdot (1 - g(z))
\end{aligned}$$

## 逻辑回归梯度下降

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

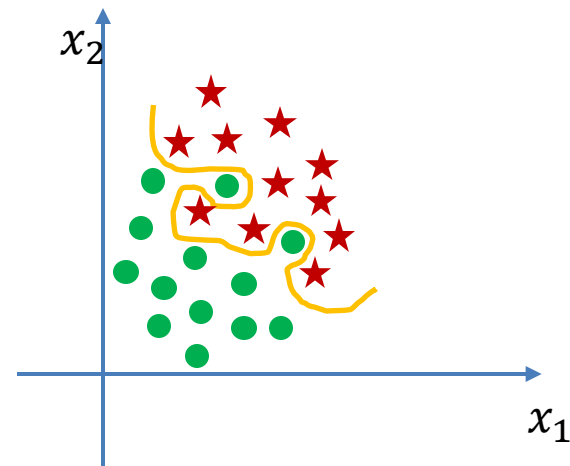
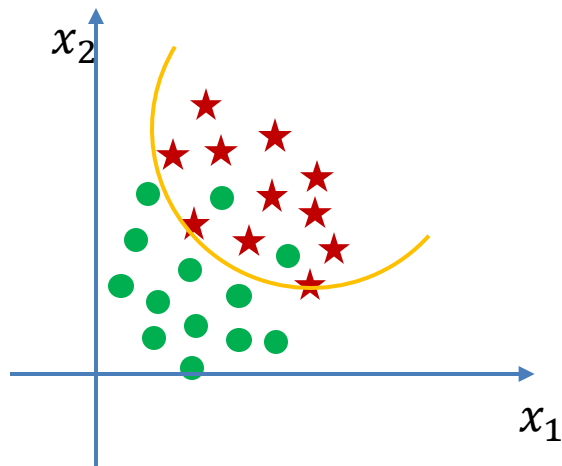
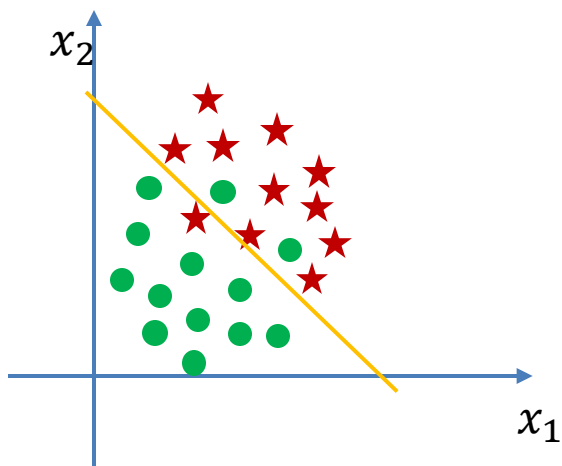
$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{1}{m} \sum_{i=1}^m \left( h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

■ 同样需要标准化

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left( h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

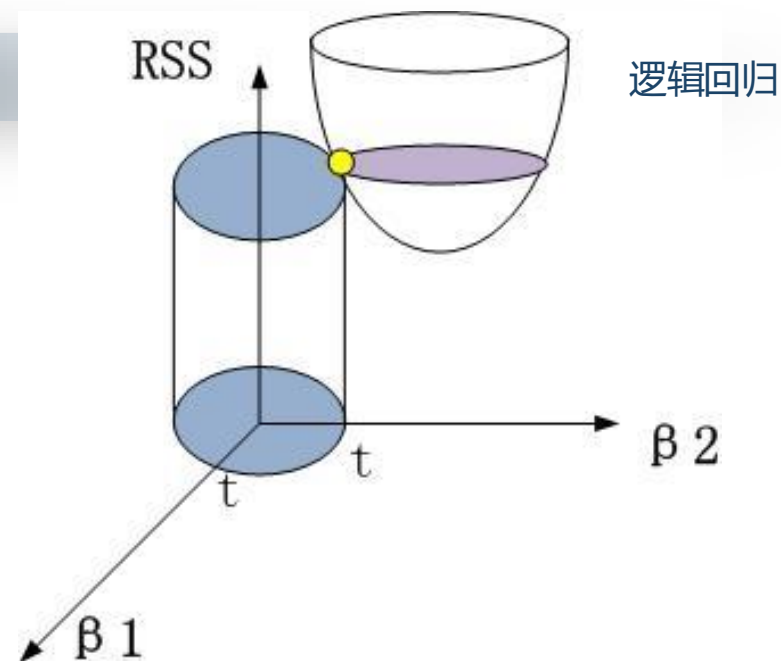
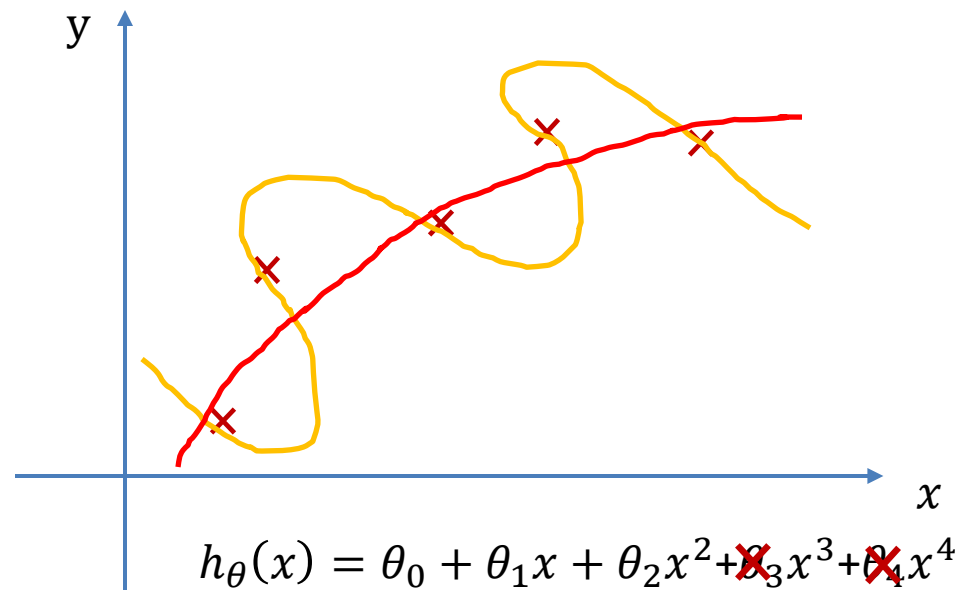
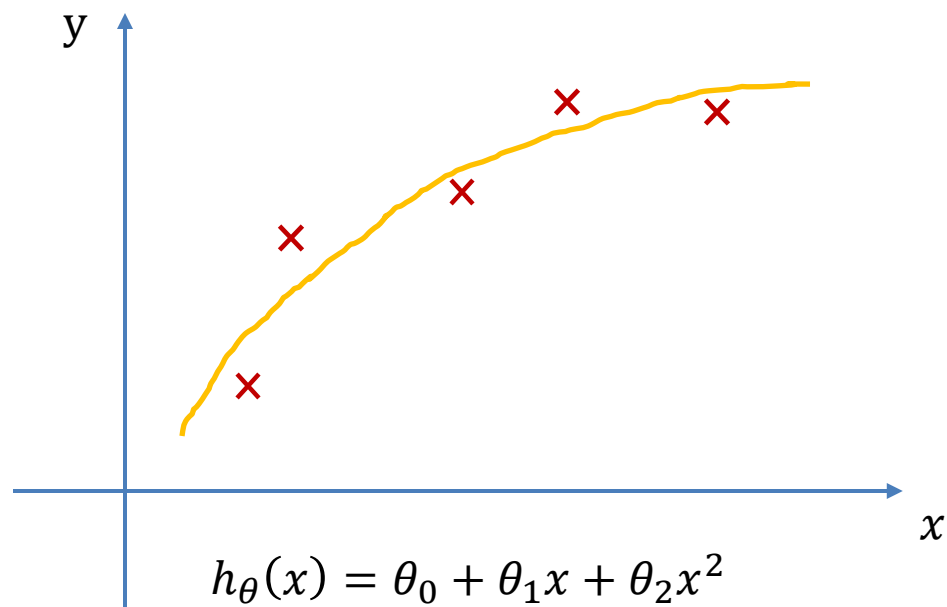
## 过拟合与欠拟合

- 过拟合出现的根本原因: 训练集和测试集数据**分布差异**
- 过拟合: 模型过于复杂
- 欠拟合: 模型过于简单



## 正则化

- 正则化本质:抑制模型在训练集上求最优解
- 训练集的最优解不是未知数据的最优解
- L2正则/L1正则





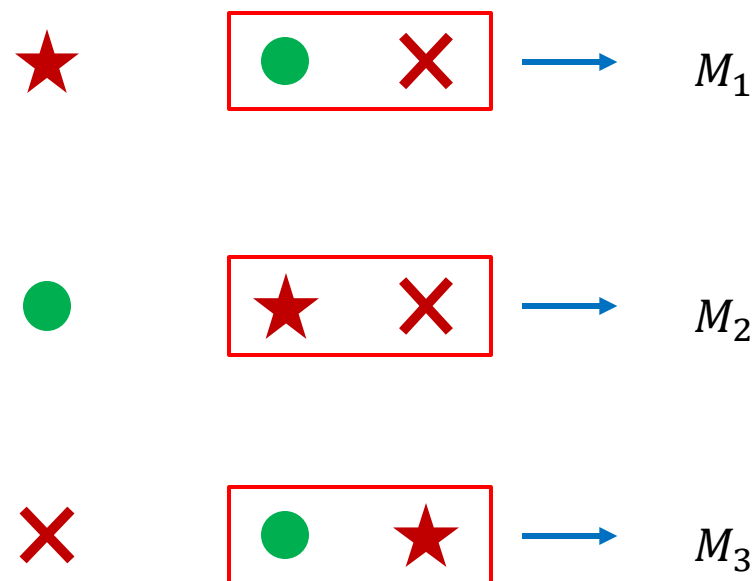
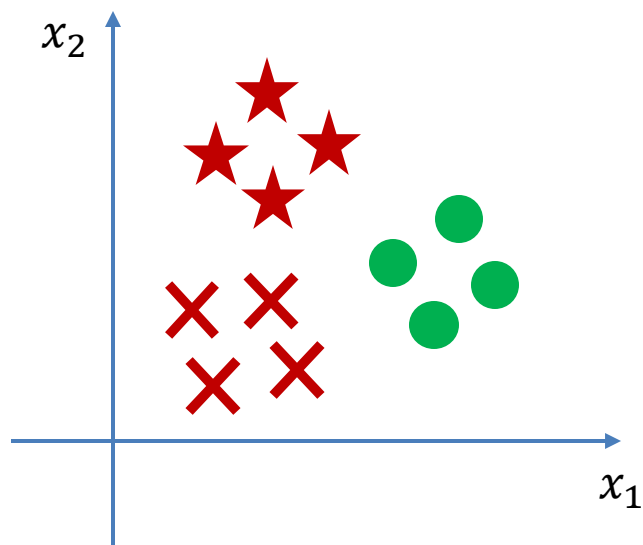
## 逻辑回归本身只能做二分类

逻辑回归本身只能做0/1二分类,可以通过三种方案实现多分类:

- **ovr(one vs rest)**
- **ovo(one vs one)**
- **softmax回归**

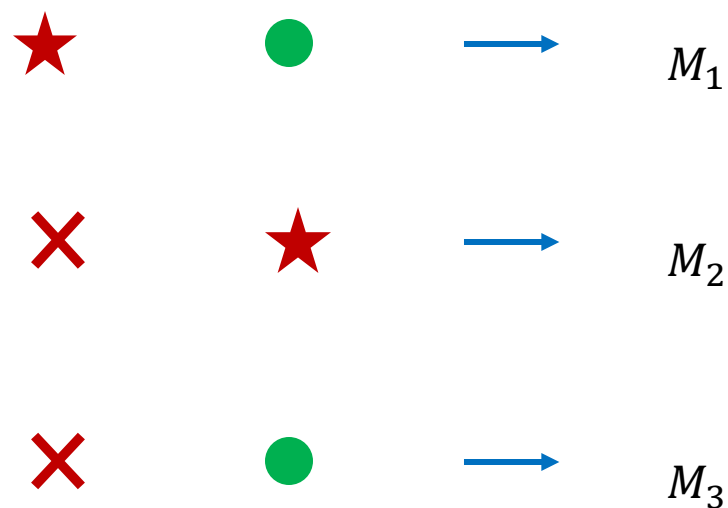
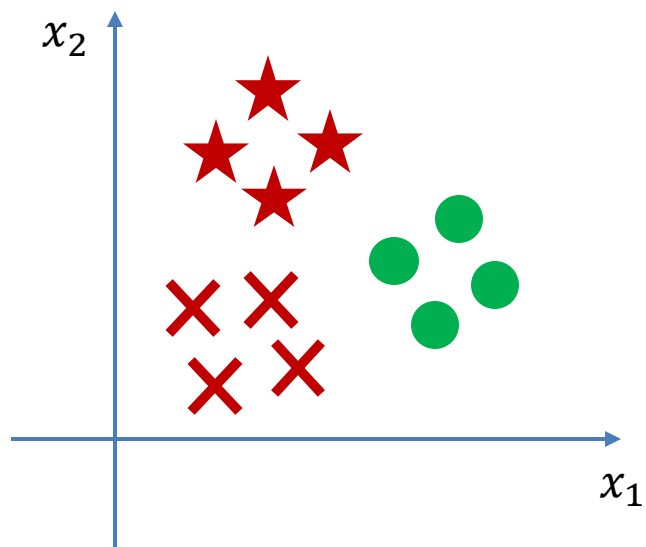
## 逻辑回归多分类：ovr

- 对于 $k$ 个类别,需要训练 $k$ 个模型
- 类别较多时,会出现**样本不平衡**问题



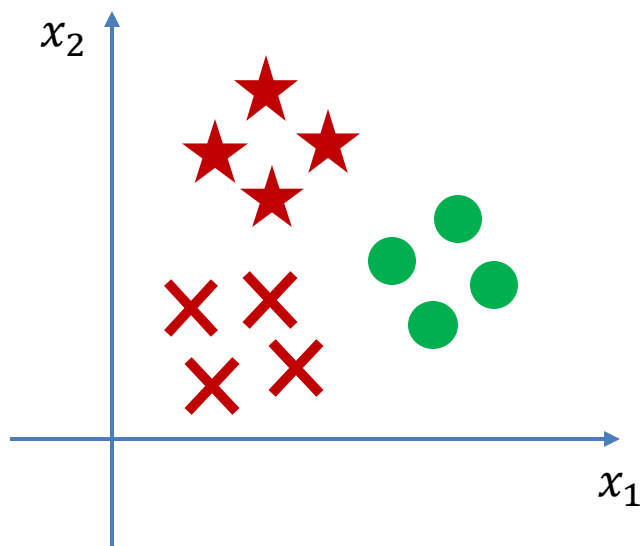
## 逻辑回归多分类：ovo

■ 对于 $k$ 个类别,需要训练 $(k*(k-1))/2$ 个模型

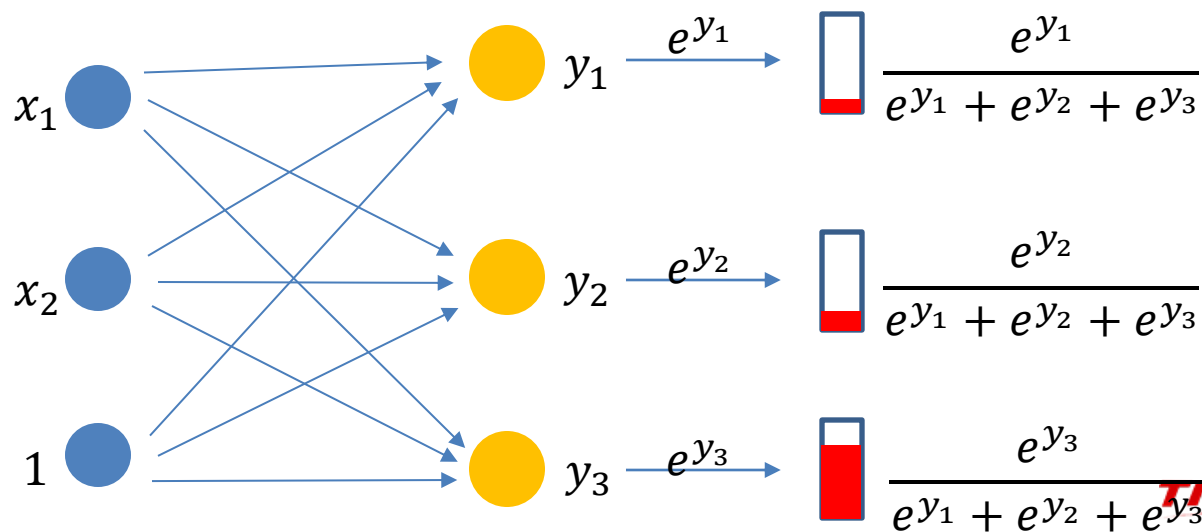


## Softmax 回归

■ 已经不是逻辑回归了



$$h_{\theta}(x^{(i)}) = \begin{bmatrix} p(y^{(i)} = 1 | x^{(i)}; \theta) \\ p(y^{(i)} = 2 | x^{(i)}; \theta) \\ \vdots \\ p(y^{(i)} = k | x^{(i)}; \theta) \end{bmatrix} = \frac{1}{\sum_{j=1}^k e^{\theta_j^T x^{(i)}}} \begin{bmatrix} e^{\theta_1^T x^{(i)}} \\ e^{\theta_2^T x^{(i)}} \\ \vdots \\ e^{\theta_k^T x^{(i)}} \end{bmatrix}$$



## 小结

本节我们主要学习了逻辑回归的基本原理：

- **模型：**  $P(y = 1|x; \theta) = \frac{1}{1 + e^{-\theta^T x}} = h_{\theta}(x)$
- **损失函数：**  $J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left( y^{(i)} \log h_{\theta}(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)})) \right)$
- **优化求解：** 梯度下降法  $\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m \left( h_{\theta}(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$
- **正则化：** L2正则/L1正则
- **多分类：** ovr/ovo/softmax回归



## 混淆矩阵

**准确率 (Accuracy)**：模型正确分类的样本数占总样本数的比例。准确率直观地反映了模型整体的正确性，但当数据不平衡时，其参考价值会降低

**召回率 (Recall)**：实际为正类的样本中，被正确预测为正类的比例。召回率反映了模型对正类样本的识别能力，适用于对假负例成本敏感的场景。又称为**查全率/灵敏度**

|                 | Predicted Positive                  | Predicted Negative                                  |                                                   |
|-----------------|-------------------------------------|-----------------------------------------------------|---------------------------------------------------|
| Actual Positive | TP<br><i>True Positive</i>          | FN<br><i>False Negative</i>                         | Sensitivity<br>$\frac{TP}{(TP + FN)}$             |
| Actual Negative | FP<br><i>False Positive</i>         | TN<br><i>True Negative</i>                          | Specificity<br>$\frac{TN}{(TN + FP)}$             |
|                 | Precision<br>$\frac{TP}{(TP + FP)}$ | Negative Predictive Value<br>$\frac{TN}{(TN + FN)}$ | Accuracy<br>$\frac{TP + TN}{(TP + TN + FP + FN)}$ |

**精确率 (Precision)**：预测为正类的样本中，实际为正类的比例。精确率关注的是模型预测为正类的准确性，适用于对假正例成本敏感的场景

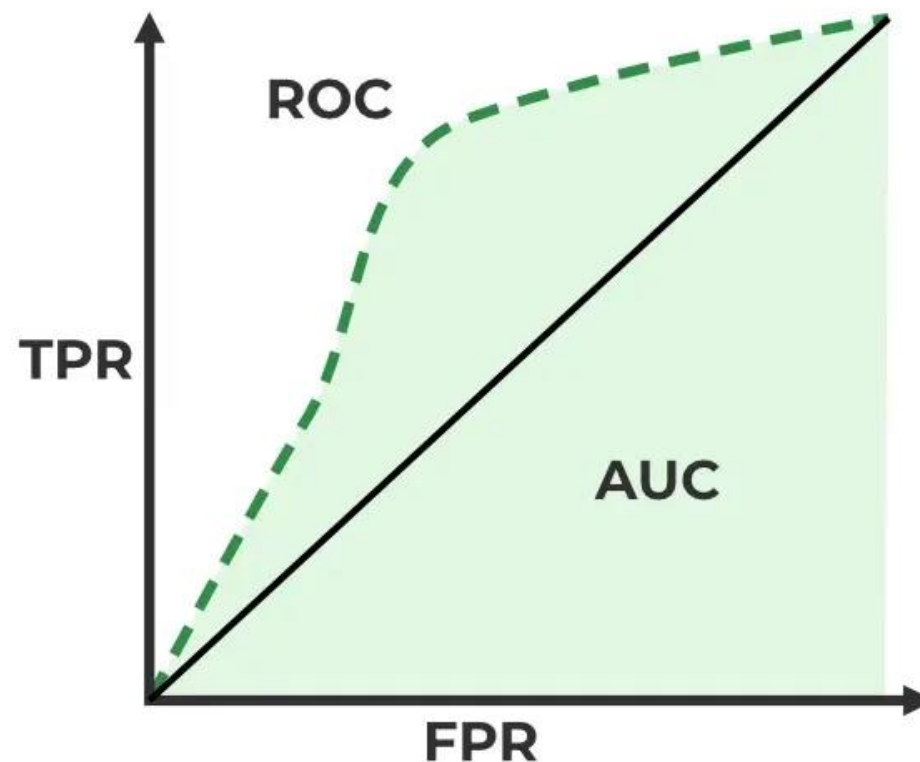
| 指标名称                                                | 公式                                                                      | 意义                                       |
|-----------------------------------------------------|-------------------------------------------------------------------------|------------------------------------------|
| 准确率 (Accuracy)                                      | $Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$                                  | 分类模型所有判断正确的结果占总观测值的比重。反映了模型整体的正确性。       |
| 精确率 (Precision) 又称之为查准率、命中率                         | $Precision = \frac{TP}{TP+FP}$                                          | 在模型观测是正类的所有结果中，模型预测对的比重。关注的是模型预测为正类的准确性。 |
| 召回率 (Recall) 或灵敏度 (Sensitivity) 又称之为TPR、灵敏度、查全率、覆盖率 | $Recall = \frac{TP}{TP+FN} = \frac{TP}{P}$                              | 在真实值是正类的所有结果中，模型预测对的比重。关注的是模型对正类样本的识别能力。 |
| 特异度 (Specificity)                                   | $Specificity = \frac{TN}{TN+FP} = \frac{TN}{N}$                         | 在真实值是负类的所有结果中，模型预测对的比重。关注的是模型对负类样本的识别能力。 |
| 假正率 (False Positive Rate, FPR) 又称之为取伪率、负样本召回率       | $FPR = \frac{FP}{TN+FP} = \frac{FP}{N}$                                 | 在真实值是负类的所有结果中，模型预测错（模型预测为正类）的比重。         |
| F1 Score                                            | $F1Score = \frac{2 \times Precision \times Recall}{Precision + Recall}$ | 精确率和召回率的调和平均数，综合考虑了精确率和召回率，适用于需要平衡两者的情况。 |

表格说明

- TP (True Positive)：实际为正类且被预测为正类的样本数。
- TN (True Negative)：实际为负类且被预测为负类的样本数。
- FP (False Positive)：实际为负类但被预测为正类的样本数。
- FN (False Negative)：实际为正类但被预测为负类的样本数。
- P (Positive)：实际为正类的样本总数。
- N (Negative)：实际为负类的样本总数。

# ROC 和 AUC

- **ROC 曲线**：通过显示真阳性率（TPR）与假阳性率（FPR）之间的权衡来评估分类模型的性能。
  - TPR（真阳性率）： $TPR = \frac{TP}{TP+FN}$
  - FPR（假阳性率）： $FPR = \frac{FP}{FP+TN}$
- **AUC（Area Under Curve）**：ROC 曲线下的面积，提供了一个定量的指标，用来衡量分类模型的整体表现。
  - AUC 越接近 1，表示分类器性能越好。
  - AUC 越接近 0.5，表示分类器性能与随机猜测无异。







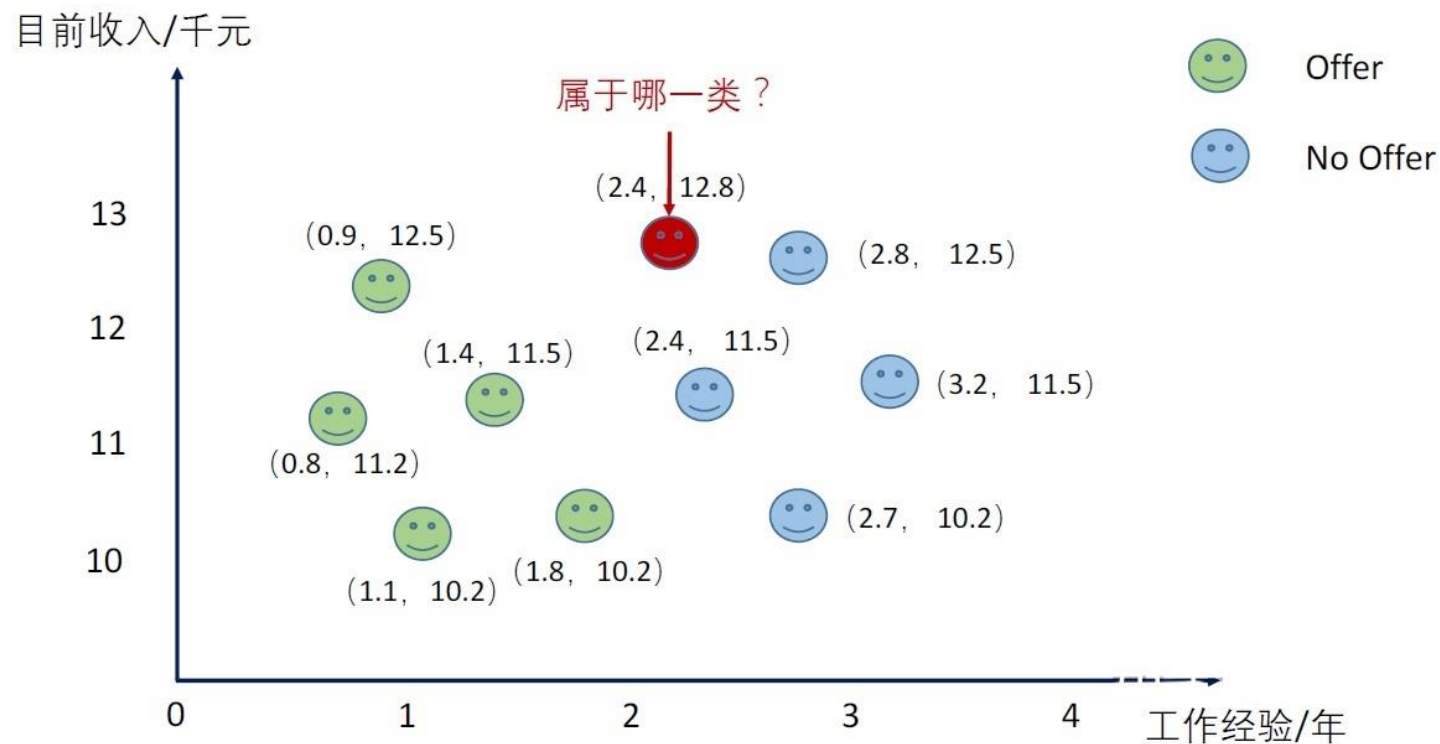
1  
3  
chapter

**K 近邻**

## K近邻算法原理

- 你的朋友好人多，你就是好人
- 你的朋友坏人多，你就是坏人

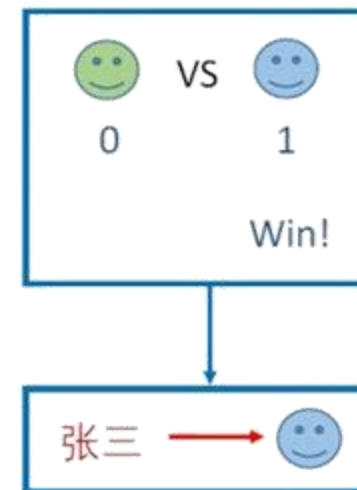
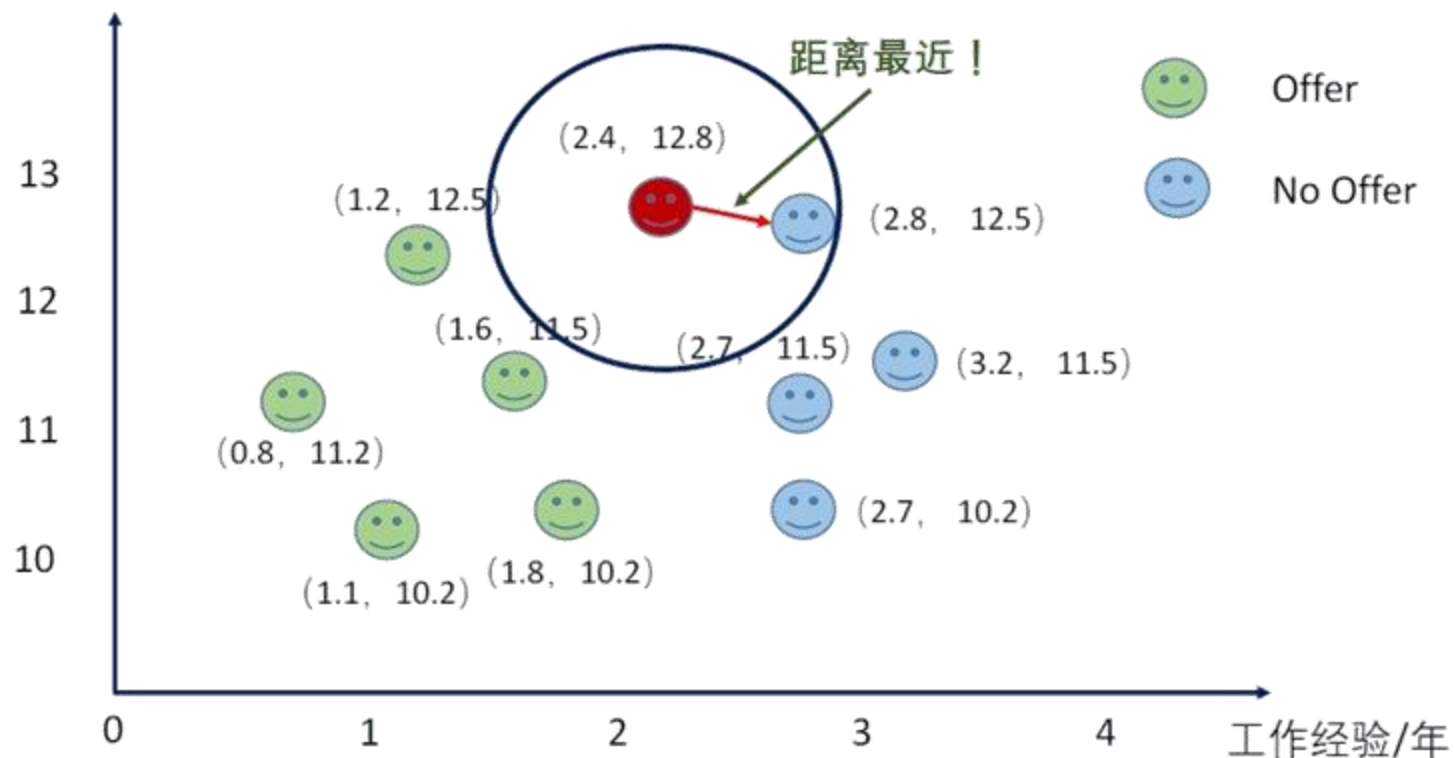
Q: 张三要参加一家公司的面试，但它通过各种渠道得知了拿到offer的人的情况和没有拿到offer的人的情况。我们一起来预测一下张三是否会拿到offer?



## K近邻算法原理

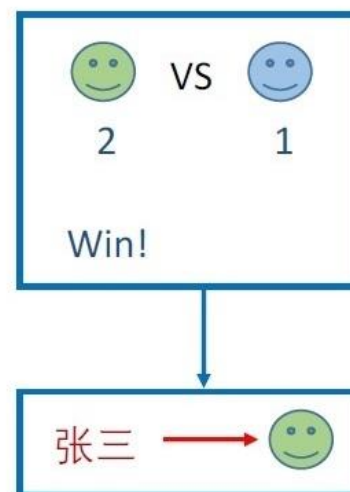
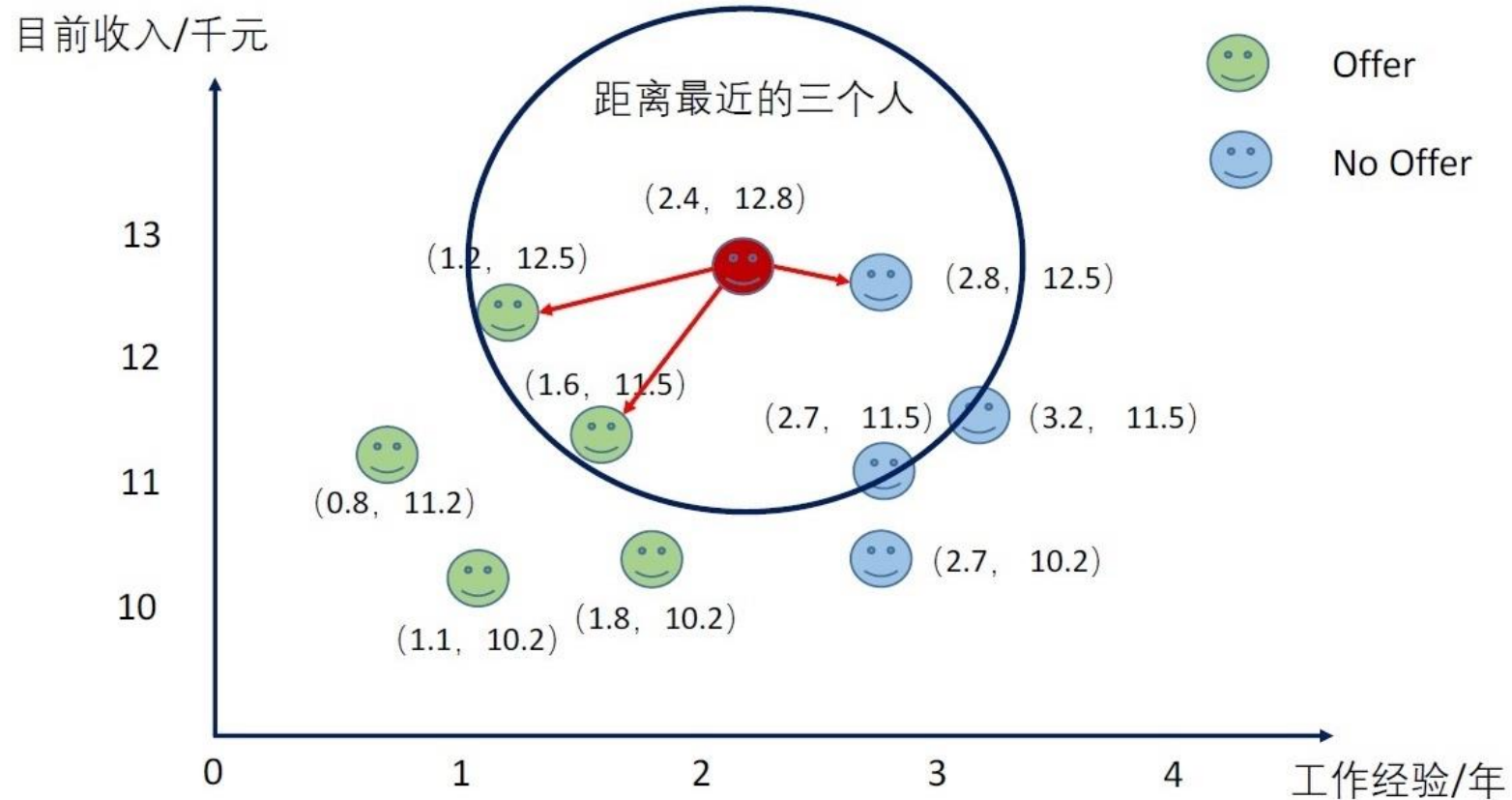
■ K=1

目前收入/千元



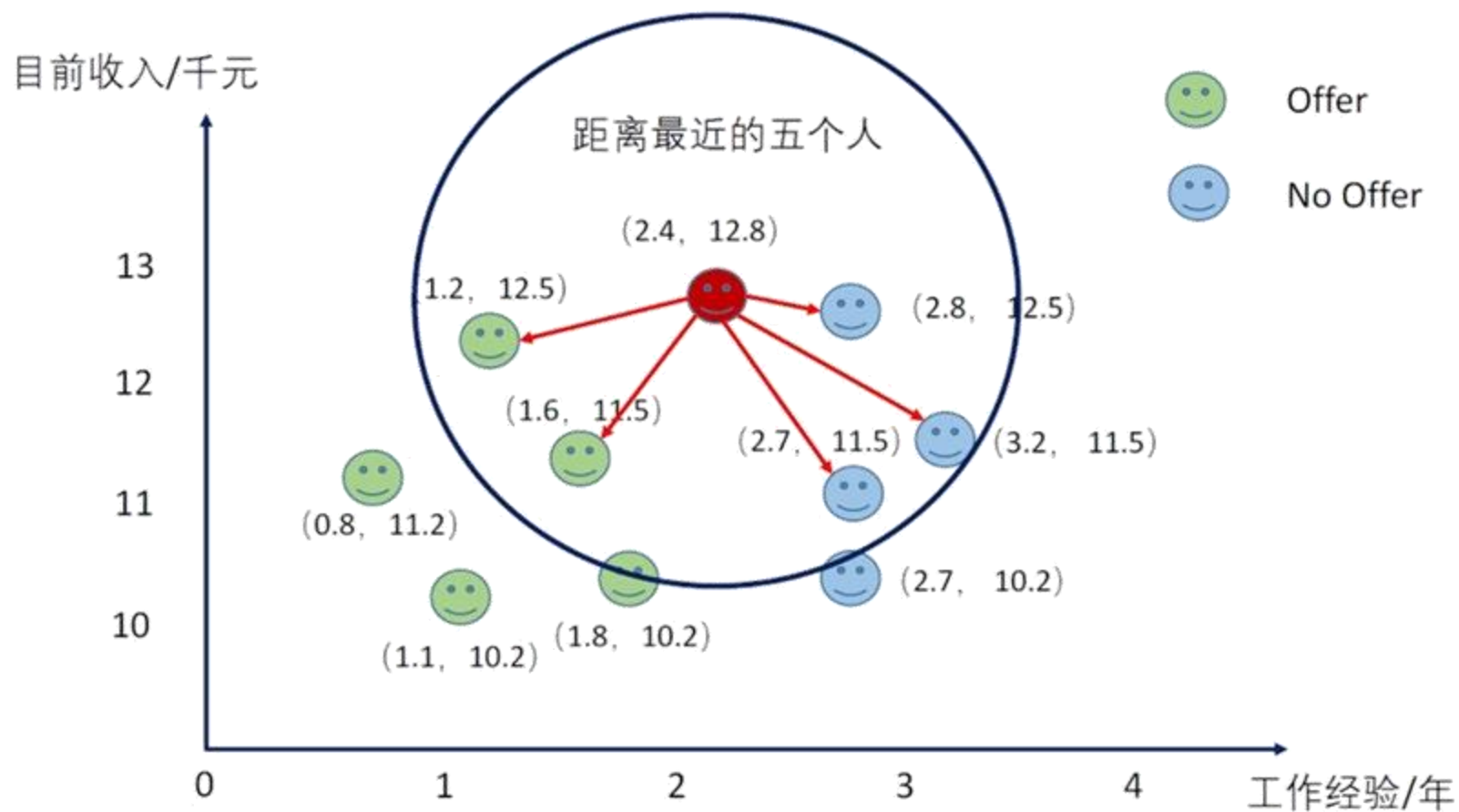
## K近邻算法原理

■ K=3



## K近邻算法原理

■ K=5



## 如何度量近邻:距离

- 欧氏距离

$$d(x, y) := \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_n - y_n)^2} = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

- 曼哈顿距离

$$d_{12} = |x_1 - x_2| + |y_1 - y_2|$$

- 切比雪夫距离

$$D_{\text{Chebyshev}}(p, q) := \max_i (|p_i - q_i|).$$

- 闵可夫斯基距离

$$d_{12} = \sqrt[p]{\sum_{k=1}^n |x_{1k} - x_{2k}|^p}$$

- 马氏距离

$$D(X_i, X_j) = \sqrt{(X_i - X_j)^T S^{-1} (X_i - X_j)}$$

■ 基于距离的算法必须对数据做 **标准化**

- 汉明距离

两个等长字符串 s1 与 s2 之间的汉明距离定义为将其中一个变为另外一个所需要作的最小替换次数。例如字符串“1111”与“1001”之间的汉明距离为 2

- 夹角余弦

$$\cos\theta = \frac{x_1x_2 + y_1y_2}{\sqrt{x_1^2 + y_1^2} \sqrt{x_2^2 + y_2^2}}$$

或者

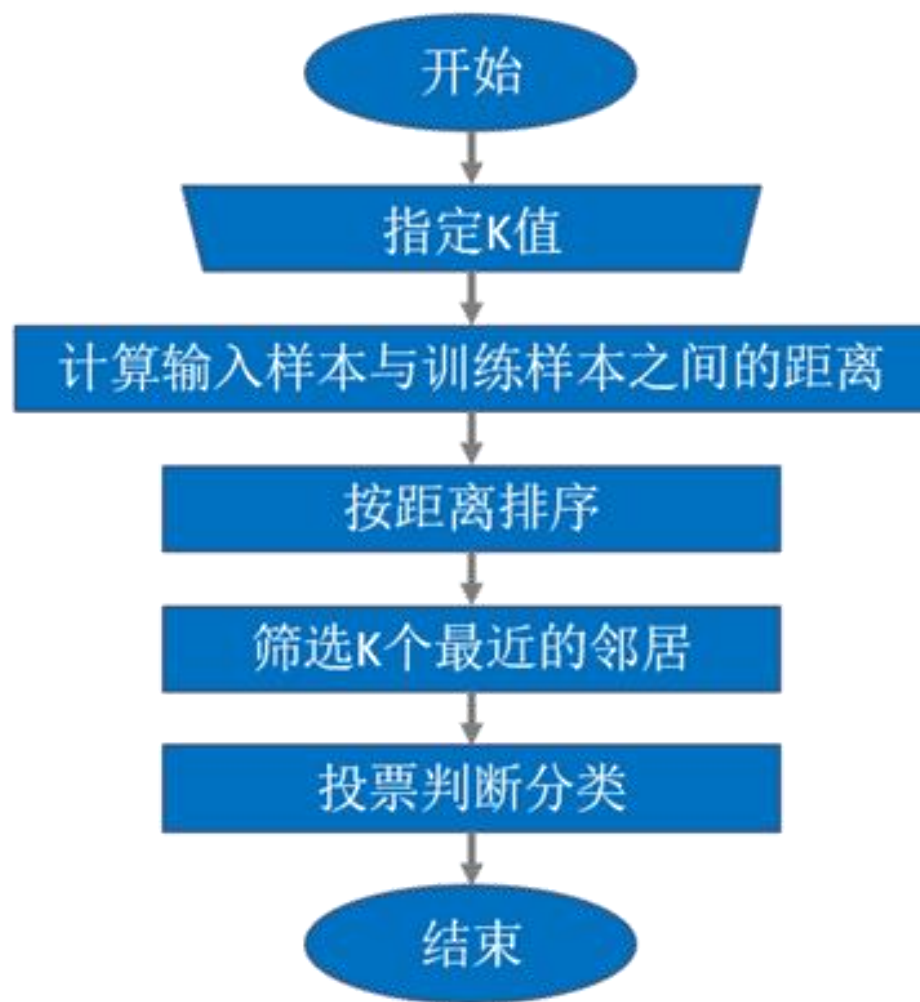
$$\cos(\theta) = \frac{a \cdot b}{|a| |b|}$$

- 皮尔逊相关系数

$$\rho_{XY} = \frac{\text{Cov}(X, Y)}{\sqrt{D(X)} \sqrt{D(Y)}} = \frac{E((X - EX)(Y - EY))}{\sqrt{D(X)} \sqrt{D(Y)}}$$



## K 近邻算法计算流程



## K近邻算法计算案例

■ **任务描述**：对 John 的违约标记 (Yes/No) 进行预测。

数据集

| 客户ID   | 年龄 | 收入   | 信用卡张数 | 违约标记 |
|--------|----|------|-------|------|
| George | 35 | 35K  | 3     | No   |
| Rachel | 22 | 50K  | 2     | Yes  |
| Steve  | 63 | 200K | 1     | No   |
| Tom    | 59 | 170K | 1     | No   |
| Anne   | 25 | 40K  | 4     | Yes  |
| John   | 37 | 50K  | 2     | ?    |





## K近邻算法计算案例

### ■ Step 1,2: 指定 K 值, 计算预测样本与训练样本的距离

Step 1: 选择k值:  $k=3$

Step 2: 计算距离



|      |    |     |   |   |
|------|----|-----|---|---|
| John | 37 | 50K | 2 | ? |
|------|----|-----|---|---|

| 客户ID   | 年龄 | 收入   | 信用卡张数 | 违约标记 | John与其他客户的距离 (欧式距离)                                       |
|--------|----|------|-------|------|-----------------------------------------------------------|
| George | 35 | 35K  | 3     | No   | $\text{sqrt} [(35-37)^2 + (35-50)^2 + (3-2)^2] = 15.16$   |
| Rachel | 22 | 50K  | 2     | Yes  | $\text{sqrt} [(22-37)^2 + (50-50)^2 + (2-2)^2] = 15$      |
| Steve  | 63 | 200K | 1     | No   | $\text{sqrt} [(63-37)^2 + (200-50)^2 + (1-2)^2] = 152.23$ |
| Tom    | 59 | 170K | 1     | No   | $\text{sqrt} [(59-37)^2 + (170-50)^2 + (1-2)^2] = 122$    |
| Anne   | 25 | 40K  | 4     | Yes  | $\text{sqrt} [(25-37)^2 + (40-50)^2 + (4-2)^2] = 15.74$   |

## K近邻算法计算案例

## ■ Step 3: 对距离排序，找出距离最近的 k 个邻居

| 客户ID   | 年龄 | 收入   | 信用卡张数 | 违约标记 | 距离排序 | John与其他客户的距离（欧式距离）                                        |
|--------|----|------|-------|------|------|-----------------------------------------------------------|
| George | 35 | 35K  | 3     | No   | 2    | $\text{sqrt} [(35-37)^2 + (35-50)^2 + (3-2)^2] = 15.16$   |
| Rachel | 22 | 50K  | 2     | Yes  | 1    | $\text{sqrt} [(22-37)^2 + (50-50)^2 + (2-2)^2] = 15$      |
| Steve  | 63 | 200K | 1     | No   | 5    | $\text{sqrt} [(63-37)^2 + (200-50)^2 + (1-2)^2] = 152.23$ |
| Tom    | 59 | 170K | 1     | No   | 4    | $\text{sqrt} [(59-37)^2 + (170-50)^2 + (1-2)^2] = 122$    |
| Anne   | 25 | 40K  | 4     | Yes  | 3    | $\text{sqrt} [(25-37)^2 + (40-50)^2 + (4-2)^2] = 15.74$   |

## K近邻算法计算案例

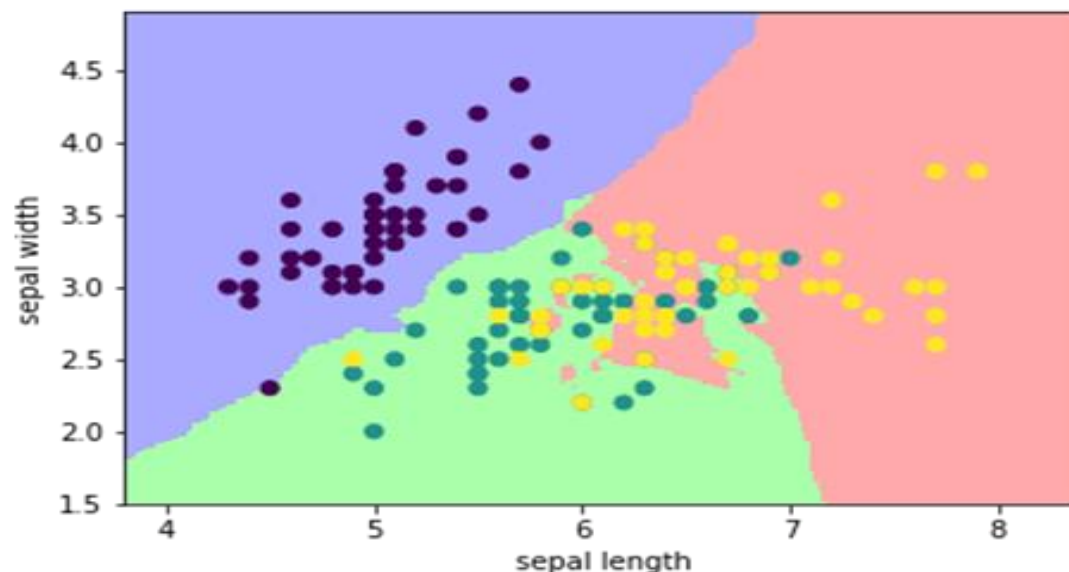
## ■ Step 4: 根据 k 个邻居的类别值投票做出预测判断

| 客户ID   | 年龄 | 收入   | 信用卡张数 | 违约标记 | 距离排序 | John与其他客户的距离（欧式距离）                                        |
|--------|----|------|-------|------|------|-----------------------------------------------------------|
| George | 35 | 35K  | 3     | No   | 2    | $\text{sqrt} [(35-37)^2 + (35-50)^2 + (3-2)^2] = 15.16$   |
| Rachel | 22 | 50K  | 2     | Yes  | 1    | $\text{sqrt} [(22-37)^2 + (50-50)^2 + (2-2)^2] = 15$      |
| Steve  | 63 | 200K | 1     | No   | 5    | $\text{sqrt} [(63-37)^2 + (200-50)^2 + (1-2)^2] = 152.23$ |
| Tom    | 59 | 170K | 1     | No   | 4    | $\text{sqrt} [(59-37)^2 + (170-50)^2 + (1-2)^2] = 122$    |
| Anne   | 25 | 40K  | 4     | Yes  | 3    | $\text{sqrt} [(25-37)^2 + (40-50)^2 + (4-2)^2] = 15.74$   |

|      |    |     |   |     |
|------|----|-----|---|-----|
| John | 37 | 50K | 2 | YES |
|------|----|-----|---|-----|

## K近邻算法特点

- **惰性算法**：没有训练,只有预测（预测的时候进行近邻计算）
- **预测效率低**：预测时需要大量的距离计算，非常耗时（特别是在样本量较大的时候）
- **预测性能好**：非线性模型，效果好
- **可解释性差**：你是好人还是坏人跟你是什么样的人没啥关系（抛弃原始特征,在映射后的高维空间建模：近邻法）



## K近邻算法调参：K值设定为多大？

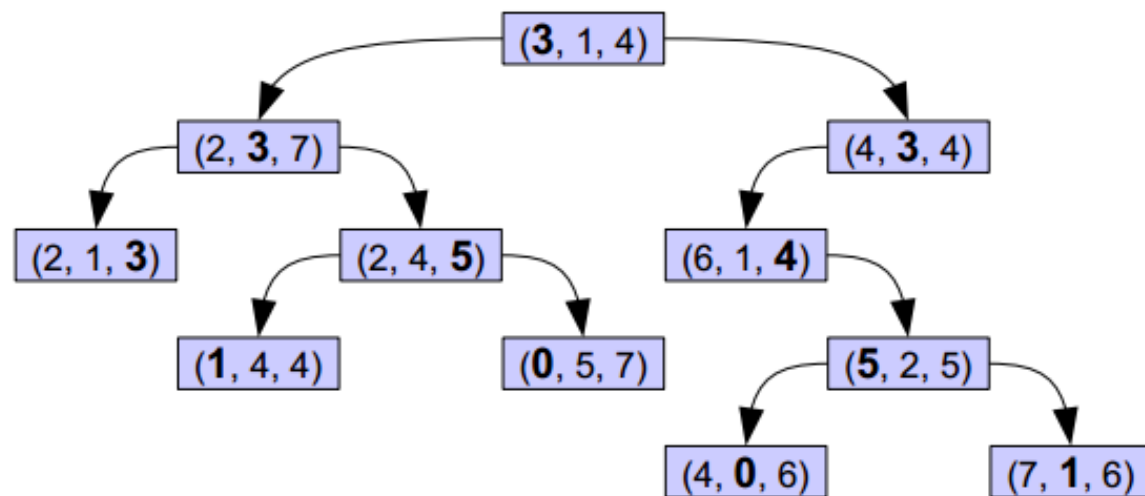
- K 太小，分类结果易受噪声点影响；k 太大，近邻中又可能包含太多的其它类别的点（**对距离加权**，可以降低 k 值设定的影响）。
- K 值通常是采用**交叉验证**来确定（以 $k = 1$ 为基准）
- **经验规则**：k 一般低于训练样本数的平方根

## 搜索近邻的方式

- **线性搜索**：计算预测样本和每一个训练样本的距离，暴力搜索，最耗时；
- **Kdtree**：训练时预先使用训练样本构建 kdtree（思想：**A 距离 B 很远, B 跟 C 很近, A 和 C 应该也很远**），预测时使用 kdtree 加速搜索，只需要计算预测样本和部分训练样本的距离；
- **Balltree**：kdtree 的改进版本，解决 kdtree 在高维（20 维以上）失效的问题。

## kdtree

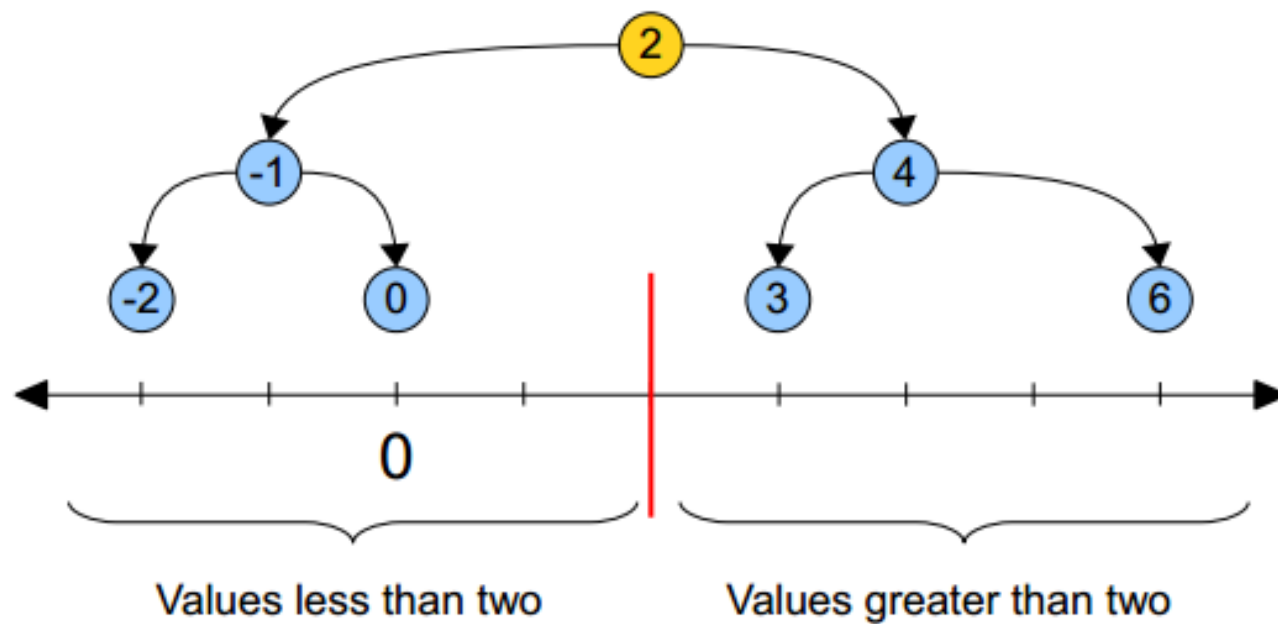
- Kd 树 (k-dimensional 树的简称) , 是一种分割 k 维数据空间的数据结构。主要应用于多维空间关键数据的搜索 (如: 范围搜索和最近邻搜索) 。



上图的树就是一棵KDTree, 形似二叉搜索树, 其实KDTree就是二叉搜索树的变种。这里的 $K = 3$ .

## 一维情况: BSTree

- 每个节点只有一个key值
- 二分法



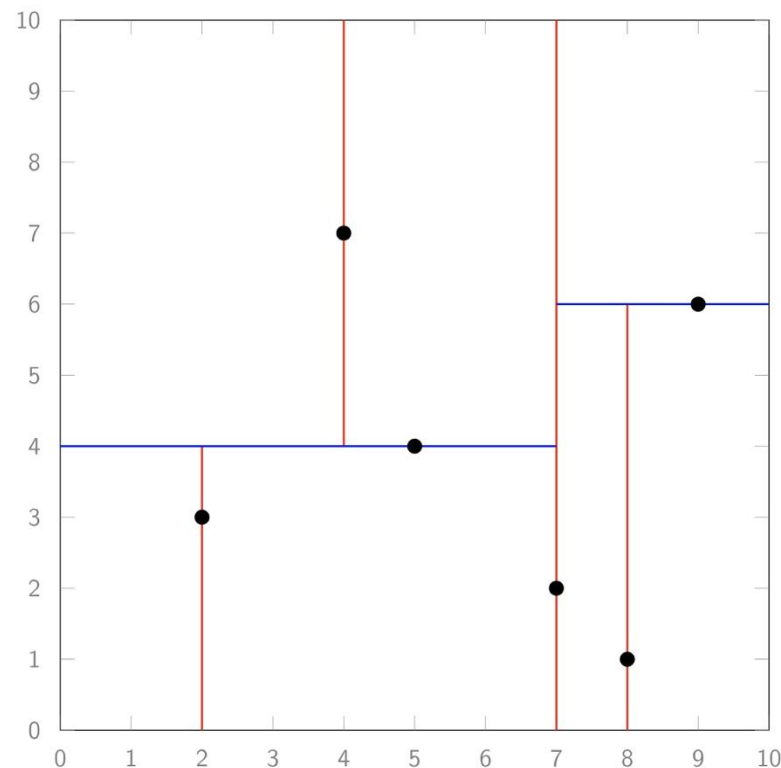
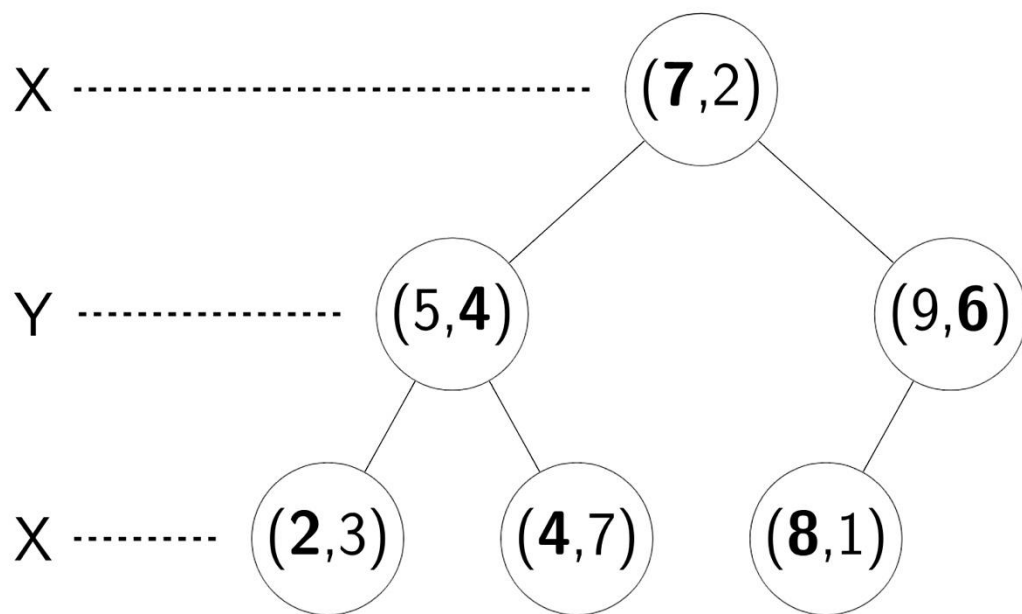
这样的分割可以扩展到二维甚至更多维的情况吗？



## KDTree

二维的节点怎么比较大小?

- KDTree 就是超平面都垂直于轴的 BSPTree (binary space partitioning tree)
- 思想: 如果 A 和 B 距离很远, B 和 C 距离很近, 那么 A 和 C 的距离也很远



## KDTree的两个重要步骤

- 树的建立;
- 最近邻域搜索 (Nearest-Neighbor Lookup)

## 树的建立

1. 构造根结点，使根结点对应于K维空间中包含所有实例点的超矩形区域；
2. 通过递归的方法，不断地对k维空间进行切分，生成子结点。
3. 上述过程直到子区域内没有实例时终止（终止时的结点为叶结点）

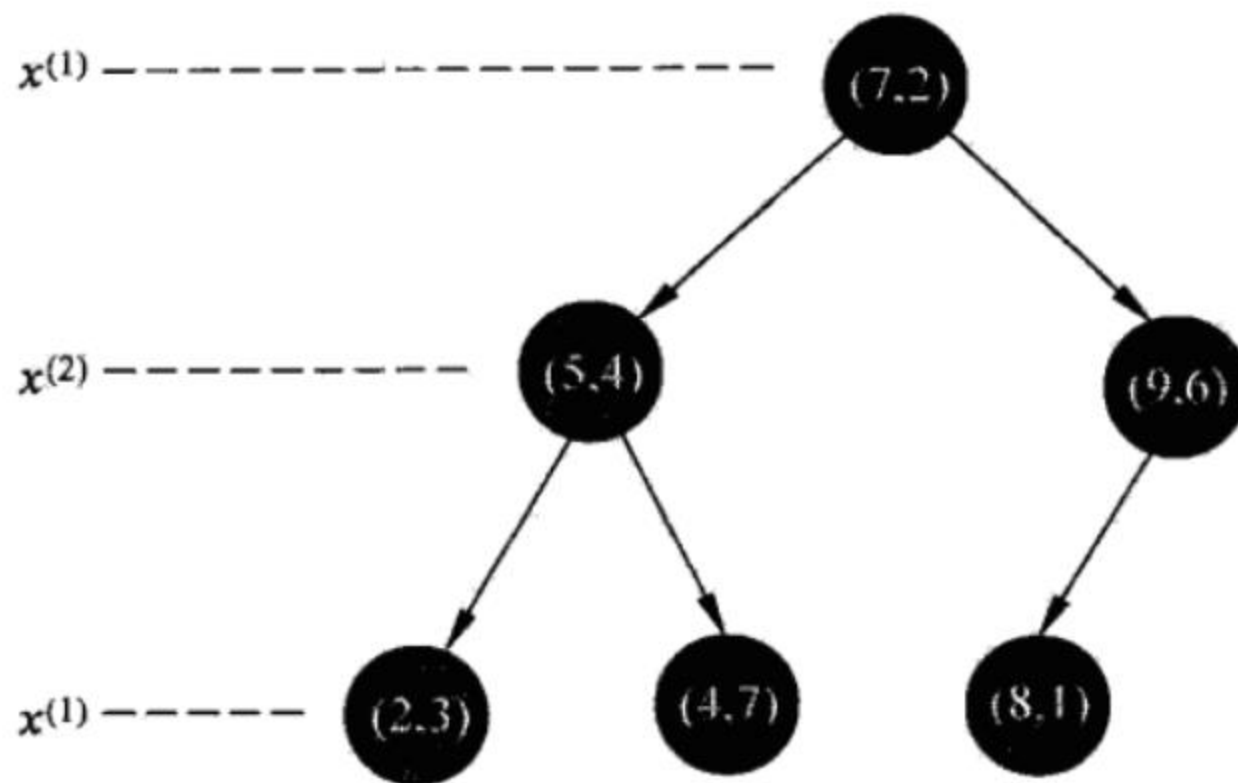
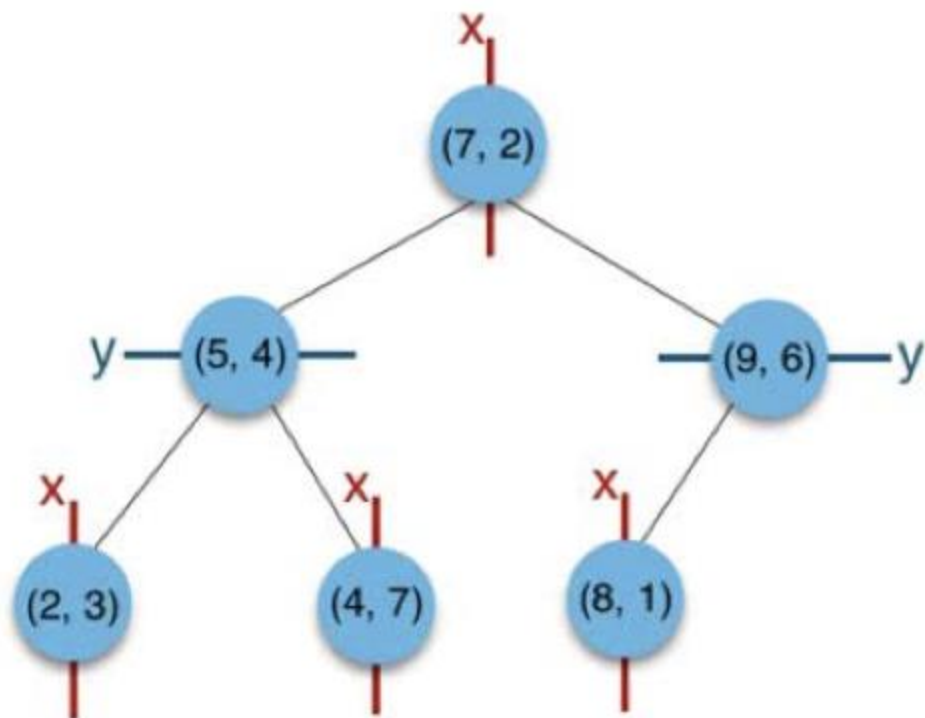


## 树的建立

- 问题1： 每次对子空间的划分时，怎样确定在哪个维度上进行划分？
  - 轴交替：各个轴轮流切分
  - **最大方差法**：选择方差最大的轴
- 问题2： 在某个维度上进行划分时，怎样确保在这一维度上的划分得到的两个子集合的数量尽量相等，即左子树和右子树中的结点个数尽量相等？
  - 使用**中位数**进行划分

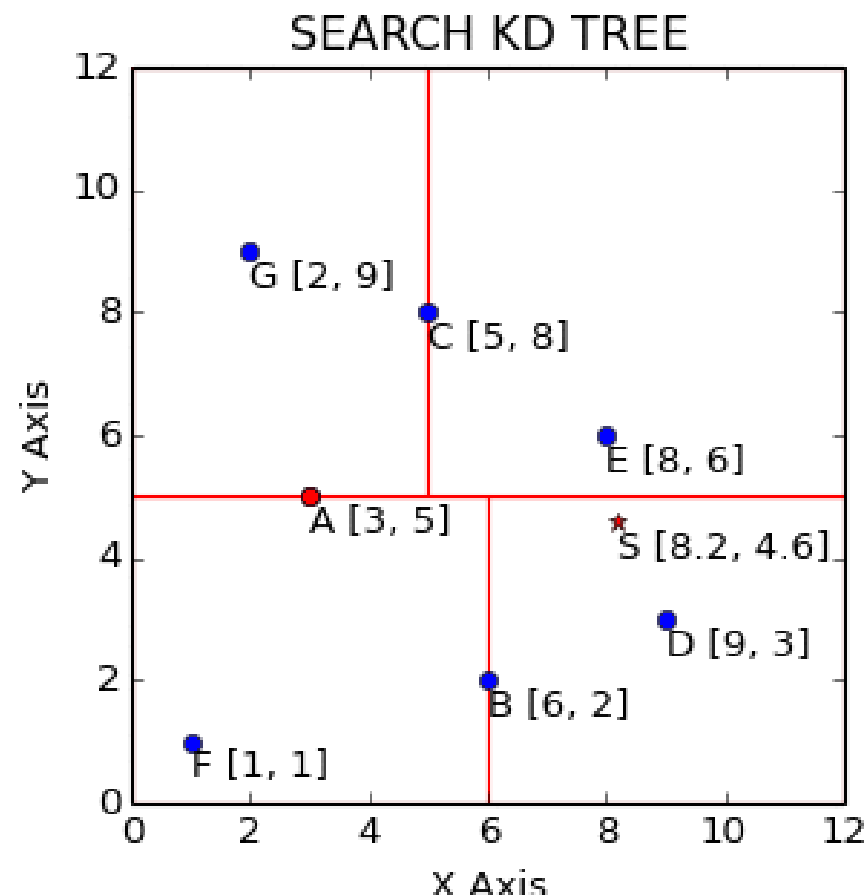
## 树的建立案例

- 给定一个二维空间数据集:  $T=\{(2,3),(5,4),(9,6),(4,7),(8,1),(7,2)\}$ , 构造一个平衡kd树

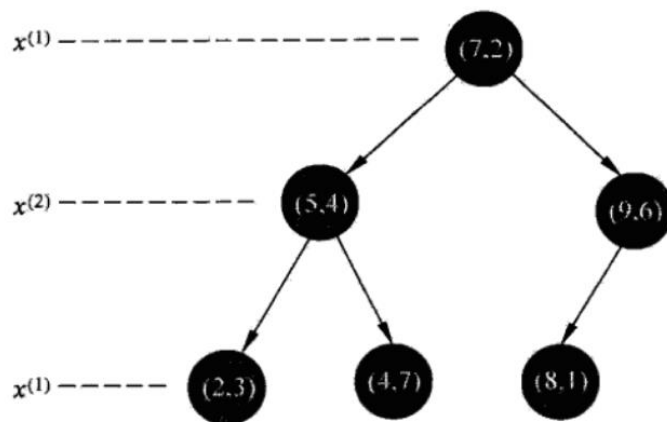
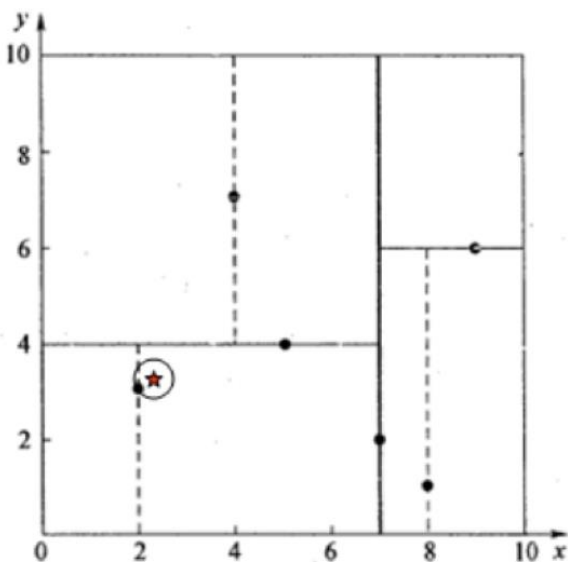


## 最近邻域搜索

- 假设标记为星星的点是 test point, 绿色的点是找到的近似点, 在回溯过程中, 需要用到一个队列, 存储需要回溯的点, 在判断其他子节点空间中是否有可能有距离查询点更近的数据点时, 做法是以查询点为圆心, 以当前的最近距离为半径画圆, 这个圆称为候选超球 (candidate hypersphere), 如果圆与回溯点的轴相交, 则需要将轴另一边的节点都放到回溯队列里面来。

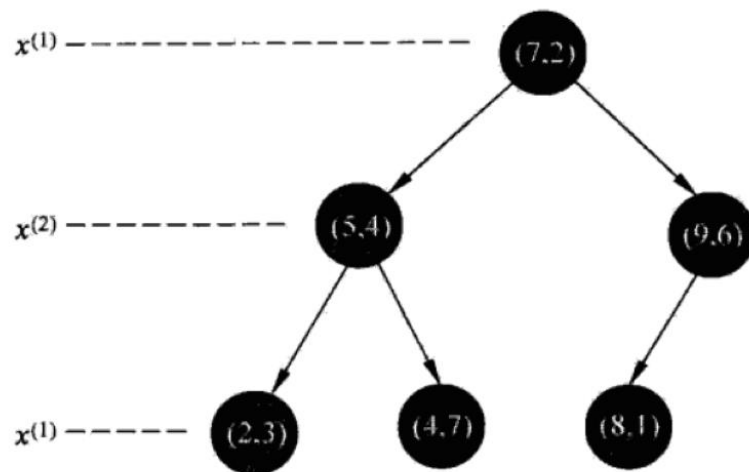
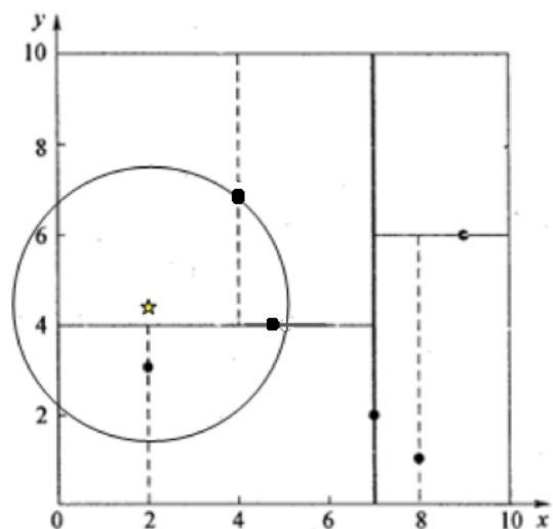


## 最近邻域搜索:查找点(2.1,3.1)



- 在(7, 2)点测试到达(5, 4)，在(5, 4)点测试到达(2, 3)，然后search\_path中的结点为 $\langle (7, 2), (5, 4), (2, 3) \rangle$ ，从search\_path中取出(2, 3)作为当前最佳结点nearest，dist为0.141；
- 然后回溯至(5, 4)，以(2.1, 3.1)为圆心，以dist=0.141为半径画一个圆，并不和超平面 $y=4$ 相交，如上图，所以不必跳到结点(5, 4)的右子空间去搜索，因为右子空间中不可能有更近样本点了。
- 于是再回溯至(7, 2)，同理，以(2.1, 3.1)为圆心，以dist=0.141为半径画一个圆并不和超平面 $x=7$ 相交，所以也不用跳到结点(7, 2)的右子空间去搜索。
- 至此，search\_path为空，结束整个搜索，返回nearest(2, 3)作为(2.1, 3.1)的最近邻点，最近距离为0.141。

## 最近邻域搜索:查找点(2,4.5)



- 在 (7, 2) 处测试到达 (5, 4)，在 (5, 4) 处测试到达 (4, 7)  
【优先选择在本域搜索】，然后search\_path中的结点为  $\langle (7, 2), (5, 4), (4, 7) \rangle$ ，从search\_path中取出 (4, 7) 作为当前最佳结点nearest，dist为3.202；
- 然后回溯至 (5, 4)，以 (2, 4.5) 为圆心，以dist=3.202为半径画一个圆与超平面 $y=4$ 相交，所以需要跳到 (5, 4) 的左子空间去搜索。所以要将 (2, 3) 加入到search\_path中，现在search\_path中的结点为  $\langle (7, 2), (2, 3) \rangle$ ；另外，(5, 4) 与 (2, 4.5) 的距离为  $3.04 < \text{dist} = 3.202$ ，所以将 (5, 4) 赋给nearest，并且dist=3.04。
- 回溯至 (2, 3)，(2, 3) 是叶子节点，直接判断 (2, 3) 是否离 (2, 4.5) 更近，计算得到距离为1.5，所以nearest更新为 (2, 3)，dist更新为 (1.5)
- 回溯至 (7, 2)，同理，以 (2, 4.5) 为圆心，以dist=1.5为半径画一个圆并不和超平面 $x=7$ 相交，所以不用跳到结点 (7, 2) 的右子空间去搜索。
- 至此，search\_path为空，结束整个搜索，返回nearest (2, 3) 作为 (2, 4.5) 的最近邻点，最近距离为1.5。

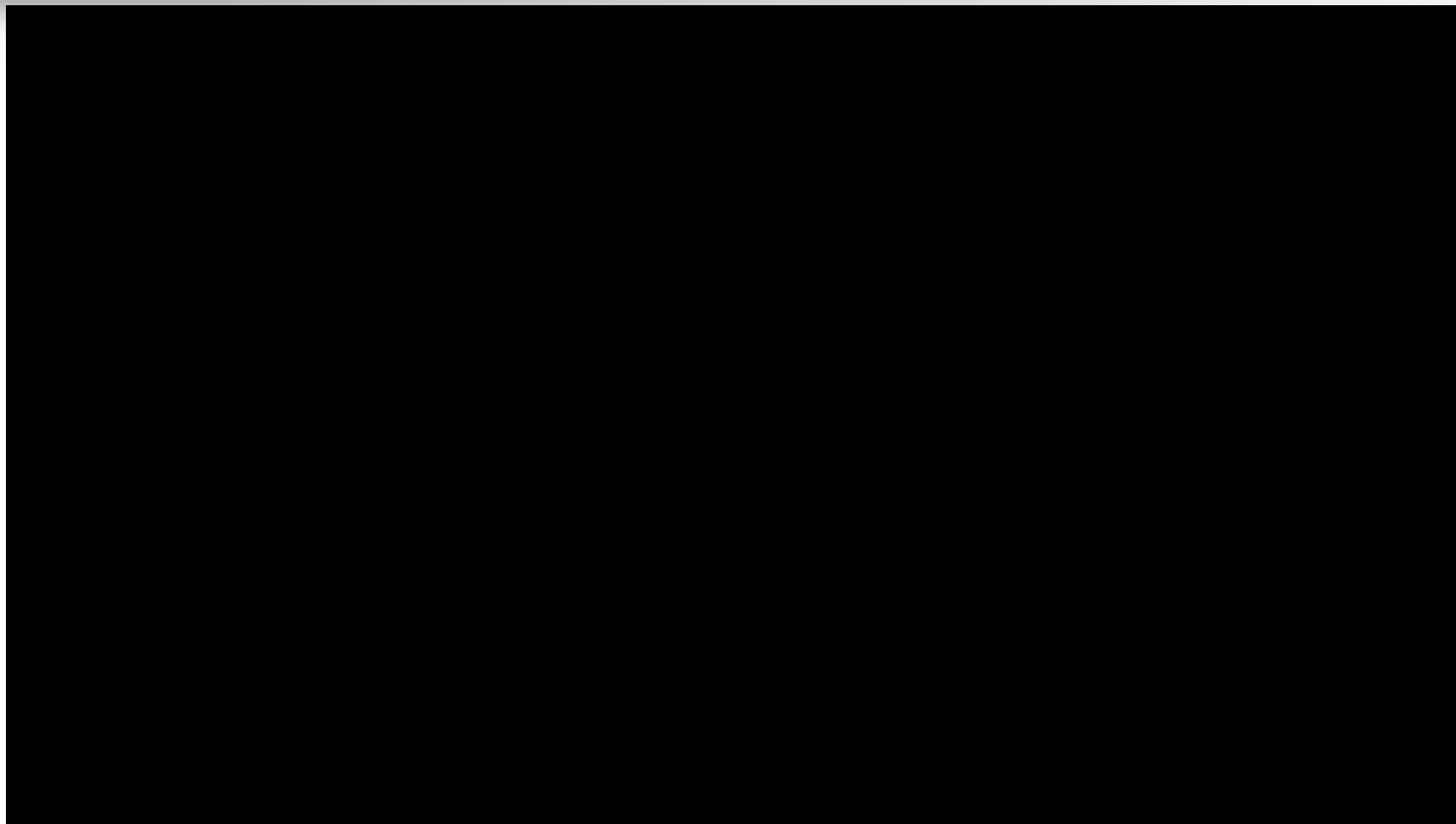




1  
4  
chapter

# 支持向量机

## Support Vector Machine (SVM) in 2 minutes



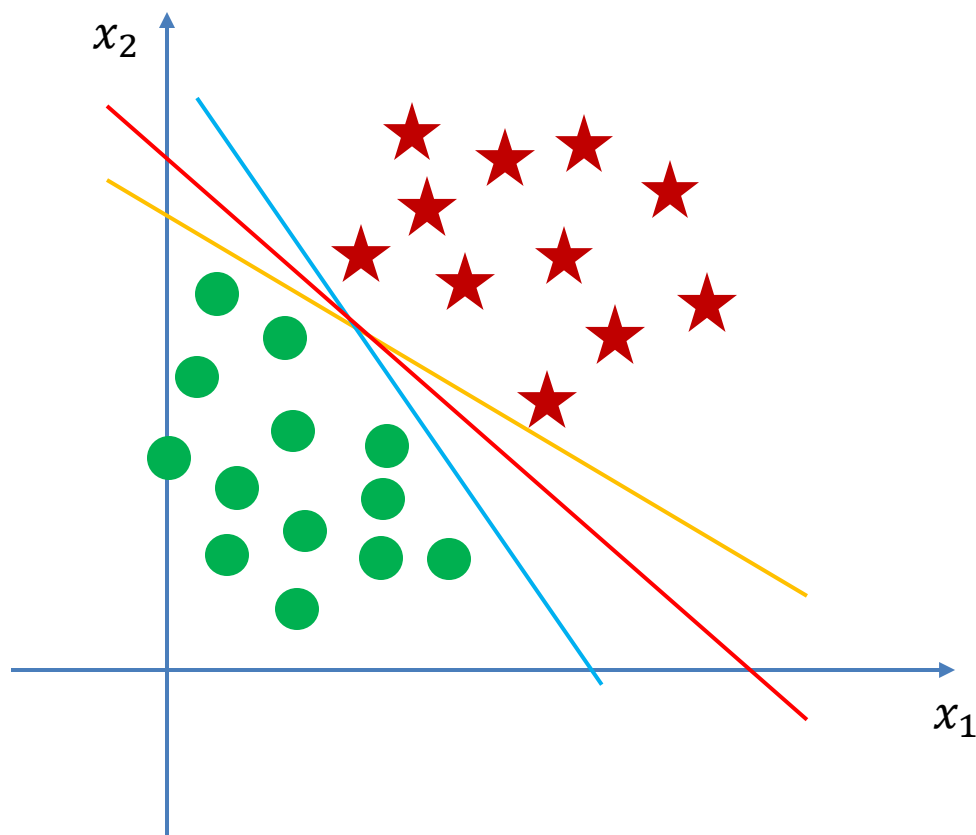
## 支持向量机原理导读

### 支持向量机

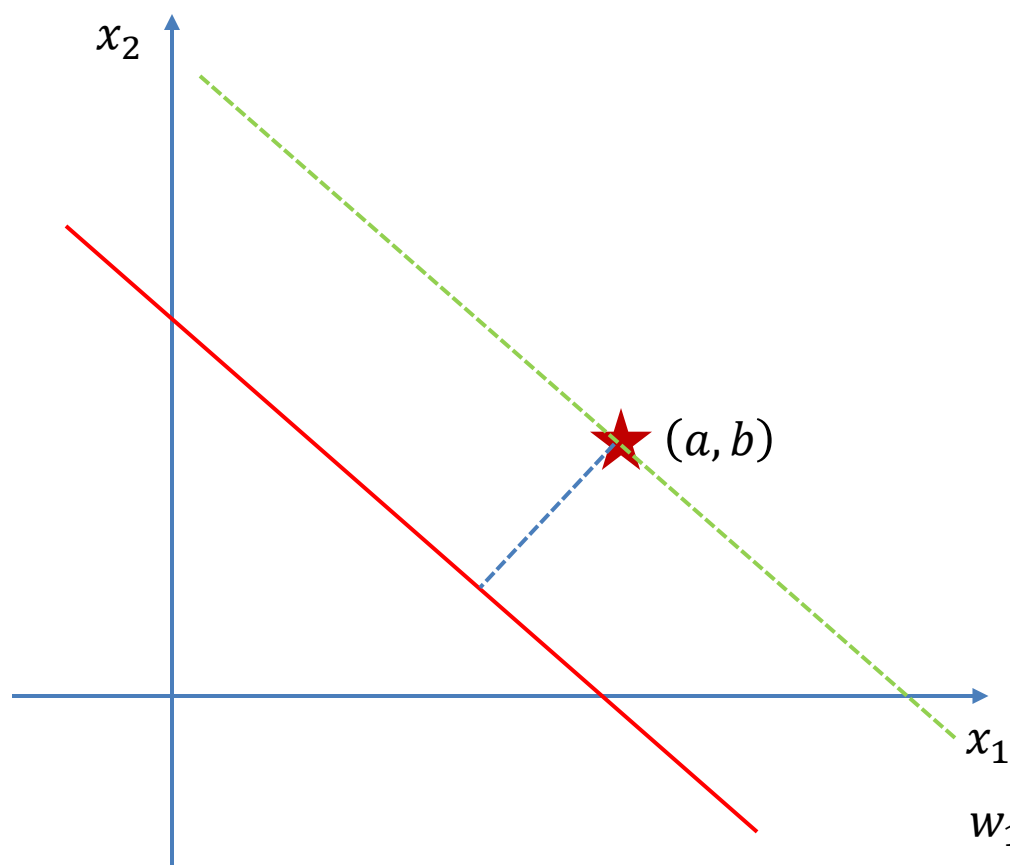


## 从线性分类说起

- 跟逻辑回归一样，试图寻找一条线将样本分隔开
- 不同点在于寻找的方式不同



## 回顾：平行线间的距离

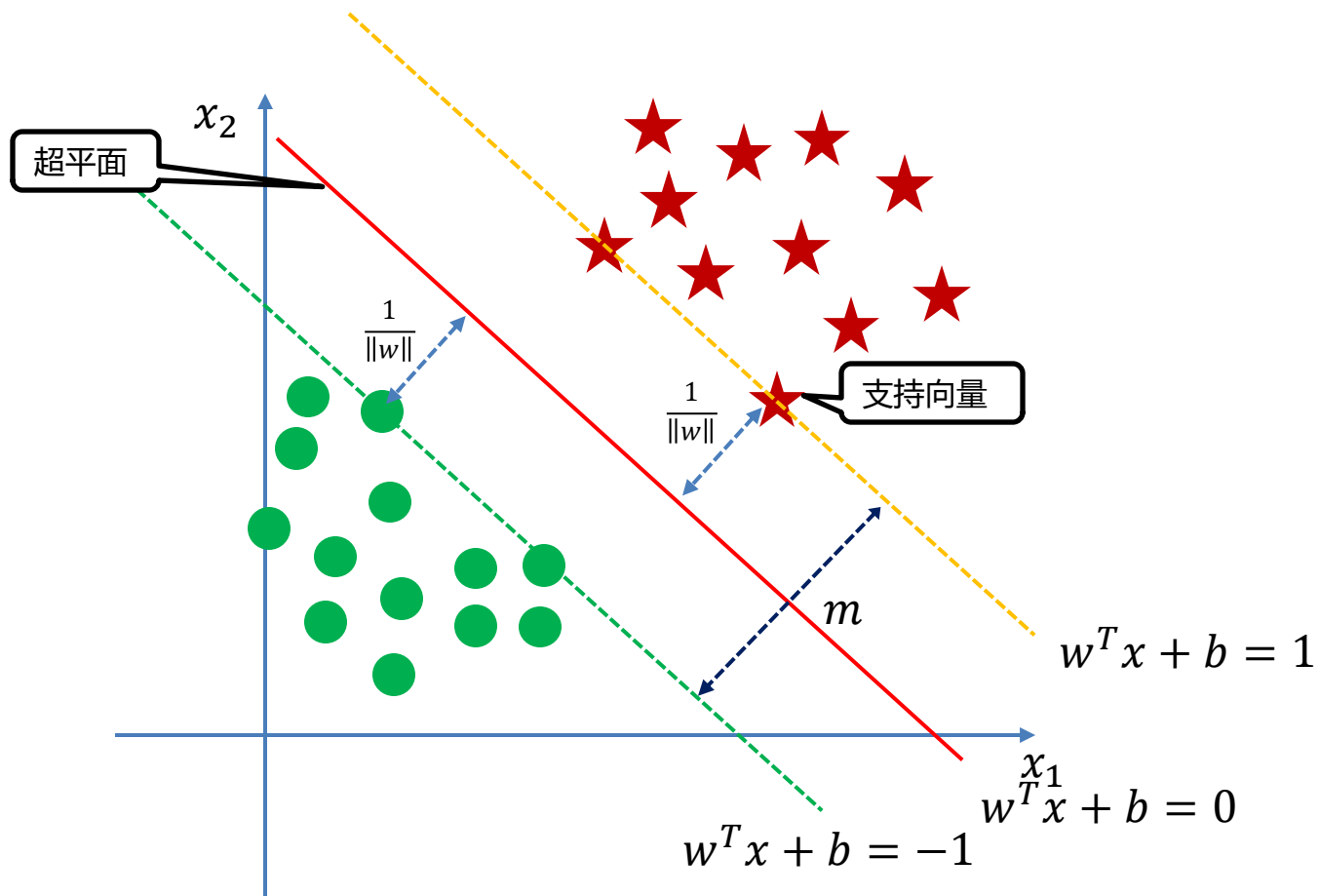


$$w_1x_1 + w_2x_2 + b = 0$$

$$w_1x_1 + w_2x_2 + b = ?$$

$$dist = \frac{? - 0}{\sqrt{w_1^2 + w_2^2}} = \frac{? - 0}{\|w\|}$$

## 支持向量机建模思想：最大间隔



$$\max_{w,b} \frac{2}{\|w\|}$$

$s.t. \quad y^{(i)}(w^T x^{(i)} + b) \geq 1, i = 1, 2, \dots, n$

## 支持向量机建模思想：问题转换1

$$\max_{w,b} \frac{2}{\|w\|}$$



$$\min_{w,b} \frac{1}{2} \|w\|^2$$

$$s.t. \quad y^{(i)}(w^T x^{(i)} + b) \geq 1, i = 1, 2, \dots, n$$

$$s.t. \quad y^{(i)}(w^T x^{(i)} + b) \geq 1, i = 1, 2, \dots, n$$

## 支持向量机建模思想：问题转换2

$$\min_{w,b} \frac{1}{2} \|w\|^2$$



拉格朗日乘子法

$$s.t. \quad y^{(i)}(w^T x^{(i)} + b) \geq 1 \\ i = 1, 2, \dots, n$$

$$L(w, b, a) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^n a^{(i)} (1 - y^{(i)}(w^T x^{(i)} + b))$$

$$s.t. \quad \begin{cases} a^{(i)} \geq 0 \\ y^{(i)}(w^T x^{(i)} + b) - 1 \geq 0 \\ a^{(i)} (1 - y^{(i)}(w^T x^{(i)} + b)) = 0 \end{cases}$$



## 支持向量机建模思想：问题转换3

$$\min_{w,b} \max_a L(w, b, a)$$

$$\begin{cases} a^{(i)} \geq 0 \\ y^{(i)} (f(x^{(i)})) - 1 \geq 0 \\ a^{(i)} (1 - f(x^{(i)})) = 0 \end{cases}$$

KKT条件、对偶问题

$$\max_a \min_{w,b} L(w, b, a)$$

支持向量机建模思想：模型求解第一步  $\max_a \min_{w,b} L(w, b, a)$

■ 第一步：将 $\alpha$ 看成常量，对 $w$ 和 $b$ 求导=0，求得 $w$ 和 $b$

$$L(w, b, a) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^n a^{(i)} (1 - y^{(i)} (w^T x^{(i)} + b))$$

$$\frac{\partial L}{\partial w} = w - \sum_{i=1}^n a^{(i)} y^{(i)} x^{(i)} = 0 \quad \longrightarrow \quad w = \sum_{i=1}^n a^{(i)} y^{(i)} x^{(i)}$$

$$\frac{\partial L}{\partial b} = - \sum_{i=1}^n a^{(i)} y^{(i)} = 0 \quad \longrightarrow \quad 0 = \sum_{i=1}^n a^{(i)} y^{(i)}$$

## 支持向量机建模思想：模型求解第二步

■ 第二步：将w带入，消除w和b

$$\begin{aligned}
 L(w, b, a) &= \frac{1}{2} \|w\|^2 + \sum_{i=1}^n a^{(i)} (1 - y^{(i)}(w^T x^{(i)} + b)) \\
 &= \frac{1}{2} w^T w - w^T \sum_{i=1}^n a^{(i)} y^{(i)} x^{(i)} - b \sum_{i=1}^n a^{(i)} y^{(i)} + \sum_{i=1}^n a^{(i)} \\
 &= \frac{1}{2} w^T \sum_{i=1}^n a^{(i)} y^{(i)} x^{(i)} - w^T \sum_{i=1}^n a^{(i)} y^{(i)} x^{(i)} - b * 0 + \sum_{i=1}^n a^{(i)} \\
 &= -\frac{1}{2} \left( \sum_{i=1}^n a^{(i)} y^{(i)} x^{(i)} \right)^T \sum_{i=1}^n a^{(i)} y^{(i)} x^{(i)} + \sum_{i=1}^n a^{(i)} \\
 &= \sum_{i=1}^n a^{(i)} - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a^{(i)} a^{(j)} y^{(i)} y^{(j)} a^{(i)} x^{(i)T} x^{(j)}
 \end{aligned}$$

$$w = \sum_{i=1}^n a^{(i)} y^{(i)} x^{(i)}$$

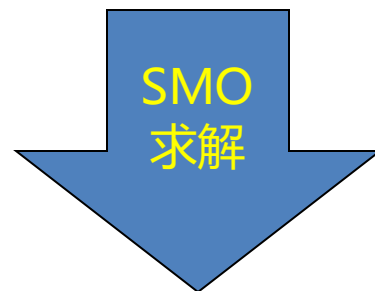
$$0 = \sum_{i=1}^n a^{(i)} y^{(i)}$$

## 支持向量机建模思想：模型求解第三步

- 第二步：使用**SMO**算法，求解参数 $\alpha$ 的值
- **注意**：每个样本都有一个 $\alpha$ ，结果只有**支持向量的样本**对应的 $\alpha$ 不为0，其他样本的 $\alpha=0$

$$\max_a \sum_{i=1}^n a^{(i)} - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n a^{(i)} a^{(j)} y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$$

$$s.t. \sum_{i=1}^n a^{(i)} y^{(i)} = 0, a^{(i)} \geq 0, i = 1, 2, \dots, n$$



$$\bar{w} = \sum_{i=1}^n \bar{a}^{(i)} y^{(i)} x^{(i)}$$

$$\bar{b} = \frac{\max_{i:y^{(i)}=-1} \bar{w}^T x^{(i)} + \min_{i:y^{(i)}=1} \bar{w}^T x^{(i)}}{2}$$

## 支持向量机建模思想：模型预测

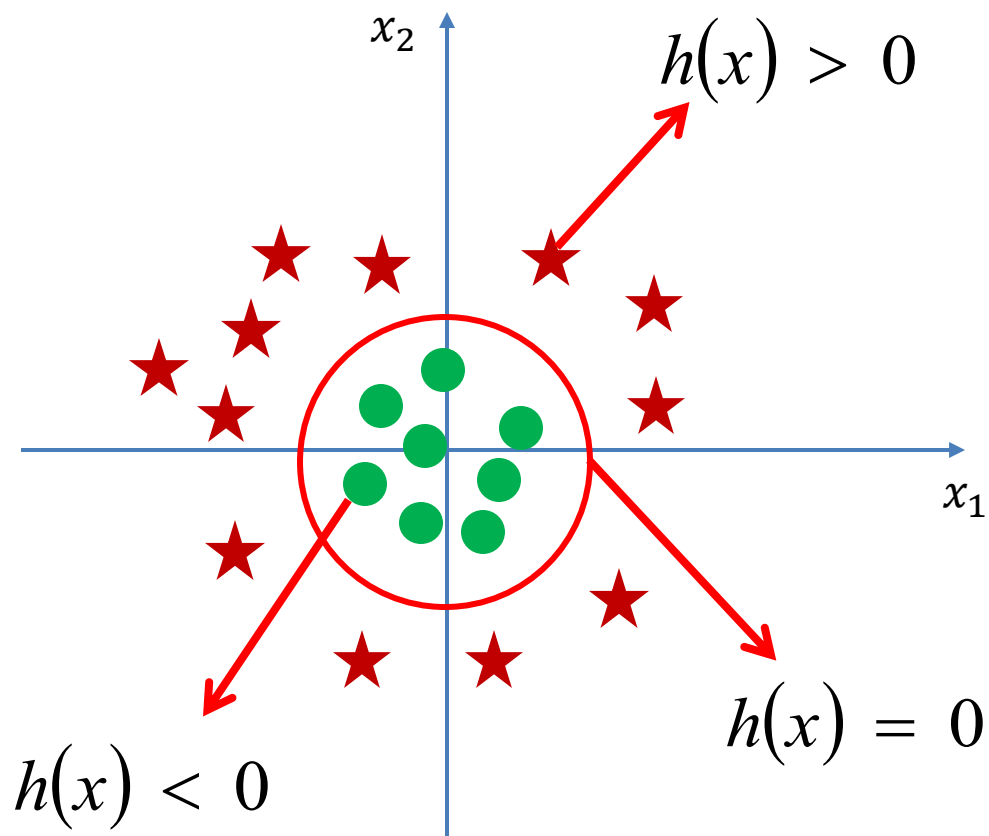
■ 对于一个新样本 $x$ ，对于一个新的点 $x$ ，带入超平面方程，得

$$f(x) = \left( \sum_{i=1}^n \bar{a}^{(i)} y^{(i)} x^{(i)} \right) x + b = \sum_{i=1}^n \bar{a}^{(i)} y^{(i)} x^{(i)T} x + b = \sum_{i=1}^n \bar{a}^{(i)} y^{(i)} \langle x^{(i)}, x \rangle + b$$

$$y = \text{sign}(f(x))$$

## 核方法：维度映射理解（多项式）

■ 如何解决低维线性不可分问题？升维



$\langle x_1, x_2 \rangle$  升维  $\langle x_1, x_2, x_1^2, x_2^2 \rangle$

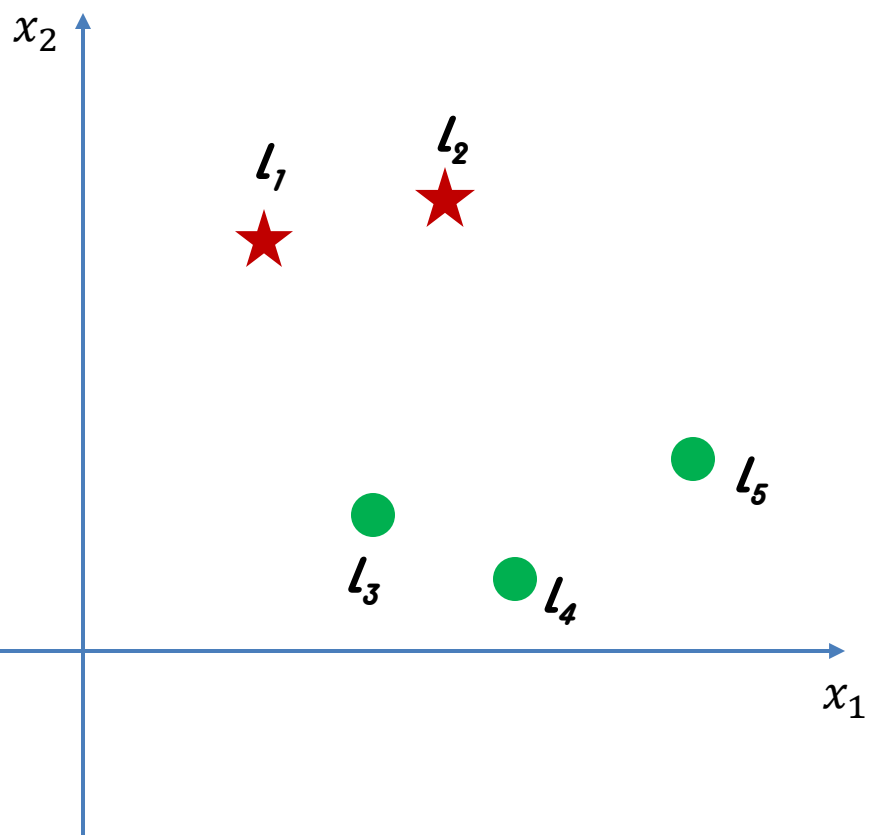
高维建模

$$h(x) = -4 + 0 \cdot x_1 + 0 \cdot x_2 + 1 \cdot x_1^2 + 1 \cdot x_2^2$$
$$= x_1^2 + x_2^2 - 4$$

## 核方法：高斯核映射

$$k(x^{(i)}, x^{(j)}) = e^{-\frac{\|x^{(i)} - x^{(j)}\|^2}{2\sigma^2}}$$

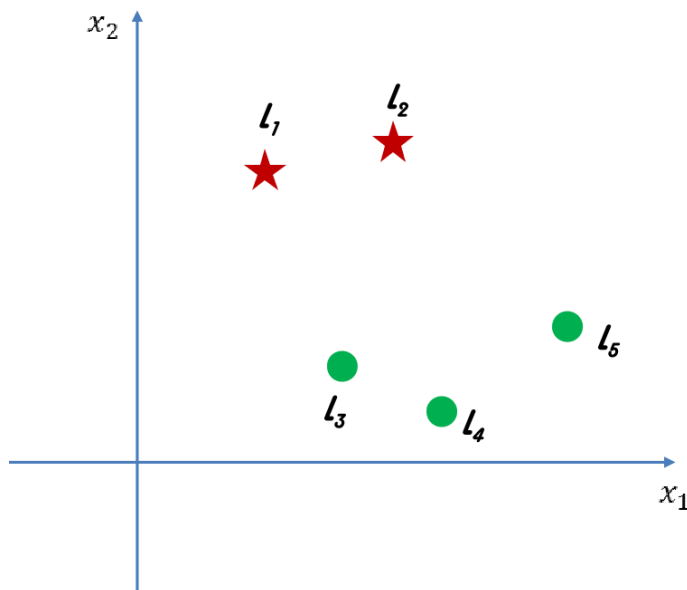
■ 基于距离的表征：利用样本跟所有样本间的距离信息对样本进行重新表征



|        | $l_1$                           | $l_2$                           | $l_3$                           | $l_4$                           | $l_5$                           |
|--------|---------------------------------|---------------------------------|---------------------------------|---------------------------------|---------------------------------|
| $l'_1$ | 1                               | $e^{-\frac{d_{12}}{2\sigma^2}}$ | $e^{-\frac{d_{13}}{2\sigma^2}}$ | $e^{-\frac{d_{14}}{2\sigma^2}}$ | $e^{-\frac{d_{15}}{2\sigma^2}}$ |
| $l'_2$ | $e^{-\frac{d_{21}}{2\sigma^2}}$ | 1                               | $e^{-\frac{d_{23}}{2\sigma^2}}$ | $e^{-\frac{d_{24}}{2\sigma^2}}$ | $e^{-\frac{d_{25}}{2\sigma^2}}$ |
| $l'_3$ |                                 |                                 |                                 |                                 |                                 |
| $l'_4$ |                                 |                                 |                                 |                                 |                                 |
| $l'_5$ |                                 |                                 |                                 |                                 |                                 |

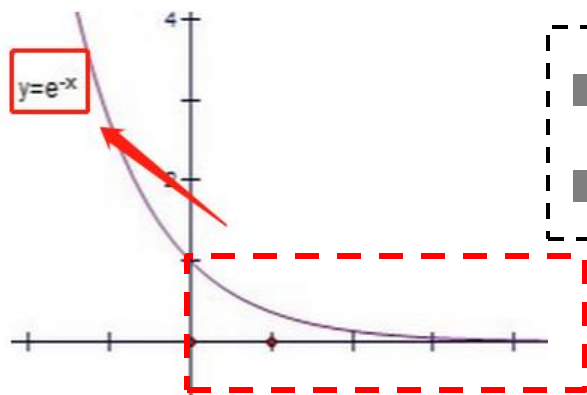
## 核方法：高斯核映射

|        | $l_1$ | $l_2$                           | $l_3$                           | $l_4$                           | $l_5$                           |
|--------|-------|---------------------------------|---------------------------------|---------------------------------|---------------------------------|
| $l'_1$ | 1     | $e^{-\frac{d_{12}}{2\sigma^2}}$ | $e^{-\frac{d_{13}}{2\sigma^2}}$ | $e^{-\frac{d_{14}}{2\sigma^2}}$ | $e^{-\frac{d_{15}}{2\sigma^2}}$ |



$$k(x^{(i)}, x^{(j)}) = e^{-\frac{\|x^{(i)} - x^{(j)}\|^2}{2\sigma^2}}$$

→ 两点距离的平方  
→ 核宽度

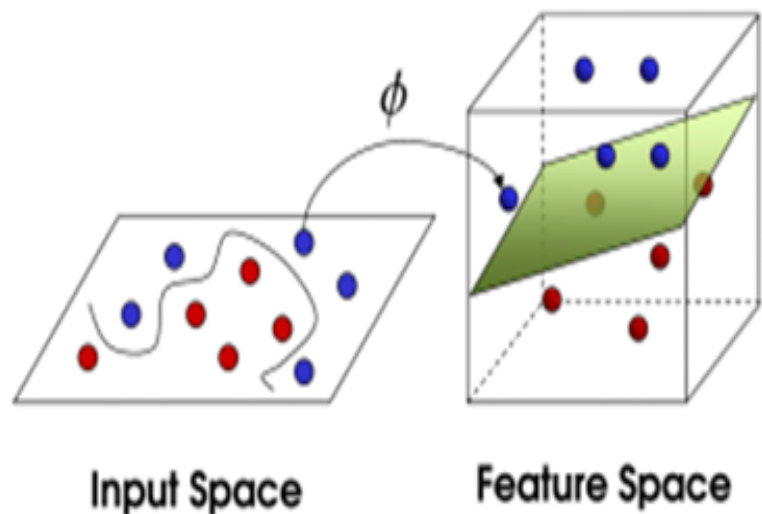


- 距离越远，影响越小
- 距离越近，影响越大

- 核宽度 $\sigma$ 越大，衰减越慢，差异越小，欠拟合
- 核宽度 $\sigma$ 越小，衰减越快，差异越大，过拟合



## 核方法：核方法本质



- 1. 首先使用一个非线性映射 $\phi$ 将数据变换到一个特征空间 $F$ ,
- 2. 然后在特征空间 $F$ 中使用线性学习器分类。分类决策函数如下:

$$f(x) = \sum_{i=1}^n \bar{a}^{(i)} y^{(i)} \varphi(x^{(i)})^T \varphi(x) + b = \sum_{i=1}^n \bar{a}^{(i)} y^{(i)} \langle \varphi(x^{(i)}), \varphi(x) \rangle + b$$

难道要在高维特征空间 $F$ 中进行内积计算?

## 核方法：核方法本质

■ 低维空间的计算实现高维空间的效果，**无需真的去做映射**

如果存在一种计算方式，能够在当前低维的特种空间直接计算 $\langle \varphi(x^{(i)}), \varphi(x) \rangle$ ，那么上述的计算非线性可分问题的两个步骤就能合成一步建立一个非线性的分类器。这个就是核函数：

$$f(x) = \sum_{i=1}^n \bar{a}^{(i)} y^{(i)} \langle \varphi(x^{(i)}), \varphi(x) \rangle + b$$



$$k(x^{(i)}, x^{(j)}) = \langle \varphi(x^{(i)}), \varphi(x^{(j)}) \rangle$$



$$f(x) = \sum_{i=1}^n \bar{a}^{(i)} y^{(i)} \underline{k(x^{(i)}, x)} + b$$

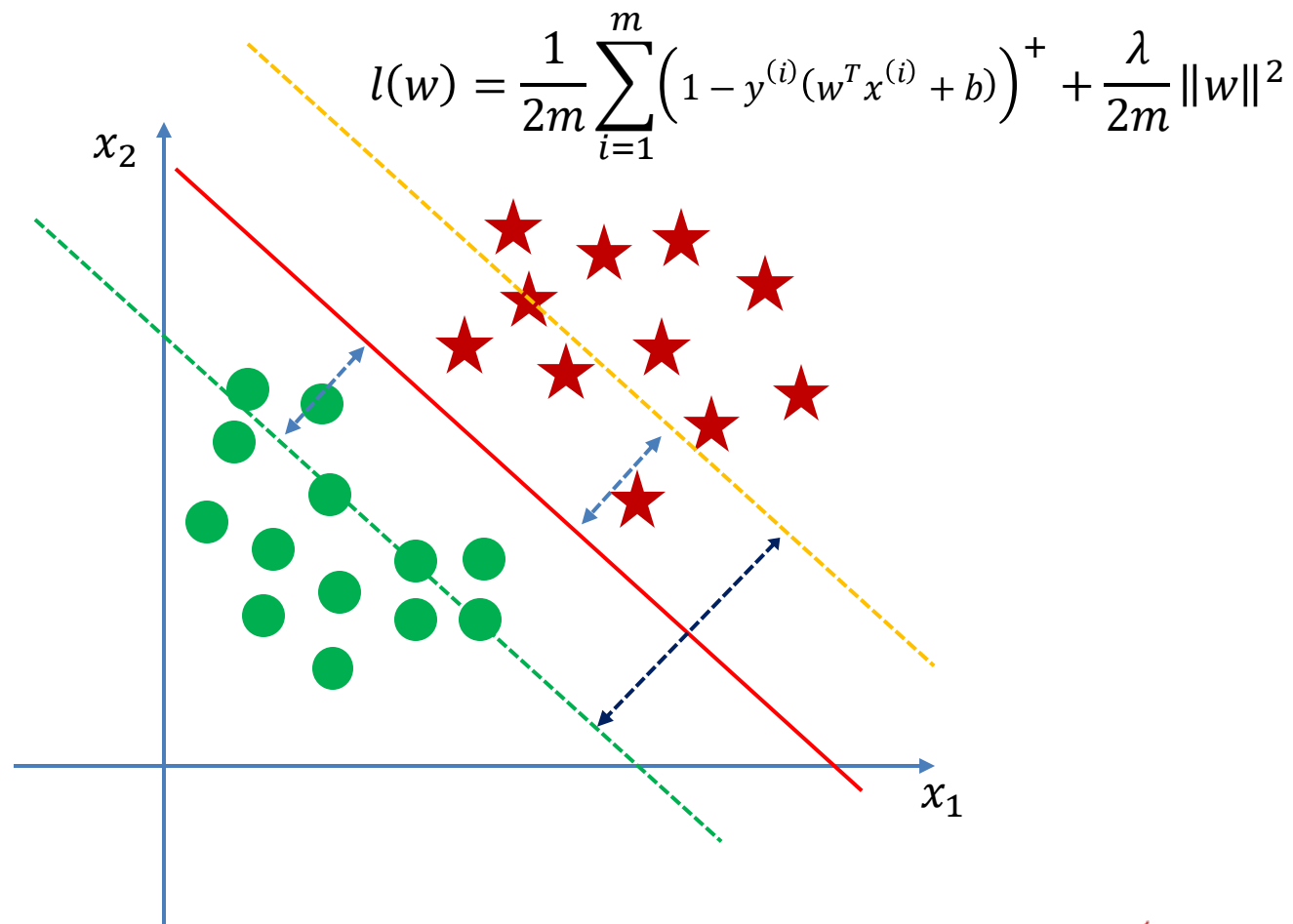
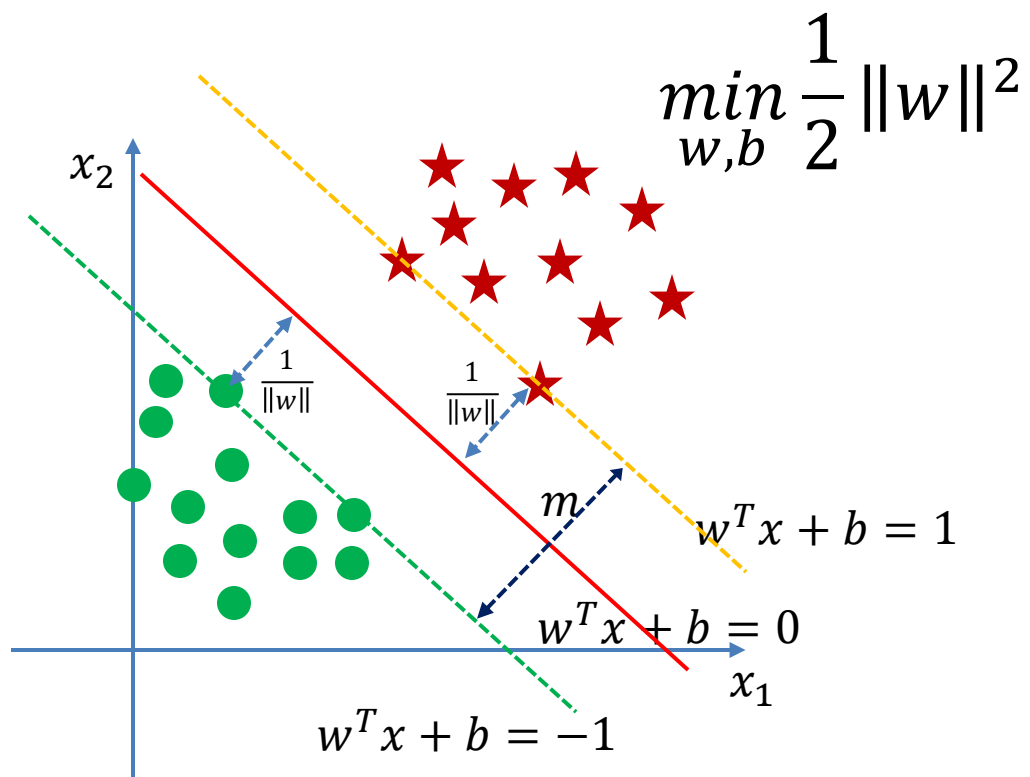
多项式核： $\kappa(x_1, x_2) = (\langle x_1, x_2 \rangle + R)^d$

高斯核函数 RBF： $\kappa(x_1, x_2) = \exp\left(-\frac{\|x_1 - x_2\|^2}{2\sigma^2}\right)$

线性核： $\kappa(x_1, x_2) = \langle x_1, x_2 \rangle$

## 软间隔

■ 允许部分样本错分



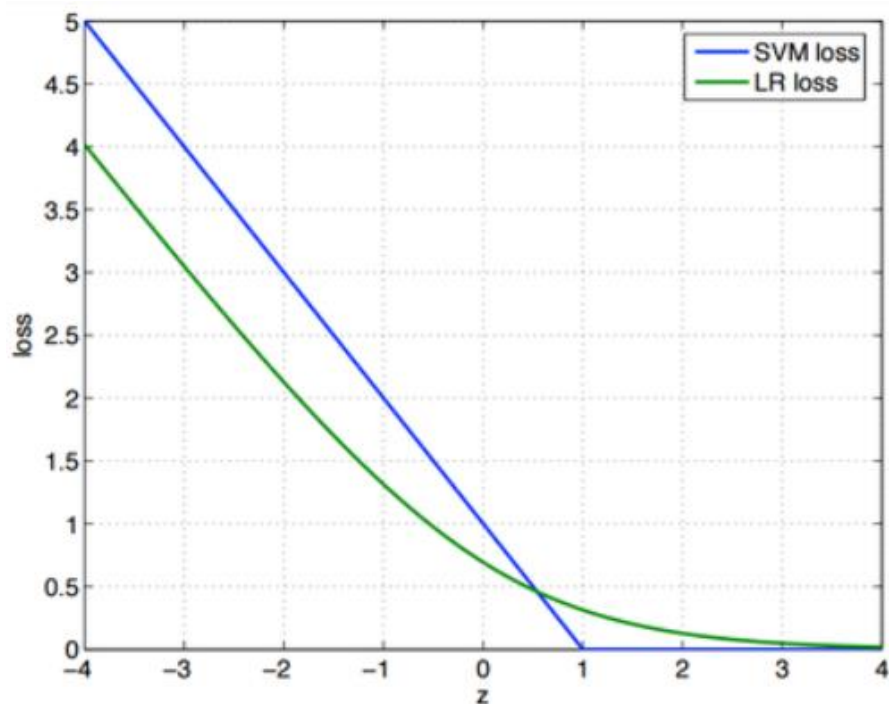
## 支持向量机和逻辑回归：损失函数比较

■ 逻辑回归损失函数:  $l(w) = \frac{1}{2m} \sum_{i=1}^m \left( y^{(i)} \log \frac{1}{1 + e^{-w^T x^{(i)} + b}} + (1 - y^{(i)}) \log \left( 1 - \frac{1}{1 + e^{-w^T x^{(i)} + b}} \right) \right) + \frac{\lambda}{2m} \|w\|^2$

■ 支持向量机损失函数:  $l(w) = \frac{1}{2m} \sum_{i=1}^m \left( 1 - y^{(i)} (w^T x^{(i)} + b) \right)^+ + \frac{\lambda}{2m} \|w\|^2$

对数似然损失 (log loss)

合页损失 (hinge loss)



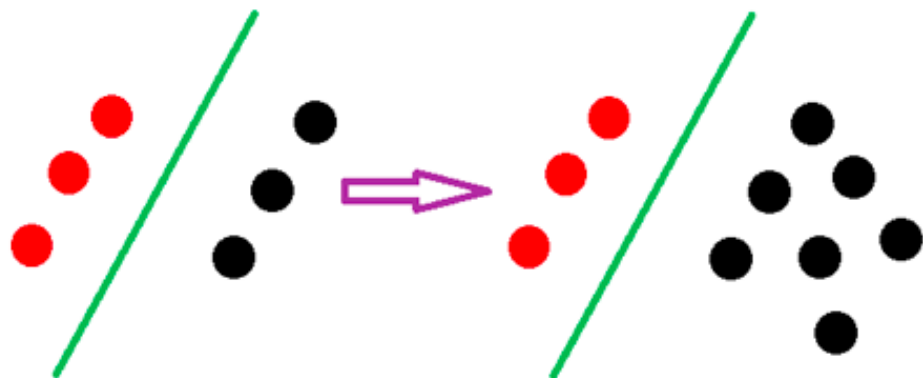
## 支持向量机和逻辑回归：相同点

- LR 和 SVM 都可以做**分类**
- LR 和 SVM 都是**监督学习**算法
- 如果**不考虑核函数**，LR 和 SVM 都是线性分类算法，也就是说他们的分类决策面都是**线性的**
- LR和 SVM 都是**判别模型**

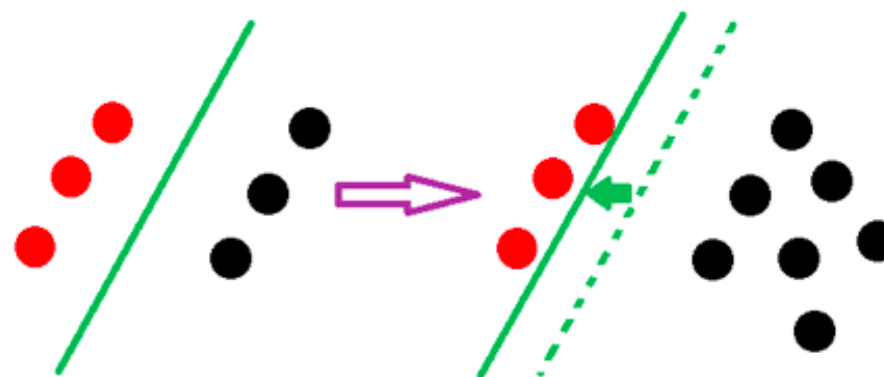
## 支持向量机和逻辑回归：不同点

- 支持向量机**只考虑局部**的边界线附近的点（支持向量），而逻辑回归**考虑全局**（似然：远离的点对边界线的确定也起作用，虽然作用会相对小一些）
- 因为上面的原因，得知：线性 SVM 不直接依赖于数据分布，分类平面不受一类点影响；LR 则受所有数据点的影响，如果数据不同类别 strongly unbalance，一般需要先对数据做 balancing。

支持向量机改变非支持向量样本并不会引起决策面的变化：



逻辑回归中改变任何样本都会引起决策面的变化：



## 支持向量机和逻辑回归：不同点

- 在解决非线性问题时，支持向量机**采用核函数**的机制，而 LR 通常**不采用核函数**的方法
- 逻辑回归是**参数**模型，支持向量机是**无参**模型
- 逻辑回归适合**大样本**训练，支持向量机适合**小样本**训练（大样本计算量太大，无法接受）
- 逻辑回归是**经验风险**最小化，支持向量机是**结构风险**最小化

## 小结

# 支持向量机

### 模型原理

最大间隔思想

拉格朗日乘子法

KKT条件和对偶问题

SMO优化求解

### 核方法

核思想：低维空间计算实现高维空间建模

核函数

线性核

多项式核

高斯核

### 硬间隔和软间隔

软间隔

合页损失函数

### 支持向量机和逻辑回归

二者损失函数比较

二者相同点和不同点

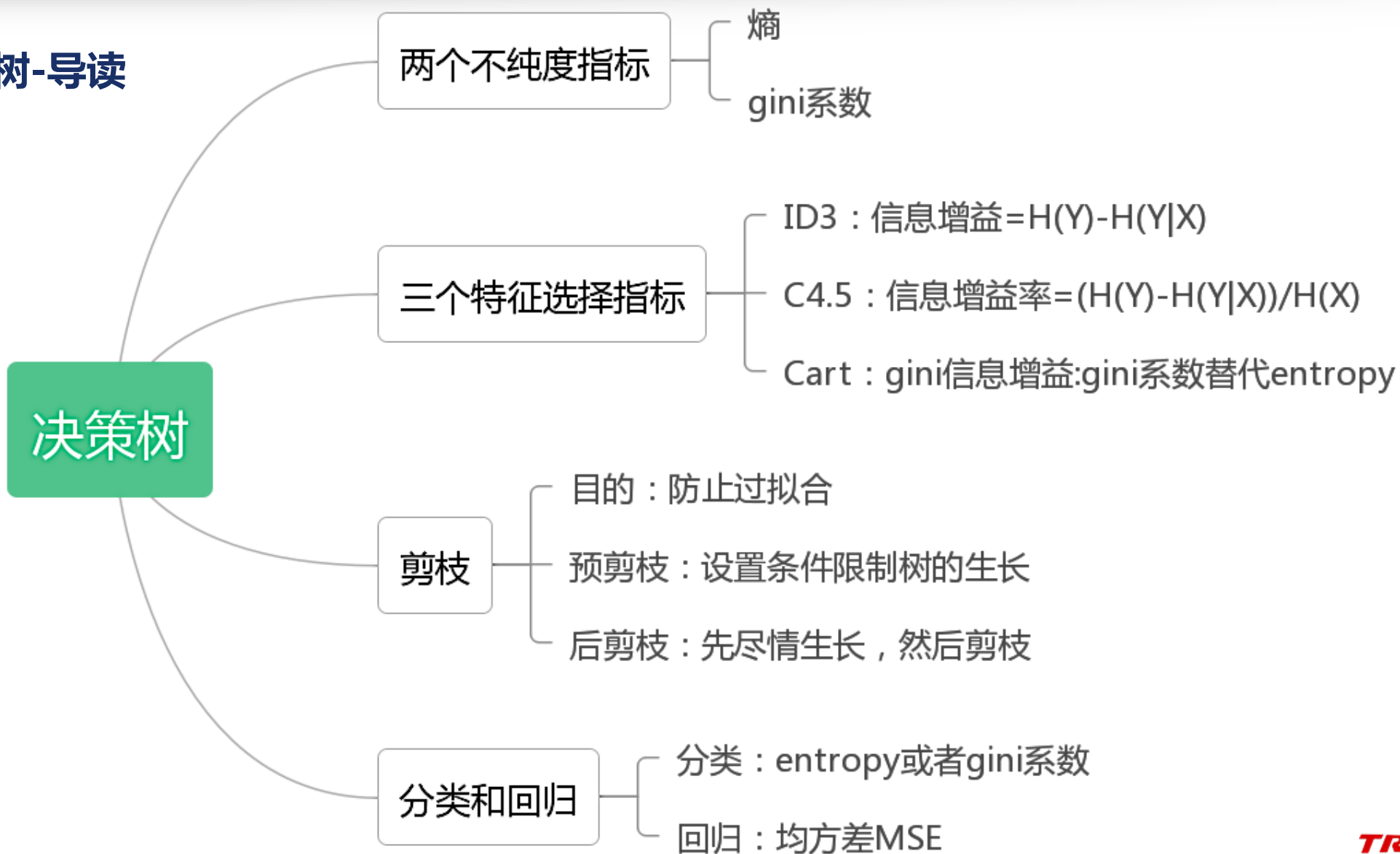




1  
5  
chapter

决策树

## 决策树-导读



## 决策树 - 什么是决策树?

顾名思义，**决策树就是一个树形结构的决策流程图。**

先来看下面的一个例子：母亲要给女儿介绍男朋友，于是有了这样的对话：

女儿：多大年纪了？

母亲：26

女儿：长的帅不帅？

母亲：挺帅的

女儿：收入高不？

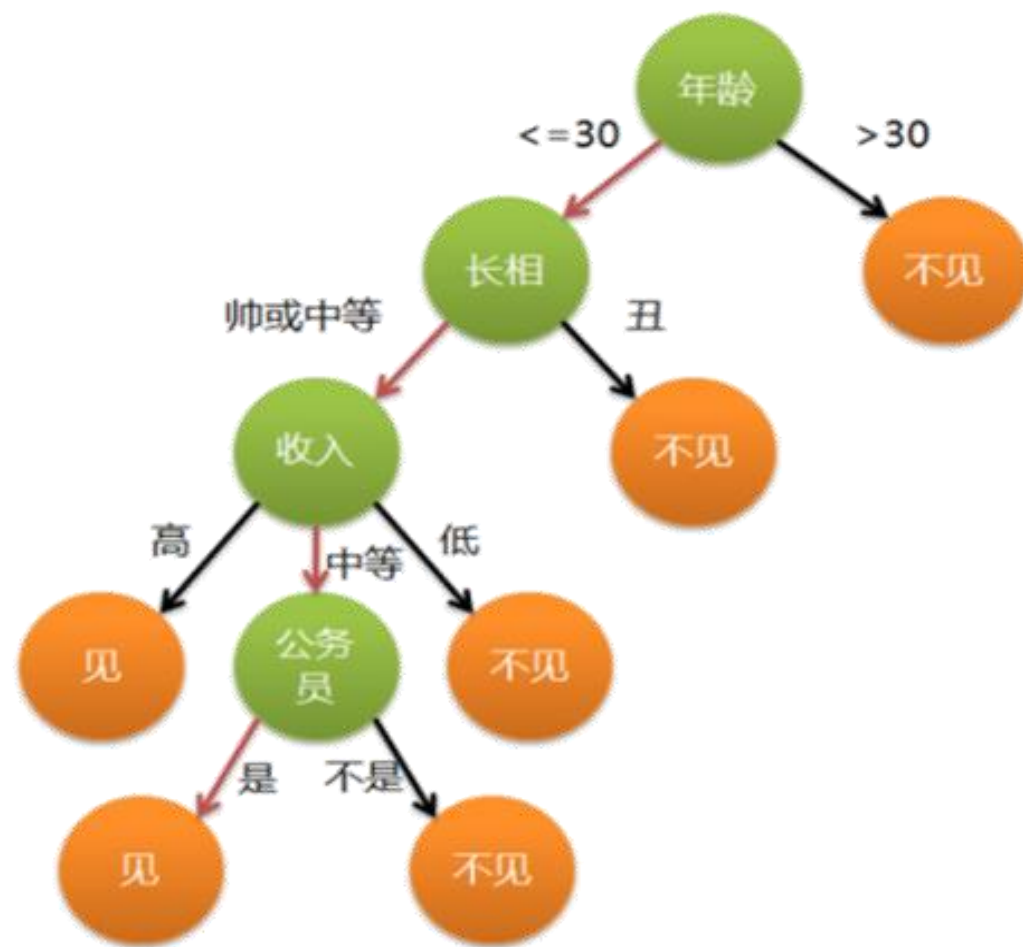
母亲：不算很高，中等情况

女儿：是公务员不？

母亲：是，在税务局上班呢

女儿：那好，我去见见。

女儿决定要不要去相亲的思考过程就是一个典型的决策树的判断过程：



## 决策树-什么是决策树？

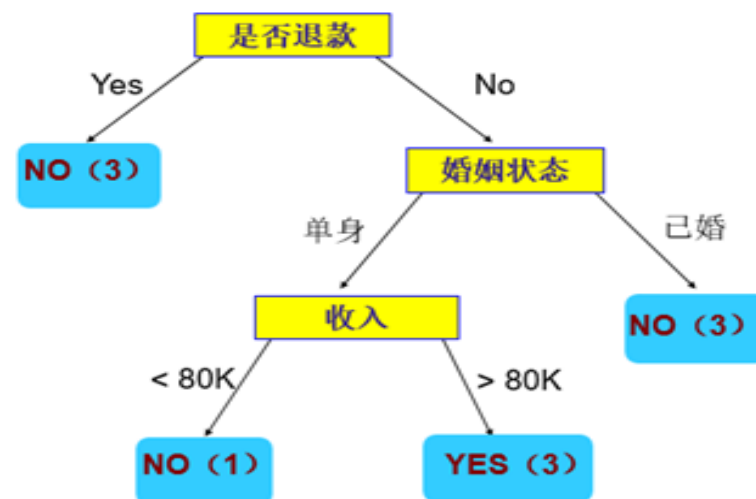
- **规则树**：专家制定规则得到的树，不是机器学习。
- **决策树**：从数据集中学习（机器学习）得到一棵规则树，就是决策树。

机器学习中的决策树是通过对训练数据集进行有目标监督的学习而得到的，例如下图所示：左边是一个训练数据集，目标变量是是否欺诈？，输入变量是退款，婚姻状态和收入。下边的决策树就是以是否欺诈？作为目标变量，使用决策树算法训练得到的一个用于判断是否欺诈的决策树模型。

| obs | 退款  | 婚姻状态 | 收入   | 欺诈？ |
|-----|-----|------|------|-----|
| 1   | Yes | 单身   | 125K | No  |
| 2   | No  | 已婚   | 100K | No  |
| 3   | No  | 单身   | 70K  | No  |
| 4   | Yes | 已婚   | 120K | No  |
| 5   | No  | 单身   | 95K  | Yes |
| 6   | No  | 已婚   | 60K  | No  |
| 7   | Yes | 单身   | 220K | No  |
| 8   | No  | 单身   | 85K  | Yes |
| 9   | No  | 已婚   | 75K  | No  |
| 10  | No  | 单身   | 90K  | Yes |

训练数据集

算法学习



## 决策树 - 核心思想

- 从根节点开始，选择一个最好的分裂属性 (**决策变量**) 来产生一个分叉，有个形象的说法叫 **树的生长**；
- 对每一个分叉 (子节点) 又继续 **寻找一个最好的分裂属性** 来 **产生分叉**；
- 让树不断的如此生长下去...直到满足某些条件后让树 **停止生长**；
- 最终得到一棵我们想要的 **决策树**。

### 你可能马上会问几个问题：

- 父节点是如何选择属性来进行分叉的，或者换句话说 **如何评估** 一个 **分叉** 是 **最好** 的？
- 树 **什么时候** 应该 **停止生长**？

## 信息熵（ entropy ）定义

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

其中X表示随机变量，随机变量X的取值为  $(x_1, x_2, \dots, x_n)$  , $p(x_i)$ 表示事件 $x_i$ 发生的概率，且有  $\sum p(x_i)=1$ .信息熵的单位为bit。

## 信息熵：猜冠军

- 2018年俄罗斯世界杯足球赛16强中，谁是世界杯冠军？
- $Ent(D) = - (p_1 * \log p_1 + p_2 * \log p_2 + \dots + p_{16} * \log p_{16})$  , 其中  $p_1, \dots, p_{16}$  分别是这 16 支球队夺冠的概率。
- 当每支球队夺冠概率相等都是  $1/16$  的时:  $Ent(D) = - (16 * 1/16 * \log 1/16) = 4$

## 信息熵：猜冠军

随堂练习：

■ 篮球比赛里，有4个球队 {A,B,C,D}，获胜概率分别为{1/2, 1/4, 1/8, 1/8}，求Ent(D)

$$\begin{aligned} Ent(D) &= - \left[ \frac{1}{2} \log_2 \left( \frac{1}{2} \right) + \frac{1}{4} \log_2 \left( \frac{1}{4} \right) + \frac{1}{8} \log_2 \left( \frac{1}{8} \right) + \frac{1}{8} \log_2 \left( \frac{1}{8} \right) \right] \\ &= \frac{1}{2} \log_2 2 + \frac{1}{4} \log_2 4 + \frac{1}{8} \log_2 8 + \frac{1}{8} \log_2 8 \\ &= \left( \frac{1}{2} + \frac{1}{2} + \frac{3}{8} + \frac{3}{8} \right) \log_2 2 \\ &= \frac{7}{4} \end{aligned}$$



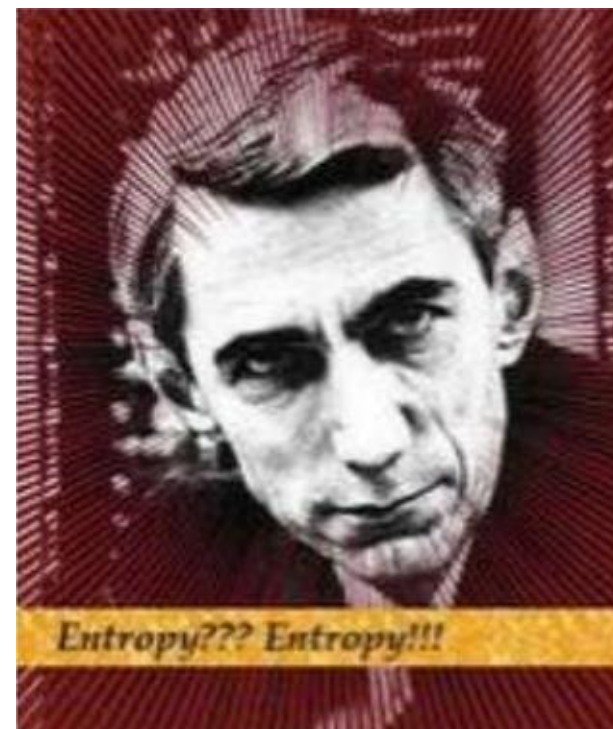
## 信息熵：信息熵就是不纯度的度量

**信息量**：指的是一个样本/事件所蕴含的信息，如果一个事件的概率越大，那么就可以认为该事件所蕴含的信息越少。极端情况下，比如：“太阳从东方升起”，因为是确定事件，所以不携带任何信息量。

**信息熵**：1948年，**香农**引入信息熵：一个系统越是有序，信息熵就越低；一个系统越是混乱，信息熵就越高。所以信息熵被认为是一个系统有**序程度**的度量。

**信息熵**就是系统**不确定性（不纯度）**的度量。

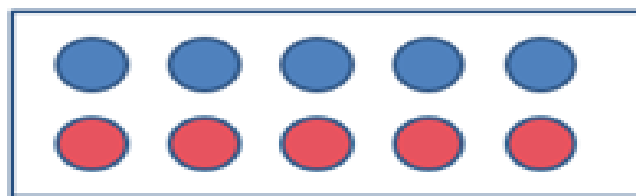
$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$



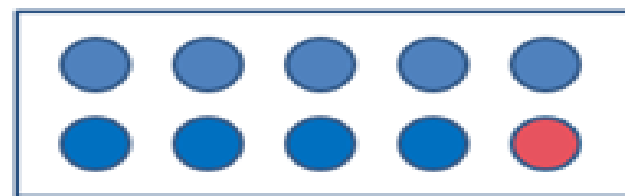
## 信息熵：信息熵就是不纯度的度量

什么叫**不纯度**呢？它是纯度的反义词

- 如果一个节点内的数据大部分都属于同一个类别，那么我们就称这个节点**很纯**，对应的不纯度就比较低。
- 反之，如果一个节点内各个类别的数量分布很均匀，那么我们就称这个节点**很不纯**，也就是意味着不纯度很高。



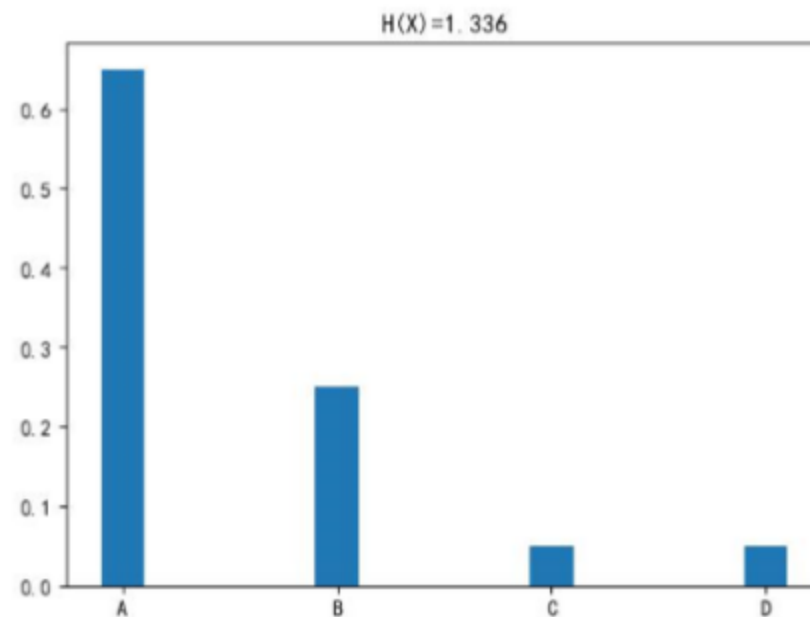
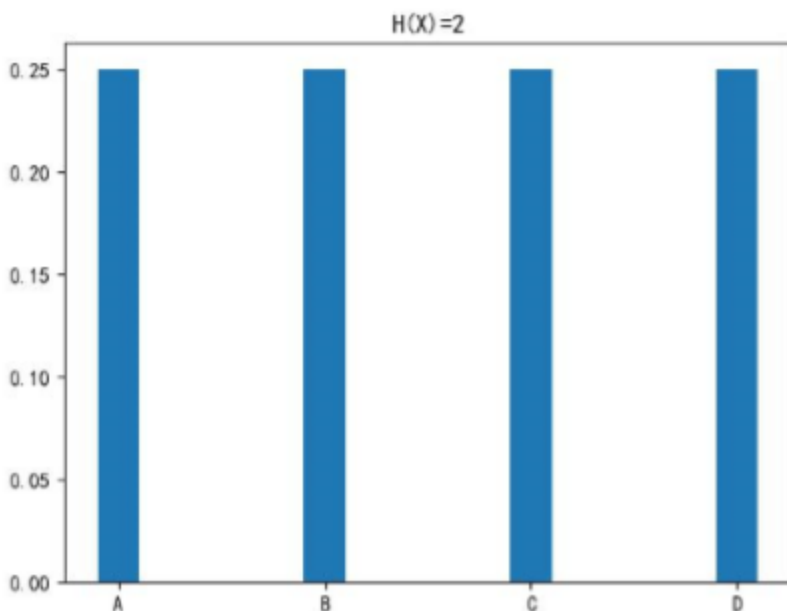
“不纯度”很高  
(纯度很低)



“不纯度”很低  
(纯度很高)

## 信息熵：信息熵与概率的关系

- **High Entropy (高信息熵)**：表示随机变量 $X$ 是均匀分布的，各种取值情况是等概率出现的，**不纯度高**。
- **Low Entropy (低信息熵)**：表示随机变量 $X$ 各种取值不是等概率出现。可能出现有的事件概率很大，有的事件概率很小，**不纯度低**。



## 信息增益(ID3): 如何分裂 - 信息增益最大

- 我们需要定义一个**最佳的标准**，到底什么样的分叉叫好？从直观上来理解，**决策树节点分叉的目的是为了让子节点变得更纯**
- 所以我们可以用**分叉前的不纯度**减去**分叉后的不纯度**来衡量分叉对于纯度的提升。因为分叉前不纯度肯定是比较高的，分叉后不纯度肯定会变小，所以这个差值就代表了纯度的提升度，我们把它叫**信息增益**。所以判断节点分叉的好坏标准就是：**分叉以后的信息增益获得最大！**
- **信息增益 = entroy(前) - entroy(后) =  $H(Y) - \overline{H(Y|X)}$**

条件熵

## 信息增益(ID3): 信息增益= $H(Y) - H(Y|X)$

- 给定条件  $X$  的情况下, 随机变量  $Y$  的信息熵就叫做**条件熵**。

| 专业 (X) | 性别 (Y) |
|--------|--------|
| 数学     | M      |
| IT     | M      |
| 英语     | F      |
| 数学     | F      |
| 数学     | M      |
| IT     | M      |
| 英语     | F      |
| 数学     | F      |

$$P(X = \text{数学}) = 0.5$$

$$P(Y = M) = 0.5$$

$$P(X = \text{数学}, Y = F) = 0.25$$

$$P(Y = M | X = \text{英语}) = 0$$

$$H(X) = 1.5$$

$$H(Y) = 1$$

## 信息增益(ID3): 信息增益= $H(Y) - H(Y|X)$

■ 当专业(X)为**数学**的时候, Y的信息熵的值为: $H(Y|X=\text{数学})$ 。

| 专业 (X) | 性别 (Y) |
|--------|--------|
| 数学     | M      |
| IT     | M      |
| 英语     | F      |
| 数学     | F      |
| 数学     | M      |
| IT     | M      |
| 英语     | F      |
| 数学     | F      |

$X=\text{数学}$



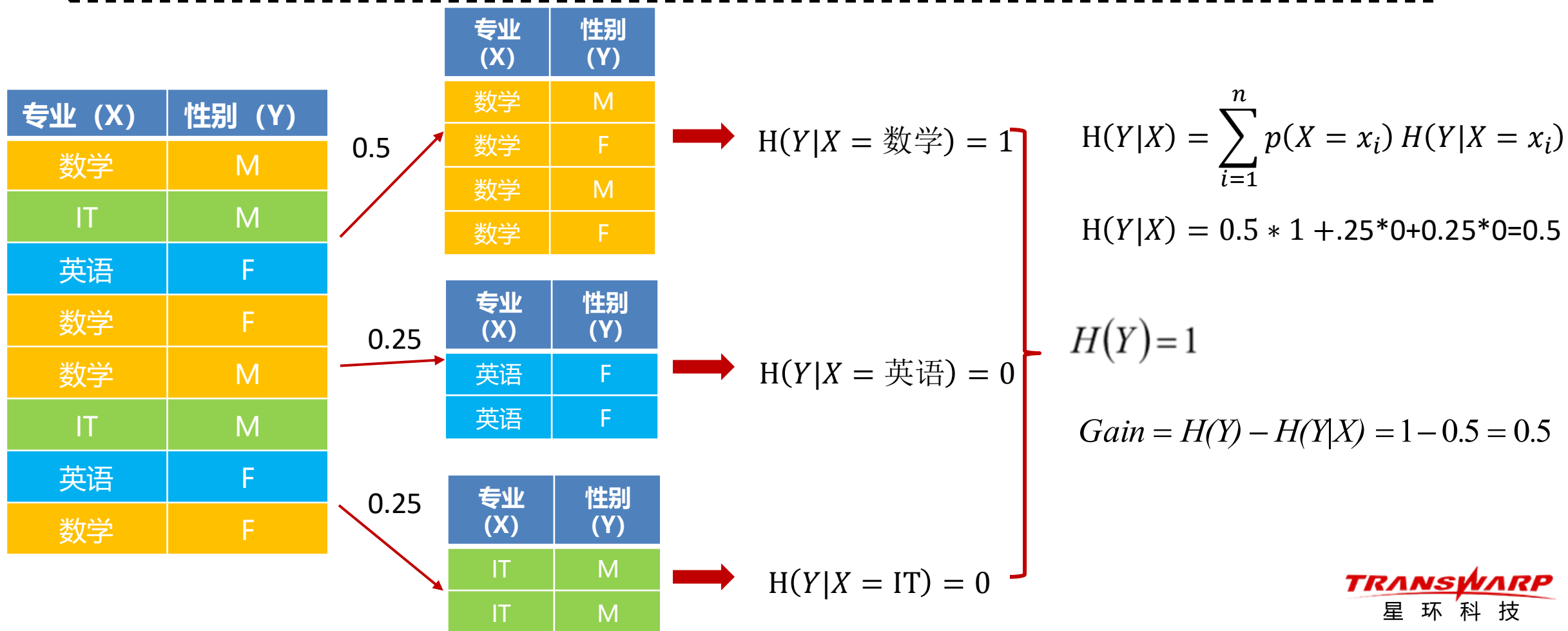
| 专业 (X) | 性别 (Y) |
|--------|--------|
| 数学     | M      |
| 数学     | F      |
| 数学     | M      |
| 数学     | F      |



$$H(Y | X = \text{数学}) = 1$$

## 信息增益(ID3): 信息增益= $H(Y) - H(Y|X)$

■ 给定条件X的情况下，所有不同 X 值情况下 Y 的信息熵的**加权平均值**叫做**条件熵**。



## 信息增益(ID3): 选择最优分割特征

- 首先, 计算整个数据 Y 的熵

| uin | gender | act_info | is_lost |
|-----|--------|----------|---------|
| 1   | 男      | 高        | 0       |
| 2   | 女      | 中        | 0       |
| 3   | 男      | 低        | 1       |
| 4   | 女      | 高        | 0       |
| 5   | 男      | 高        | 0       |
| 6   | 男      | 中        | 0       |
| 7   | 男      | 中        | 1       |
| 8   | 女      | 中        | 0       |
| 9   | 女      | 低        | 1       |
| 10  | 女      | 中        | 0       |
| 11  | 女      | 高        | 0       |
| 12  | 男      | 低        | 1       |
| 13  | 女      | 低        | 1       |
| 14  | 男      | 高        | 0       |
| 15  | 男      | 高        | 0       |

|    | positive | negative | 汇总 |
|----|----------|----------|----|
| 整体 | 5        | 10       | 15 |
| 男性 | 3        | 5        | 8  |
| 女性 | 2        | 5        | 7  |
| 高  | 0        | 6        | 6  |
| 中  | 1        | 4        | 5  |
| 低  | 4        | 0        | 4  |

$$Ent(D) = -\frac{5}{15} \log_2\left(\frac{5}{15}\right) - \frac{10}{15} \log_2\left(\frac{10}{15}\right) = 0.9182$$



## 信息增益(ID3): 选择最优分割特征

### ■ 计算特征“性别”的信息增益

|    | positive | negative | 汇总 |
|----|----------|----------|----|
| 整体 | 5        | 10       | 15 |
| 男性 | 3        | 5        | 8  |
| 女性 | 2        | 5        | 7  |
| 高  | 0        | 6        | 6  |
| 中  | 1        | 4        | 5  |
| 低  | 4        | 0        | 4  |

$$Ent(D, \text{性别}) = \sum_{v=1}^V \frac{D^v}{D} Ent(D^v) = \frac{D^1}{D} Ent(D^1) + \frac{D^2}{D} Ent(D^2)$$

$$Ent(D^1) = -\frac{3}{8} \log_2\left(\frac{3}{8}\right) - \frac{5}{8} \log_2\left(\frac{5}{8}\right) = 0.9543$$

$$Ent(D^2) = -\frac{2}{7} \log_2\left(\frac{2}{7}\right) - \frac{5}{7} \log_2\left(\frac{5}{7}\right) = 0.8631$$

$$Gain(D, \text{性别}) = Ent(D) - Ent(D|a) = Ent(D) - \sum_{v=1}^V \frac{D^v}{D} Ent(D^v)$$

$$= Ent(D) - \frac{8}{15} Ent(D^1) - \frac{7}{15} Ent(D^2)$$

$$= 0.0064$$

## 信息增益(ID3): 选择最优分割特征

- 计算特征“活跃度”的信息增益

|    | positive | negative | 汇总 |
|----|----------|----------|----|
| 整体 | 5        | 10       | 15 |
| 男性 | 3        | 5        | 8  |
| 女性 | 2        | 5        | 7  |
| 高  | 0        | 6        | 6  |
| 中  | 1        | 4        | 5  |
| 低  | 4        | 0        | 4  |

$$Ent(D^1) = 0$$

$$Gain(D, \text{性别}) = 0.0064$$

$$Ent(D^2) = 0.7219$$

$$Gain(D, \text{活跃度}) = 0.6776$$

$$Ent(D^3) = 0$$

$$\begin{aligned} Gain(D, \text{活跃度}) &= Ent(D) - Ent(D|a) = Ent(D) - \sum_{v=1}^V \frac{D^v}{D} Ent(D^v) \\ &= Ent(D) - \frac{6}{15} Ent(D^1) - \frac{5}{15} Ent(D^2) - \frac{4}{15} Ent(D^3) \\ &= 0.6776 \end{aligned}$$

## 信息增益(ID3): 信息增益的bug

## ■ 偏向分叉多的特征

| uin | gender | act_info | is_lost |
|-----|--------|----------|---------|
| 1   | 男      | 高        | 0       |
| 2   | 女      | 中        | 0       |
| 3   | 男      | 低        | 1       |
| 4   | 女      | 高        | 0       |
| 5   | 男      | 高        | 0       |
| 6   | 男      | 中        | 0       |
| 7   | 男      | 中        | 1       |
| 8   | 女      | 中        | 0       |
| 9   | 女      | 低        | 1       |
| 10  | 女      | 中        | 0       |
| 11  | 女      | 高        | 0       |
| 12  | 男      | 低        | 1       |
| 13  | 女      | 低        | 1       |
| 14  | 男      | 高        | 0       |
| 15  | 男      | 高        | 0       |

$$Ent(D) = -\frac{5}{15}\log_2\left(\frac{5}{15}\right) - \frac{10}{15}\log_2\left(\frac{10}{15}\right) = 0.9182$$

$$Gain(D, \text{性别}) = 0.0064$$

$$Gain(D, \text{活跃度}) = 0.6776$$

$$Gain(D, uin) = 0.9182$$

## 信息增益率(C4.5): 解决信息增益的bug

- 为了克服信息增益的这个缺点，我们引入一个叫 **信息增益率** 的指标，它是在信息增益的基础上对多分叉进行了惩罚，当分叉比较多的时候，就惩罚信息增益值让它变低一些，具体的做法就是用信息增益值除以分叉的一个信息值，如下图所示：

$$Gain\_ratio = \frac{Gain(D, x)}{IV(x)} = \frac{H(y) - H(y|x)}{H(x)}$$

特征x帮y减的熵（传递给y的信息量）

特征x的熵（信息量）

## 对y的灵魂拷问：喜欢什么样的x君？

- 信息增益**：喜欢给我花的钱多的x
- 信息增益率**：喜欢舍得给我花钱的x

## CART 树: gini 系数-另一个不纯度指标

■ **Gini 系数计算公式**: 1减去节点内每一个类的概率平方和。

$$gini = 1 - \sum_{i=1}^n (p(x_i))^2$$

节点1:

|    |   |
|----|---|
| C1 | 0 |
| C2 | 6 |

$$P(C1) = 0/6 = 0 \quad P(C2) = 6/6 = 1$$

$$Gini = 1 - P(C1)^2 - P(C2)^2 = 1 - 0 - 1 = 0$$

节点2:

|    |   |
|----|---|
| C1 | 1 |
| C2 | 5 |

$$P(C1) = 1/6 \quad P(C2) = 5/6$$

$$Gini = 1 - (1/6)^2 - (5/6)^2 = 0.278$$

节点3:

|    |   |
|----|---|
| C1 | 2 |
| C2 | 4 |

$$P(C1) = 2/6 \quad P(C2) = 4/6$$

$$Gini = 1 - (2/6)^2 - (4/6)^2 = 0.444$$

## CART 树: gini 系数和信息熵比较

- **功能相同**: 都是用来衡量不纯度, 系统不确定性的度量。
- **计算方式不同**: 信息熵是**对数运算**, 效率低; gini系数是**乘法运算**, 效率高。
- **二者可相互替换, 作用没有任何区别。**

$$H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$$

$$gini = 1 - \sum_{i=1}^n (p(x_i))^2$$

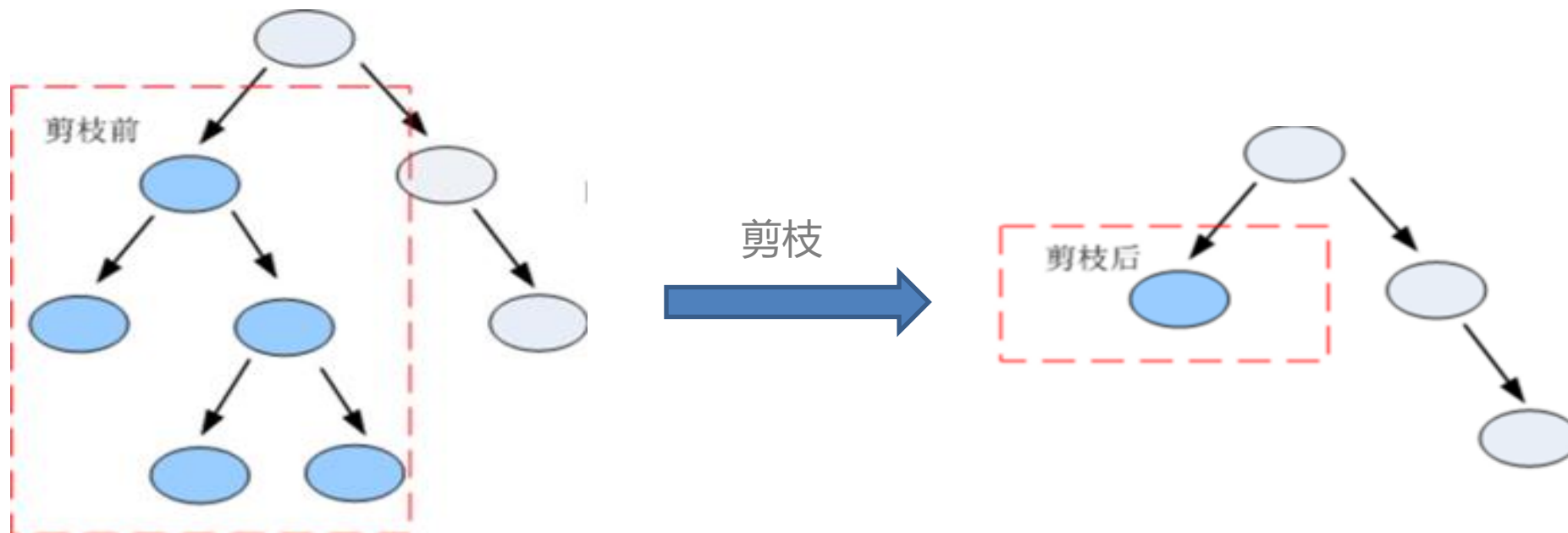
Cart树：算法原理

- 使用**gini系数**代替**信息熵**的计算。
- 使用**信息增益**而不是**信息增益率**。如何解决偏向多分叉特征问题？
- **强制二叉树**解决偏向多分叉特征问题。

ID3算法使用 **信息增益** 来判断 最佳分叉  
C4.5算法的决策树允许多分叉，使用 **信息增益率** 来判断 最佳分叉  
CART算法的决策树只允许二分叉，使用 **Gini信息增益** 来判断 最佳分叉，为什么不是用信息增益率？因为CART只允许二分叉，这样就不存在信息增益的那个缺点了，用信息增益和信息增益率的效果都是一样的。

| 算法   | 支持模型  | 树结构 | 特征选择     | 连续值处理 | 缺失值处理 | 剪枝  |
|------|-------|-----|----------|-------|-------|-----|
| ID3  | 分类    | 多叉树 | 信息增益     | 不支持   | 不支持   | 不支持 |
| C4.5 | 分类    | 多叉树 | 信息增益率    | 支持    | 支持    | 支持  |
| CART | 分类、回归 | 二叉树 | 基尼系数、均方差 | 支持    | 支持    | 支持  |

## 树的剪枝：决策树什么时候停止生长



- ◆ 剪枝的目的很简单：**降低决策树复杂度，避免过拟合**。想想极端情况，如果让决策树无限的生长下去，最终的结果就是每一个叶子节点中都只有1个样本，那么模型在训练集上有100%的准确率，但是在测试集上估计效果和瞎猜是一样的。



## 树的剪枝：剪枝的方法

◆ **预剪枝：** 设定一些规则来避免树的过度生长

- 信息增益（率）少于阈值就不再生长
- 节点的样本数少于阈值（例如1%）就不再生长
- 不允许叶子节点的样本数少于某个阈值（例如0.5%）
- 不允许树的深度超过阈值（例如8层）

◆ **后剪枝：** 先让决策树无限的生长成一棵很大的树（这个时候肯定过拟合了），然后再剪枝从下往上依次判断：如果剪掉子树（把父节点作为叶子节点）能否让验证集的误差下降，如果可以，那么就减掉，不断重复此操作。

## 决策树可以用于回归问题吗？

我们前面介绍的都是决策树用于解决分类问题，实际上 **决策树也是可以解决回归问题** 的，用于回归问题的决策树又叫 **回归树**。**回归树** 与 **分类树** 最大的区别就是 **节点的不纯度** 的 **定义不同**：

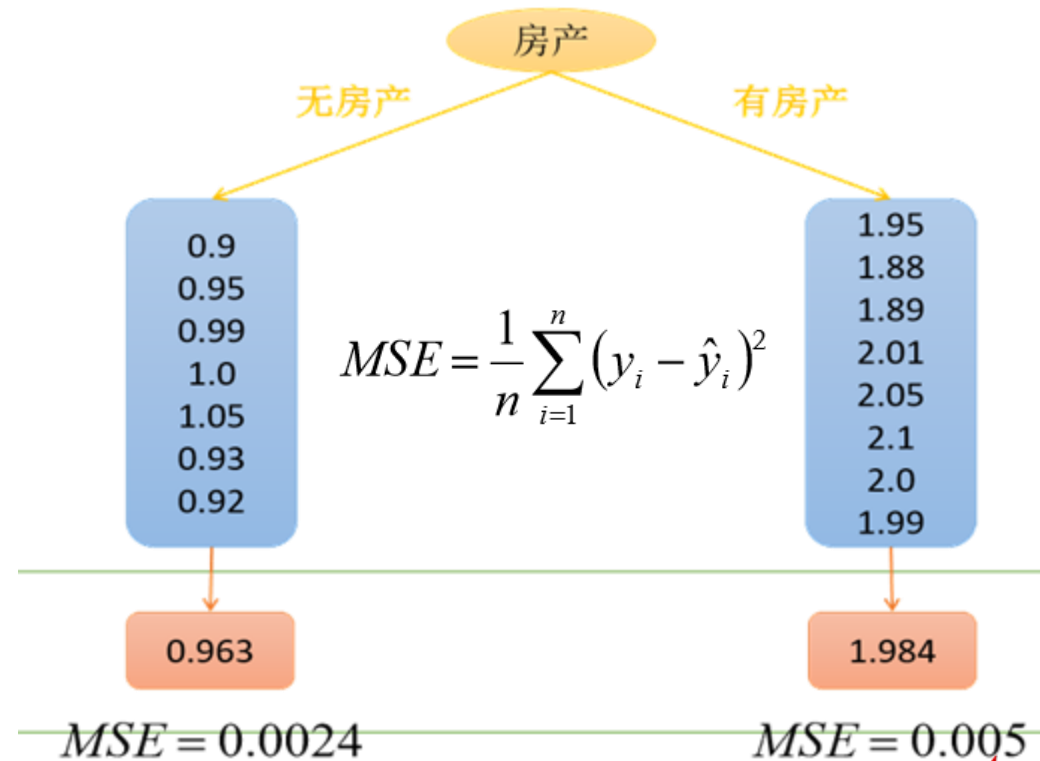
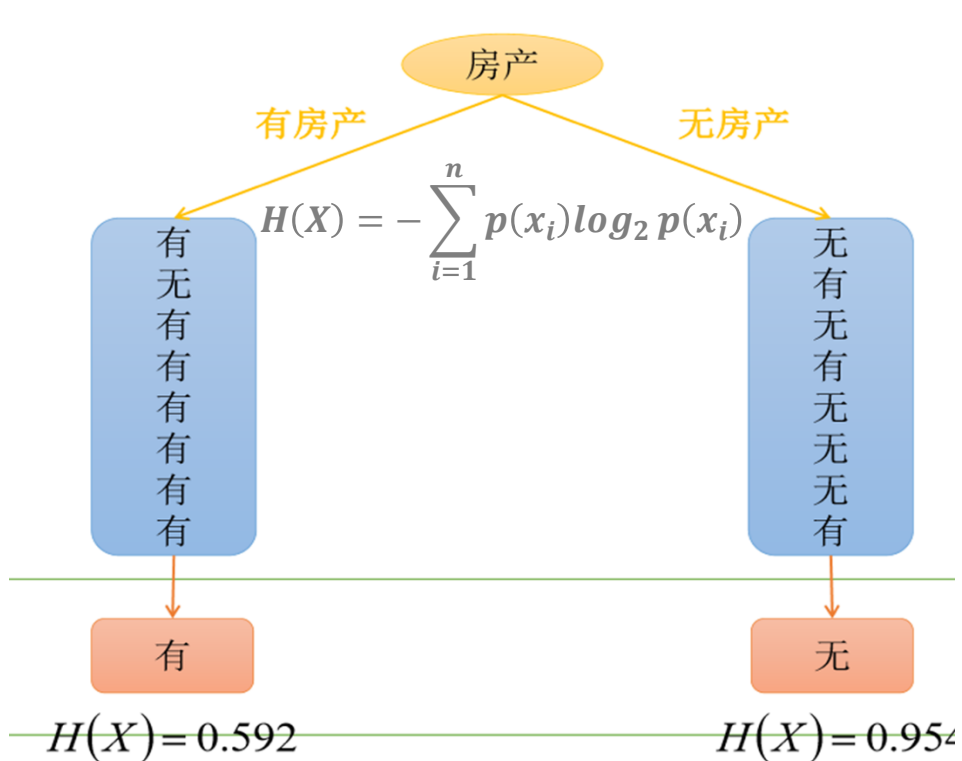
- **分类树** 用 **Gini** 或者 **Entropy** 来作为不纯度的定义
- **回归树** 用的是节点内的 **目标变量y的方差** 来定义。如果节点内的 **方差越大**，说明 **节点越不纯**，反之方差越小节点就越纯。

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

## 决策树可以用于回归问题吗？

$$gini = 1 - \sum_{i=1}^n (p(x_i))^2$$

- **分类树** 用 **Gini** 或者 **Entropy** 来作为不纯度的定义
- **回归树** 用的是节点内的 **目标变量y的方差** 来定义。



## 小结

## 决策树

## 两个不纯度指标

熵  $H(X) = - \sum_{i=1}^n p(x_i) \log_2 p(x_i)$

gini系数  $gini = 1 - \sum_{i=1}^n (p(x_i))^2$

## 三个特征选择指标

ID3 : 信息增益 =  $H(Y) - H(Y|X)$

C4.5 : 信息增益率 =  $(H(Y) - H(Y|X)) / H(X)$

Cart : gini信息增益: gini系数替代entropy

## 剪枝

目的 : 防止过拟合

预剪枝 : 设置条件限制树的生长

后剪枝 : 先尽情生长 , 然后剪枝

## 分类和回归

分类 : entropy或者gini系数

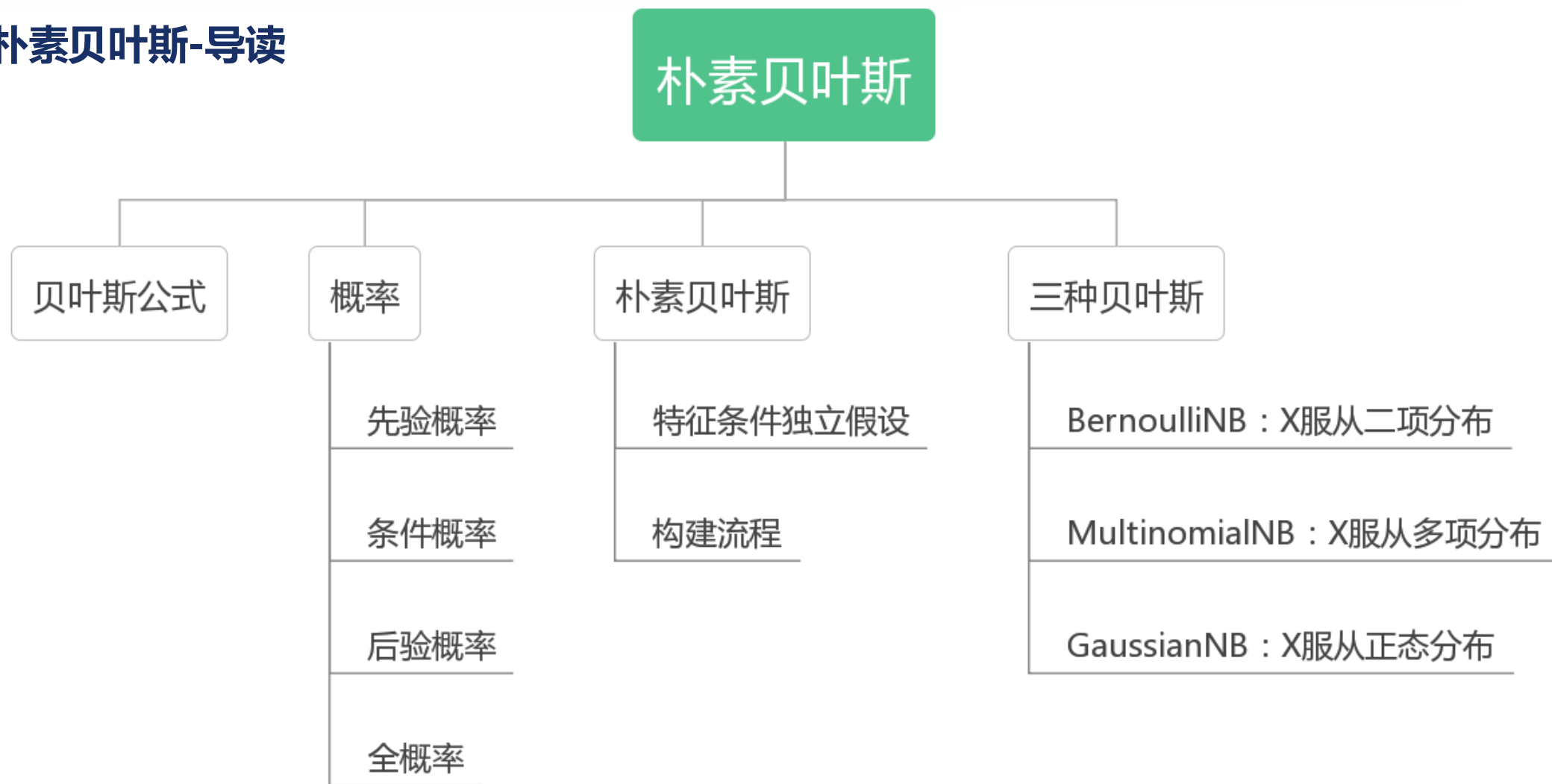
回归 : 均方差MSE  $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$



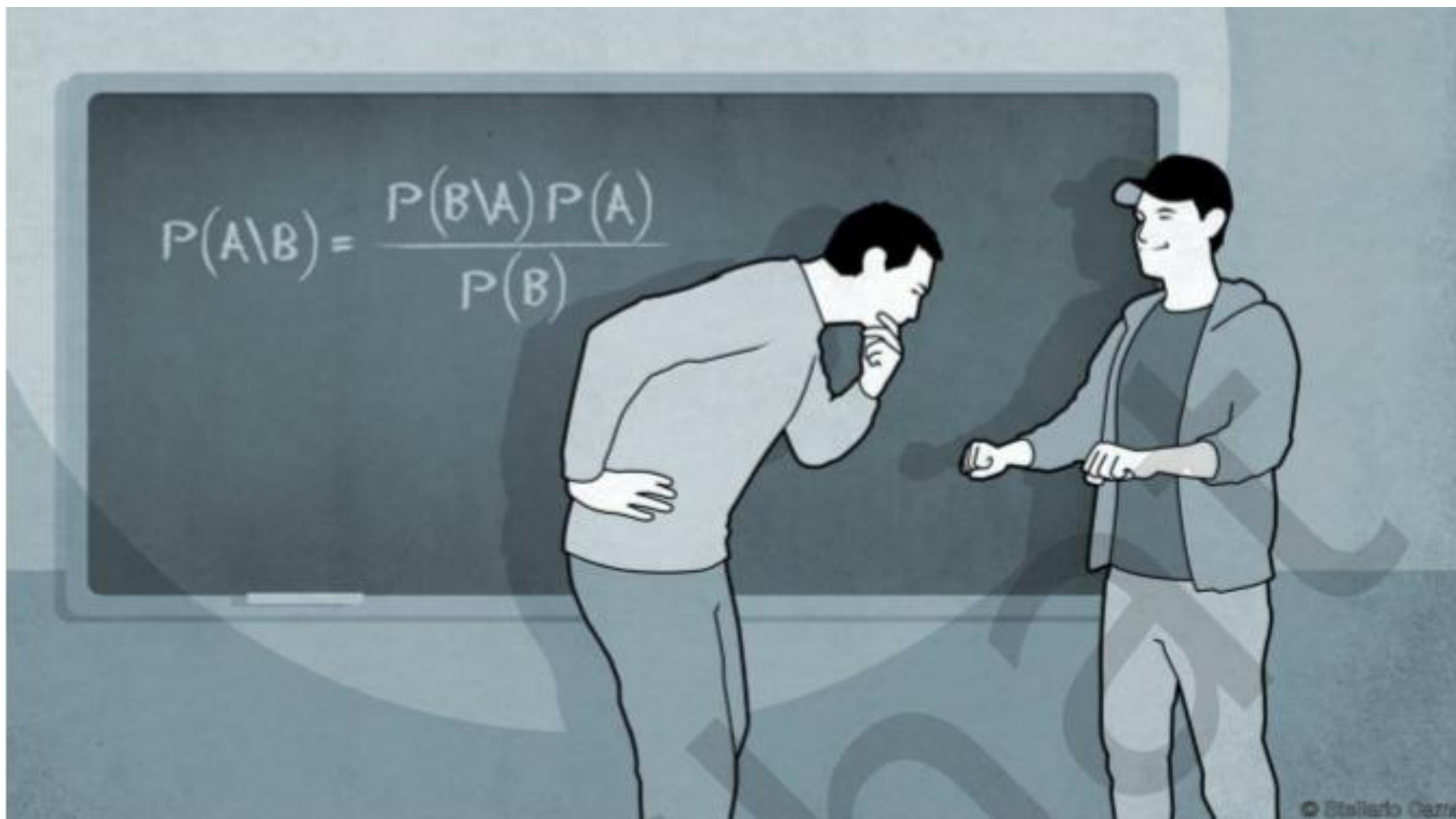
11  
6  
chapter

# 朴素贝叶斯

## 朴素贝叶斯-导读



## 贝叶斯公式





## 概率

- $P(A)$ : A的**先验概率**或**边缘概率**。之所以称为"先验"是因为它不考虑任何B方面的因素
- $P(A|B)$ : 已知B发生后A的**条件概率**，也由于得自B的取值而被称作A的**后验概率**
- $P(B|A)$ : 已知A发生后B的**条件概率**，也由于得自A的取值而被称作B的**后验概率**
- **全概率**: 如果B和B'构成样本空间的一个划分，那么事件A的概率为: B和B'的概率分别乘以A对这两个事件的概率之和。

$$p(B|A) = \frac{p(A|B)p(B)}{p(A)}$$

后验概率

条件概率/似然度

先验概率

$$P(A) = \sum_{i=1}^n P(A, B_i) = \sum_{i=1}^n P(A|B_i)P(B_i)$$

全概率



## 贝叶斯公式计算案例

- 某 AI 公司招聘工程师，来了**8**名应聘者，这**8**个人里，有**5**个人是**985**院校毕业的，另外**3**人**不是**。面试官拿出一道算法题准备考察他们。根据以前的面试经验，面试官知道：**985**毕业生做对这道题的概率是**80%**，**非985**毕业生做对率只有**30%**。
- 现在，面试官从8个人里随手指了一个人——小甲，让 TA 出来做题。结果小甲做对了，**那么请问，小甲是985院校毕业的概率是多大？**

$$\begin{aligned}
 & \text{定义: } x: \{985: 1, \text{非}985: 0\}, y: \{\text{对}: 1, \text{错}: 0\}. \quad P(x=1|y=1) = \frac{P(y=1|x=1) \cdot P(x=1)}{P(y=1)} \\
 & P(x=1) = \frac{5}{8} \quad P(y=1|x=1) = 0.8 \\
 & P(x=0) = \frac{3}{8} \quad P(y=1|x=0) = 0.3 \\
 & \text{求 } P(x=1|y=1)? \\
 & = \frac{P(y=1|x=1) \cdot P(x=1)}{P(y=1|x=1) \cdot P(x=1) + P(y=1|x=0) \cdot P(x=0)} \\
 & = \frac{0.8 \times \frac{5}{8}}{0.8 \times \frac{5}{8} + 0.3 \times \frac{3}{8}} = \frac{40}{49}
 \end{aligned}$$

## 朴素贝叶斯算法原理 - 一个例子

- 我们先来看一个例子，医院早上收了六个门诊病人，记录如下表：

| 症状 <sup>+</sup>  | 职业 <sup>+</sup>   | 疾病 <sup>+</sup>  |
|------------------|-------------------|------------------|
| 打喷嚏 <sup>+</sup> | 护士 <sup>+</sup>   | 感冒 <sup>+</sup>  |
| 打喷嚏 <sup>+</sup> | 农夫 <sup>+</sup>   | 过敏 <sup>+</sup>  |
| 头痛 <sup>+</sup>  | 建筑工人 <sup>+</sup> | 脑震荡 <sup>+</sup> |
| 头痛 <sup>+</sup>  | 建筑工人 <sup>+</sup> | 感冒 <sup>+</sup>  |
| 打喷嚏 <sup>+</sup> | 教师 <sup>+</sup>   | 感冒 <sup>+</sup>  |
| 头痛 <sup>+</sup>  | 教师 <sup>+</sup>   | 脑震荡 <sup>+</sup> |

现在又来了第七个病人，是一个打喷嚏的建筑工人。请问他患上感冒的概率有多大？

## 朴素贝叶斯算法原理-一个栗子

让我们来求助一下贝叶斯公式：

$$P(A|B) = \frac{P(B|A) * P(A)}{P(B)}$$

根据贝叶斯公式，我们可以这样来计算来了一个打喷嚏的建筑工人患感冒的条件概率：

$$P(\text{感冒}|\text{打喷嚏, 建筑工人}) = \frac{P(\text{打喷嚏, 建筑工人}|\text{感冒}) * P(\text{感冒})}{P(\text{打喷嚏, 建筑工人})}$$

假设“打喷嚏”和“建筑工人”是两个独立事件，上面的公式又可写为：

$$P(\text{感冒}|\text{打喷嚏, 建筑工人}) = \frac{P(\text{打喷嚏}|\text{感冒}) * P(\text{建筑工人}|\text{感冒}) * P(\text{感冒})}{P(\text{打喷嚏}) * P(\text{建筑工人})}$$

■ 所以计算打喷嚏的建筑工人患感冒的概率就变成了计算如下的概率：

| 概率                         | 分母 | 分子 | 概率值   |
|----------------------------|----|----|-------|
| $P(\text{打喷嚏} \text{感冒})$  | 3  | 2  | $2/3$ |
| $P(\text{建筑工人} \text{感冒})$ | 3  | 1  | $1/3$ |
| $P(\text{感冒})$             | 6  | 3  | $1/2$ |
| $P(\text{打喷嚏})$            | 6  | 3  | $1/2$ |
| $P(\text{建筑工人})$           | 6  | 2  | $1/3$ |

## 朴素贝叶斯算法原理-一个栗子

| 概率                           | 分母 | 分子 | 概率值   |
|------------------------------|----|----|-------|
| $P(\text{打喷嚏}   \text{感冒})$  | 3  | 2  | $2/3$ |
| $P(\text{建筑工人}   \text{感冒})$ | 3  | 1  | $1/3$ |
| $P(\text{感冒})$               | 6  | 3  | $1/2$ |
| $P(\text{打喷嚏})$              | 6  | 3  | $1/2$ |
| $P(\text{建筑工人})$             | 6  | 2  | $1/3$ |

$$P(\text{感冒} | \text{打喷嚏, 建筑工人}) = \frac{P(\text{打喷嚏} | \text{感冒}) * P(\text{建筑工人} | \text{感冒}) * P(\text{感冒})}{P(\text{打喷嚏}) * P(\text{建筑工人})} = \frac{2/3 * 1/3 * 1/2}{1/2 * 1/3} = 0.667$$

- 同理，可以计算这个病人患**过敏**或者**脑震荡**的概率，比较这几个概率，就可以知道他最可能得什么病。这就是贝叶斯公式进行分类的一个简单案例，它的核心就是利用贝叶斯公式来计算事件发生的条件概率。

## 朴素贝叶斯算法原理：核心原理

假设特征  $x$  之间条件独立  $P(x_i | y, x_1, \dots, x_{i-1}, x_{i+1}, \dots, x_m) = P(x_i | y)$

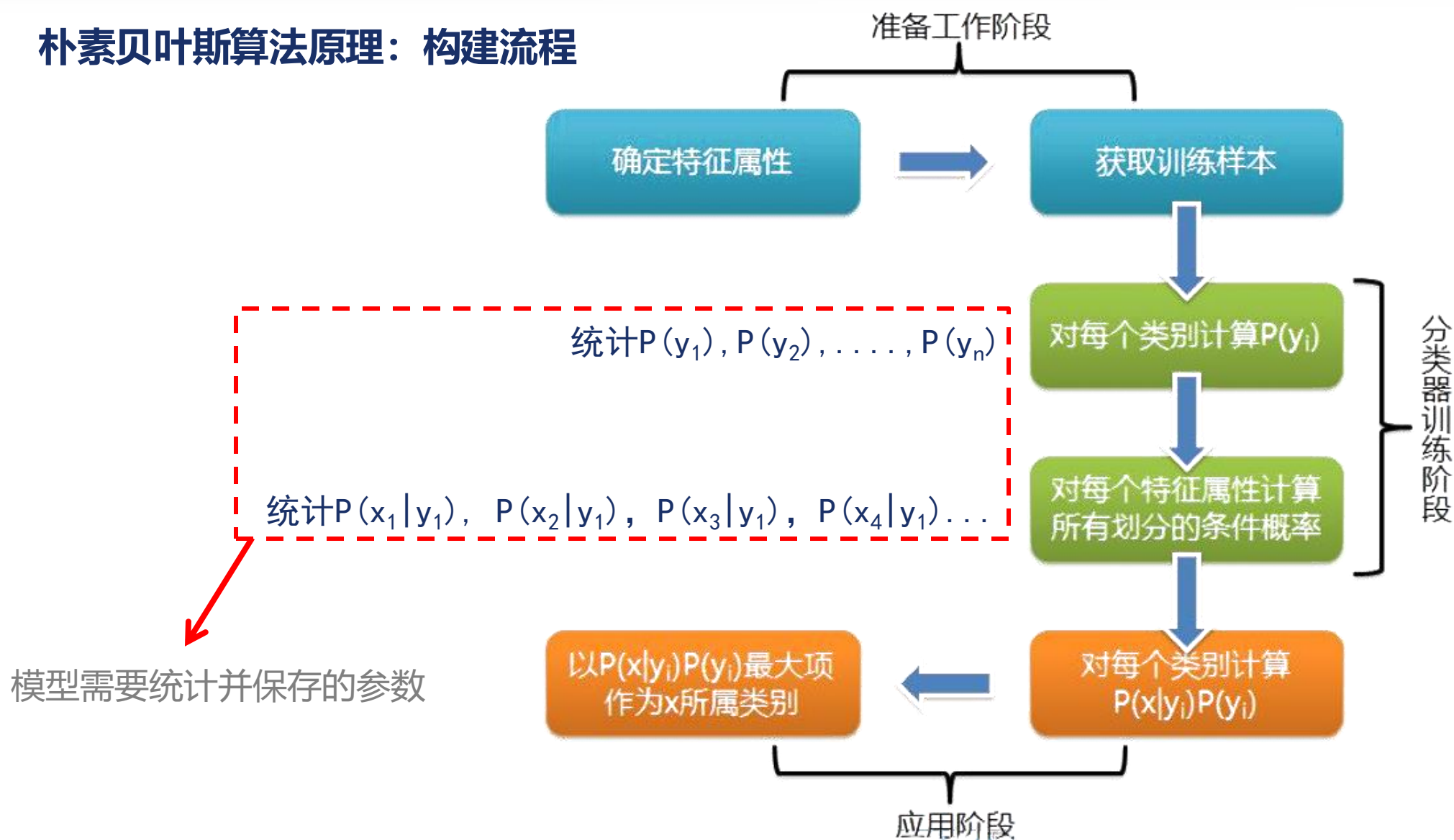
$$P(y | x_1, x_2, \dots, x_m) = \frac{P(y) \boxed{P(x_1, x_2, \dots, x_m | y)}}{P(x_1, x_2, \dots, x_m)} = \frac{P(y) \prod_{i=1}^m P(x_i | y)}{\boxed{P(x_1, x_2, \dots, x_m)}} \propto P(y) \prod_{i=1}^m P(x_i | y)$$

是常数

■ 分别求出**各类别概率**，取**概率最大的类别**作为预测类别：

$$\hat{y} = \arg \max_y P(y) \prod_{i=1}^m P(x_i | y)$$

## 朴素贝叶斯算法原理：构建流程



## 三种贝叶斯：如何计算条件概率 $p(x|y)$

- **GaussianNB**: X变量服从高斯分布（正态分布）。如果样本特征的分布大部分是连续值，使用GaussianNB会比较好。

$$g(x, \eta, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\eta)^2}{2\sigma^2}} \quad P(x_k | y_k) = g(x_k, \eta_{y_k}, \sigma_{y_k})$$

- **MultinomialNB**: X变量服从多项分布。如果样本特征的大部分是多元离散值，使用MultinomialNB比较合适。

$$\theta_{yi} = \frac{N_{yi} + \alpha}{N_y + \alpha * n}$$

拉普拉斯平滑：防止零概率

- **BernoulliNB**: X变量服从二项分布。如果样本特征是二元离散值或者很稀疏的多元离散值，应该使用BernoulliNB。

二项分布：只能是1或者0

$$P(x_k | y) = P(1 | y) \bar{x}_k + (1 - P(1 | y))(1 - x_k)$$



## 小结

- **贝叶斯公式**
- **先验概率、条件概率、后验概率、全概率**
- **朴素**：假设特征之间条件独立
- **模型参数**：所有y类别的先验概率 $p(y)$ 和每个y类别跟每个特征x属性值的条件概率 $p(x|y)$
- 三种贝叶斯：**GaussianNB**（高斯分布）、**MultinomialNB**（多项分布）、**BernoulliNB**（二项分布）
- 优点：算法逻辑简单,易于实现；分类过程中时空开销小
- **缺点**：理论上，朴素贝叶斯模型与其他分类方法相比具有**最小的误差率**。但是实际上并非总是如此，这是因为朴素贝叶斯模型假设属性之间相互独立，这个假设在实际应用中**往往是不成立的**，在属性个数比较多或者属性之间相关性较大时，分类效果不好。

